

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1

HỌC PHẦN PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH



BÁO CÁO ASSIGNMENT 04

**ĐỀ TÀI: CONVOLUTIONAL NEURAL NETWORKS
TỪ TÍCH CHẬP TOÁN HỌC ĐẾN NUMPY, PYTORCH VÀ TENSORFLOW**

Lớp: D23CTPM01

Họ và tên: Nguyễn Duy Nghĩa

Mã sinh viên: B23DCCN600

Giảng viên hướng dẫn: PGS.TS Trần Đình Quế

Học kỳ: Học kỳ 1 năm học 2026 – 2027

Hà Nội, Tháng 09/2026

Mục lục

Chương 1. Cơ sở lý thuyết về mạng nơ-ron tích chập	1
1.1. Vì sao tầng liên kết đầy đủ không đủ cho dữ liệu có cấu trúc	1
1.2. Phép tích chập: định nghĩa, ví dụ số và bản đồ đặc trưng	2
1.3. Đệm viền và bước trượt	4
1.4. Tích chập giãn nở và trường tiếp nhận	5
1.5. Tầng gộp và định tuyến gradient	7
1.6. Vector hoá bằng im2col và khởi tạo He Normal	8
1.7. Kiến trúc như một phép hợp hàm và bài toán quyết định tầng cuối	10
1.8. Softmax, Cross-Entropy và một mẹo số học bắt buộc	12
1.9. Ba cách cài đặt: điều gì đổi, điều gì không	12
1.10. Các hướng kiến trúc phát triển từ tích chập cơ bản	13
1.11. Bộ chỉ số đánh giá dùng xuyên suốt báo cáo	14
Chương 2. Thiết kế thực nghiệm và giao thức đối chuẩn	16
2.1. Ba mức độ kiểm soát khi xây dựng cùng một kiến trúc	16
2.2. Năm miền dữ liệu và ánh xạ sang toán tử tích chập	16
2.3. Tách dữ liệu và phòng rò rỉ	17
2.4. Cấu hình kiến trúc giữ cố định giữa ba khung	18
2.5. Vòng lặp huấn luyện viết tay	18
2.6. Nguyên tắc đọc kết quả	19
Chương 3. Tích chập một chiều trên chuỗi văn bản	21
3.1. Dữ liệu và quy trình tiền xử lý chuỗi	21
3.2. Kiến trúc và vai trò của gộp cục đại toàn cục	22
3.3. Hiện thực tích chập một chiều bằng NumPy thuần	22
3.4. Diễn biến huấn luyện	23
3.5. Kết quả đối chuẩn ba khung	24
3.6. Ma trận nhầm lẫn và cấu trúc lỗi	25
Chương 4. Tích chập một chiều trên dữ liệu bảng y tế	27
4.1. Dữ liệu và tám đặc trưng lâm sàng	27
4.2. Kiến trúc và luồng biến đổi kích thước tensor	27
4.3. Diễn biến huấn luyện	28
4.4. Kết quả đối chuẩn ba khung	28
4.5. Ma trận nhầm lẫn và cái giá của âm tính giả	30

4.6. Thực nghiệm hoán vị cột: kiểm chứng cảnh báo phương pháp luận	30
Chương 5. Tích chập một chiều cho bài toán hồi quy	32
5.1. Dữ liệu, lọc ngoại lai và tám đặc trưng kỹ nghệ	32
5.2. Vì sao huấn luyện trên thang logarit	33
5.3. Diễn biến huấn luyện	33
5.4. Kết quả đối chuẩn ba khung	34
5.5. Đồ thị đối chiếu giá trị thực và giá trị dự đoán	35
Chương 6. Tích chập hai chiều trên ảnh chữ số viết tay	36
6.1. Khảo sát dữ liệu	36
6.2. Cấu hình huấn luyện và một sai lệch được công bố trước	38
6.3. Hiện thực tích chập hai chiều bằng NumPy thuần	38
6.4. Kiểm chứng đạo hàm bằng sai phân hữu hạn	40
6.5. Mô hình cơ sở đối chiếu mô hình cải tiến	41
6.6. Đối chuẩn với PyTorch và TensorFlow	43
6.7. Độ khó theo từng lớp và phân tích lỗi tự tin cao	45
Chương 7. Tích chập hai chiều trên ảnh màu tự nhiên	47
7.1. Khảo sát dữ liệu	47
7.2. Cấu hình huấn luyện	49
7.3. Kết quả mô hình thuần NumPy	50
7.4. Đối chuẩn với PyTorch và TensorFlow	52
7.5. Độ khó theo lớp và cấu trúc nhằm lẫn ngữ nghĩa	54
Chương 8. Thực nghiệm đối kháng giữa mạng truyền thẳng và mạng tích chập ...	56
8.1. Thiết kế thực nghiệm có kiểm soát và giả thuyết nghiên cứu	56
8.2. Kết quả đối đầu trực diện	56
8.3. Lý giải dưới góc độ lý thuyết học máy	59
8.4. Trực quan hoá không gian biểu diễn ẩn bằng PCA	59
8.5. Cấu trúc ngữ nghĩa xuất hiện mà không được dạy	61
Chương 10. Mô hình đã học được gì và có đáng tin không	63
10.1. Hiệu chỉnh xác suất và sai số hiệu chỉnh kỳ vọng	63
10.2. Bộ lọc tầng tích chập thứ nhất	65
10.3. Biểu diễn thay đổi qua chiều sâu	67
10.4. Mô hình thực sự nhìn vào đâu	67
Chương 11. Bóc tách những gì các chương trước phải gộp chung	70

11.1. Thiết kế giai thừa đầy đủ cho gói cải tiến	70
11.2. Đường cong hiệu năng theo cỡ dữ liệu huấn luyện	72
11.3. Tách phần do dữ liệu khỏi phần còn lại	72
Chương 12. Tổng hợp đối chuẩn, giới hạn và kết luận	75
12.1. Bảng tổng hợp toàn bộ thực nghiệm	75
12.2. So sánh ba cách cài đặt	76
12.3. Tính công bằng và giới hạn của phép đối chuẩn	76
12.4. Kiểm toán khả năng tái lập	77
12.5. Kết luận	78
12.6. Hướng phát triển tiếp theo	79

Tài nguyên và mã nguồn dự án

Mọi con số, bảng biểu và hình vẽ trong báo cáo này đều được sinh ra từ một lần chạy thật của các notebook liệt kê bên dưới. Không có giá trị nào được chép tay vào văn bản. Toàn bộ mã nguồn, dữ liệu và bản báo cáo này được quản lý phiên bản công khai.

Kho lưu trữ mã nguồn

Hạng mục	Đường dẫn
Kho lưu trữ GitHub của dự án	https://github.com/Nghaiz/Intelligent-System-Development
Thư mục bài tập số 4	https://github.com/Nghaiz/Intelligent-System-Development/tree/main/src/Assignment%2004
Báo cáo nghiên cứu (tệp này)	Report/Assignment 04/A04_CT_nghiand.600.pdf
Hợp đồng tích hợp giữa các notebook	src/Assignment 04/CONTRACT.md
Hướng dẫn cài đặt và chạy lại	src/Assignment 04/README.md
Danh sách phụ thuộc	src/Assignment 04/requirements.txt

Năm bộ dữ liệu thực nghiệm

Sử dụng ở	Bộ dữ liệu	Nguồn công khai
Chương 3 — Chuỗi văn bản	Women's Clothing E-Commerce Reviews (23.486 đánh giá)	https://www.kaggle.com/datasets/nicapotato/womens-ecommerce-clothing-reviews
Chương 4 — Bảng phân loại	Diabetes Prediction Dataset (100.000 hồ sơ lâm sàng)	https://www.kaggle.com/datasets/iammustafatz/diabetes-prediction-dataset
Chương 5 — Bảng hồi quy	USA Real Estate Dataset (150.000 giao dịch)	https://www.kaggle.com/datasets/ahmedshahriarsakib/usa-real-estate-dataset
Chương 6 — Ảnh xám	MNIST Handwritten Digits (70.000 ảnh $28 \times 28 \times 1$)	https://www.kaggle.com/datasets/hojjatk/mnist-dataset
Chương 7 — Ảnh màu	CIFAR-10 Natural Images (60.000 ảnh $32 \times 32 \times 3$)	https://www.cs.toronto.edu/~kriz/cifar.html

Tổng cộng **403,486 bản ghi** trải trên ba dạng biểu diễn: chuỗi ký tự, bảng số và ảnh raster.

Mười sáu notebook thực nghiệm

Notebook	Đường dẫn trong kho mã nguồn
Chuỗi văn bản — 1D CNN thuần NumPy	<code>customer_comments/notebooks/01_comments_1d_cnn_numpy.ipynb</code>
Chuỗi văn bản — 1D CNN PyTorch	<code>customer_comments/notebooks/02_comments_1d_cnn_pytorch.ipynb</code>
Chuỗi văn bản — 1D CNN TensorFlow/Keras	<code>customer_comments/notebooks/03_comments_1d_cnn_tensorflow.ipynb</code>
Bảng y tế — 1D CNN thuần NumPy	<code>diabetes/notebooks/01_diabetes_1d_cnn_numpy.ipynb</code>
Bảng y tế — 1D CNN PyTorch	<code>diabetes/notebooks/02_diabetes_1d_cnn_pytorch.ipynb</code>
Bảng y tế — 1D CNN TensorFlow/Keras	<code>diabetes/notebooks/03_diabetes_1d_cnn_tensorflow.ipynb</code>
Bất động sản — 1D CNN hồi quy thuần NumPy	<code>house_price/notebooks/01_house_price_1d_cnn_numpy.ipynb</code>
Bất động sản — 1D CNN hồi quy PyTorch	<code>house_price/notebooks/02_house_price_1d_cnn_pytorch.ipynb</code>
Bất động sản — 1D CNN hồi quy TensorFlow/Keras	<code>house_price/notebooks/03_house_price_1d_cnn_tensorflow.ipynb</code>
MNIST — Khảo sát dữ liệu	<code>mnist/notebooks/00_mnist_visualization.ipynb</code>
MNIST — 2D CNN thuần NumPy (cơ sở và cải tiến)	<code>mnist/notebooks/01_mnist_cnn_scratch.ipynb</code>
MNIST — Đối chuẩn PyTorch và Keras	<code>mnist/notebooks/02_mnist_frameworks_comparison.ipynb</code>
CIFAR-10 — Khảo sát dữ liệu	<code>cifar10/notebooks/00_cifar10_visualization.ipynb</code>

Notebook	Đường dẫn trong kho mã nguồn
CIFAR-10 — 2D CNN thuần NumPy (cơ sở và cải tiến)	cifar10/notebooks/01_cifar10_cnn_scratch.ipynb
CIFAR-10 — Đối chuẩn PyTorch và Keras	cifar10/notebooks/02_cifar10_frameworks_comparison.ipynb
Đối kháng MLP với CNN và phân tích không gian ẩn PCA	mlp_vs_cnn/notebooks/03_mlp_vs_cnn_pca.ipynb

Đường dẫn tính từ thư mục `src/Assignment 04/`. Mỗi notebook tự nạp dữ liệu, tự huấn luyện, tự vẽ hình và tự ghi tệp JSON kết quả vào `<miền>/reports/`. Bộ dựng báo cáo `Report/Assignment 04/build_report.py` chỉ đọc lại các tệp JSON đó.

Chương 1. Cơ sở lý thuyết về mạng nơ-ron tích chập

Bài tập số 3 dừng lại ở mạng truyền thẳng: một chuỗi phép nhân ma trận xen kẽ hàm phi tuyến, trong đó mỗi nơ-ron nhìn thấy toàn bộ vector đầu vào. Bài tập số 4 đặt ra một câu hỏi hẹp hơn nhưng sâu hơn: *có thể thiết kế một tầng mạng chuyên đi tìm những hoa văn cục bộ hay không?* Câu trả lời là phép tích chập.

Ràng buộc trung tâm của báo cáo vẫn giữ nguyên tinh thần của bài trước: **mọi kiến trúc tích chập đều được hiện thực trước bằng NumPy thuần** — tự viết cửa sổ trượt, tự viết đạo hàm theo bộ lọc, tự viết định tuyến gradient qua tầng gộp, tự viết Adam — rồi mới dựng lại đúng kiến trúc đó bằng PyTorch và TensorFlow để đối chiếu. Cách làm này biến khung thư viện từ một chiếc hộp đen thành một lớp tăng tốc cho công thức mà ta đã tự tay kiểm chứng.

1.1. Vì sao tầng liên kết đầy đủ không đủ cho dữ liệu có cấu trúc

Một tầng Dense tính $H = \sigma(XW + b)$. Công thức này rất tổng quát, và chính sự tổng quát ấy là vấn đề khi dữ liệu mang cấu trúc hình học.

Xét một bức ảnh màu $224 \times 224 \times 3$. Làm phẳng nó thành vector cho ra 150.528 số. Nói vector đó với một tầng ẩn 1.000 nơ-ron cần hơn 150 triệu trọng số chỉ cho riêng tầng đầu tiên. Ba hệ quả lập tức xuất hiện:

1. **Bùng nổ tham số.** Số trọng số tỉ lệ thuận với độ phân giải. Tăng ảnh lên gấp đôi mỗi chiều làm tham số tăng gấp bốn, kéo theo quá khớp và chi phí bộ nhớ.
2. **Phá huỷ quan hệ lân cận.** Thao tác làm phẳng coi mọi điểm ảnh là độc lập và hoán vị được. Nhưng điểm ảnh tại (i, j) rõ ràng có quan hệ với $(i \pm 1, j)$ và $(i, j \pm 1)$; mạng buộc phải học lại quan hệ đó từ đầu thay vì được cho sẵn.
3. **Không có tính bất biến vị trí.** Nếu một vật thể dịch sang phải 5 điểm ảnh, toàn bộ tập trọng số đã học cho vị trí cũ trở nên vô dụng. Mạng phải học lại cùng một hoa văn ở từng vị trí khả dĩ.

Mạng tích chập giải quyết cả ba bằng hai nguyên lý duy nhất: **liên kết cục bộ** (mỗi nơ-ron chỉ nhìn một cửa sổ nhỏ) và **chia sẻ trọng số** (cùng một bộ lọc được dùng lại ở mọi vị trí). Số tham số khi đó chỉ phụ thuộc kích thước bộ lọc và số kênh, hoàn toàn độc lập với kích thước ảnh.

$$\text{CNN} = \text{Cục bộ} + \text{Chia sẻ trọng số} + \text{Phi tuyến}$$

1.2. Phép tích chập: định nghĩa, ví dụ số và bản đồ đặc trưng

Với bộ lọc kích thước $K_h \times K_w$, bước trượt s và đệm viền P , phần tử tại vị trí (i, j) của bản đồ đặc trưng thứ k được tính bằng một tổng có trọng số trên một khối con của đầu vào:

$$Y_{i,j,k} = b_k + \sum_u \sum_v \sum_c X_{is+u-P, js+v-P, c} \cdot W_{u,v,c,k}$$

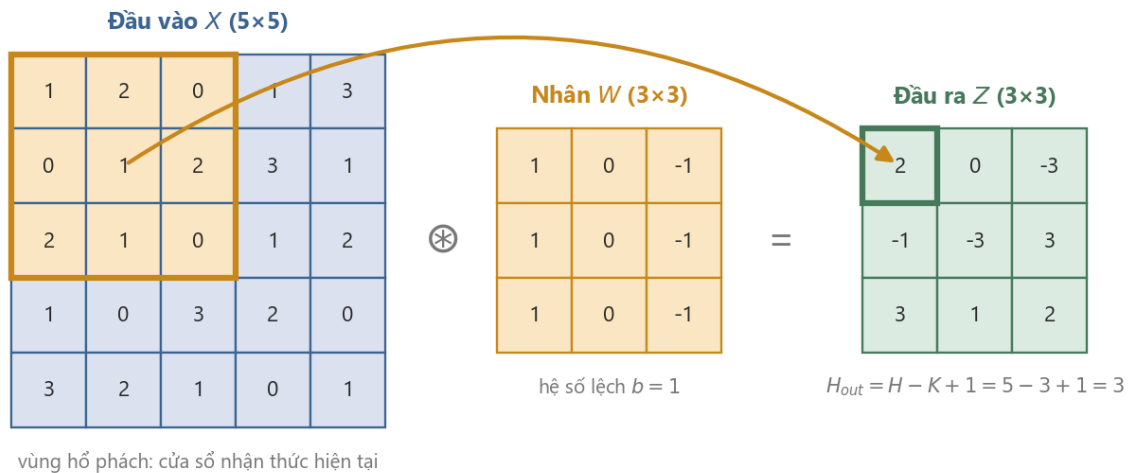
Kích thước không gian đầu ra suy ra trực tiếp từ hình học của sổ trượt:

$$H_{out} = \lfloor (H + 2P - d(K-1) - 1) / s \rfloor + 1$$

trong đó d là độ giãn nở, sẽ bàn ở mục 1.4. Với cấu hình thông dụng $d = 1$, công thức rút gọn thành $\lfloor (H + 2P - K) / s \rfloor + 1$.

Phép tích chập 2 chiều: cửa sổ trượt, nhân từng phần tử rồi cộng dồn

một cửa sổ 3×3 của $X \rightarrow$ đúng một ô của Z



Nhân từng phần tử rồi cộng dồn:

$$\begin{aligned}
 Z &= 1 \cdot 1 + 2 \cdot 0 + 0 \cdot (-1) \\
 &+ 0 \cdot 1 + 1 \cdot 0 + 2 \cdot (-1) \\
 &+ 2 \cdot 1 + 1 \cdot 0 + 0 \cdot (-1) \\
 &+ b = 1 + 1 = 2
 \end{aligned}$$

$$Z = \sum_p \sum_q X_{p,q} W_{p,q} + b$$

Nhân trượt trên toàn ảnh với cùng một bộ trọng số
 $\rightarrow$ chia sẻ tham số và bất biến tịnh tiến.

Hình 1. Bản chất tính toán của phép tích chập hai chiều: tích vô hướng từng phần tử giữa cửa sổ tiếp nhận cục bộ và bộ lọc trọng số, cộng thêm độ chệch.

Một ví dụ số một chiều làm rõ cơ chế. Cho $x = [2, 1, 3, 4, 2]$ và bộ lọc $k = [0,5; -1; 0,5]$ với $b = 0$. Tại vị trí đầu tiên, cửa sổ là $[2, 1, 3]$:

$$z_1 = 2(0,5) + 1(-1) + 3(0,5) = 1$$

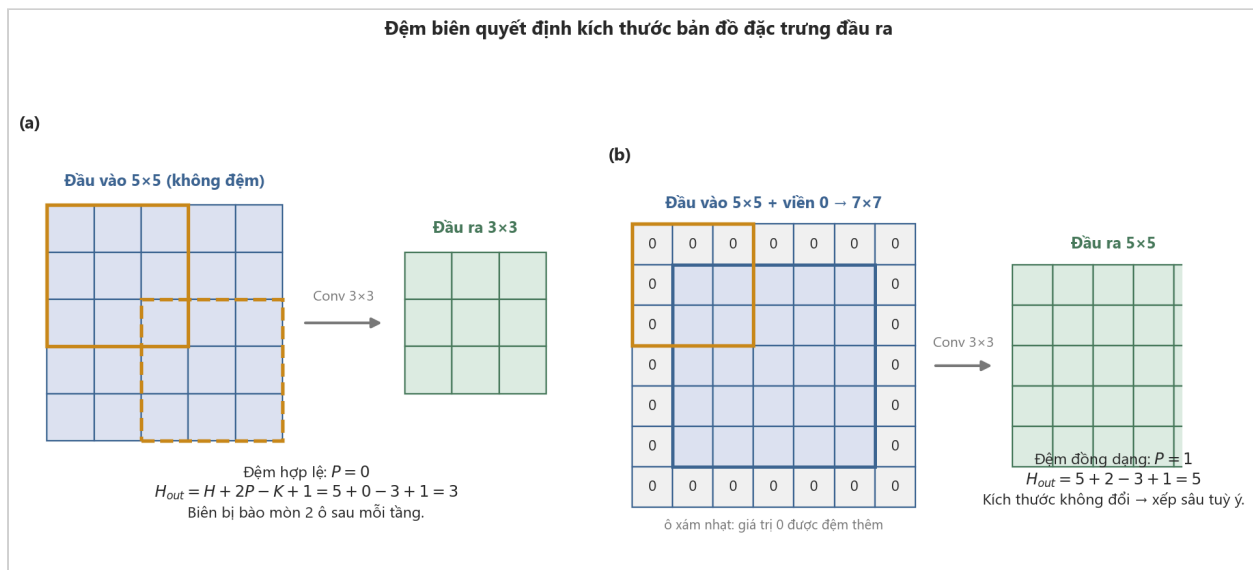
Trượt sang phải một bước, cửa sổ thành $[1, 3, 4]$ và $z_2 = 1(0,5) + 3(-1) + 4(0,5) = -0,5$. Tiếp tục cho $z_3 = -1,5$. Kết quả đầy đủ là $[1; -0,5; -1,5]$. Điểm mấu chốt: cả ba giá trị dùng cùng một bộ ba trọng số. Đó chính là chia sẻ trọng số, và nó là lý do một bộ lọc học được khái niệm "cạnh dọc" chứ không phải "cạnh dọc ở cột thứ 12".

Ví dụ số này được lặp lại bên trong mọi notebook NumPy của báo cáo như một phép kiểm tra tính đúng đắn: mã nguồn phải tái tạo đúng ba con số trên trước khi được phép huấn luyện bất cứ thứ gì.

Một lưu ý thuật ngữ đáng nhắc. Phép tích chập trong giải tích lật ngược bộ lọc trước khi nhân chập để bảo đảm tính giao hoán. Các thư viện học sâu bỏ bước lật đó, tức là thực chất tính *tương quan chéo*. Vì trọng số được học tự do từ dữ liệu, một bộ lọc lật ngược cũng học được không kém, nên năng lực biểu diễn không đổi và cộng đồng quy ước gọi chung là tích chập.

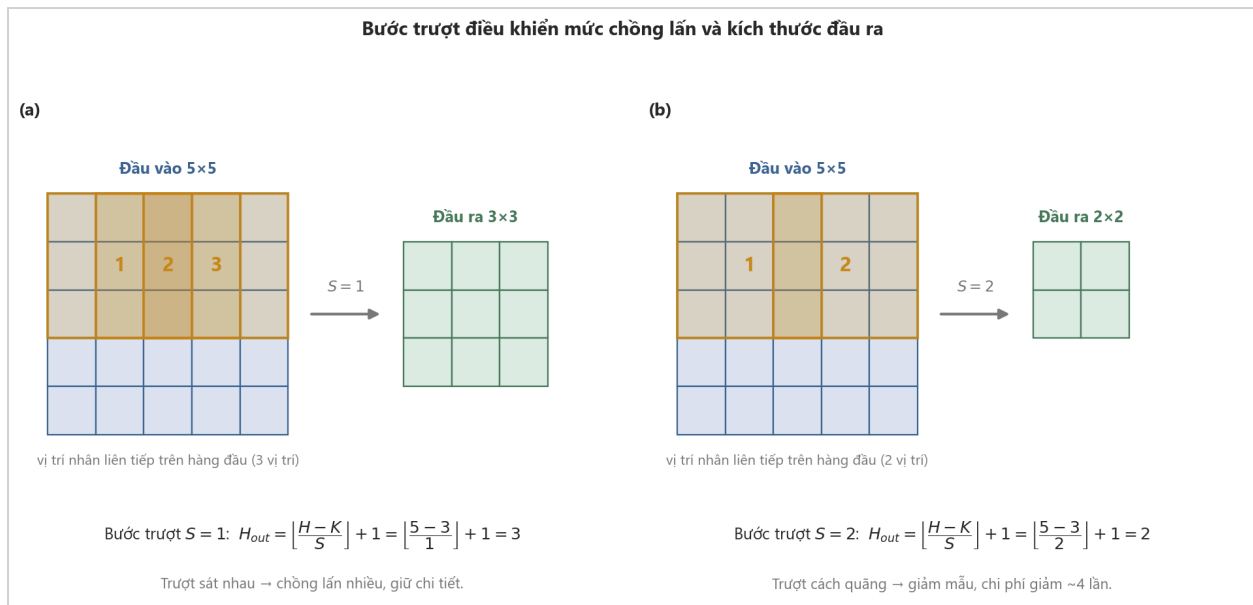
1.3. Đệm viền và bước trượt

Không đệm viền, mỗi tầng tích chập 3×3 làm ảnh co lại hai điểm ảnh mỗi chiều. Xếp chồng mười tầng là mất hai mươi điểm ảnh, đủ để xóa sạch một ảnh 28×28 . Đệm viền bằng 0 với $P = 1$ giữ nguyên kích thước và cho phép dựng mạng sâu tùy ý. Nó còn sửa một khiếm khuyết tinh tế hơn: khi không đệm, điểm ảnh ở góc chỉ tham gia đúng một cửa sổ, trong khi điểm ảnh ở giữa tham gia chín cửa sổ, nên thông tin biên bị đánh giá thấp một cách có hệ thống.



Hình 2. Đệm viền: (a) không đệm làm suy giảm độ phân giải sau mỗi tầng; (b) đệm đối xứng $P = 1$ với bộ lọc 3×3 bảo toàn nguyên vẹn kích thước.

Bước trượt đóng vai trò ngược lại. Với $s = 1$, bộ lọc quét từng điểm ảnh và cho bản đồ đặc trưng mịn nhất. Với $s = 2$, bộ lọc nhảy cách một điểm ảnh, giảm một nửa độ phân giải. So với tầng gộp, bước trượt lớn thực hiện cùng chức năng nén nhưng phép nén đó có tham số học được.



Hình 3. Bước trượt: (a) $s = 1$ quét liên tục từng điểm ảnh; (b) $s = 2$ nhảy hai điểm ảnh mỗi lần và thực hiện giảm độ phân giải.

1.4. Tích chập giãn nở và trường tiếp nhận

Tích chập giãn nở chèn khoảng trống vào giữa các phần tử của bộ lọc. Một bộ lọc 3×3 với $d = 2$ bao phủ một vùng rộng 5×5 trên đầu vào nhưng vẫn chỉ mang đúng chín trọng số. Đây là cách mở rộng tầm nhìn theo cấp số nhân mà không trả thêm một tham số nào.

Giãn nở mở rộng vùng nhận thức với cùng số trọng số

(a)

Giãn nở $d = 1 \rightarrow$ tầm phủ 3×3

w_{11}	w_{12}	w_{13}		
w_{21}	w_{22}	w_{23}		
w_{31}	w_{32}	w_{33}		

ô hồ phách: 9 trọng số · dấu × : ô bị bỏ qua

$$K' = d(K - 1) + 1 = 1 \cdot 2 + 1 = 3$$

Số trọng số vẫn là $3 \times 3 = 9$

Nhân 3×3 chuẩn: các ô liền kề.

(b)

Giãn nở $d = 2 \rightarrow$ tầm phủ 5×5

w_{11}	×	w_{12}	×	w_{13}
×	×	×	×	×
w_{21}	×	w_{22}	×	w_{23}
×	×	×	×	×
w_{31}	×	w_{32}	×	w_{33}

ô hồ phách: 9 trọng số · dấu × : ô bị bỏ qua

$$K' = d(K - 1) + 1 = 2 \cdot 2 + 1 = 5$$

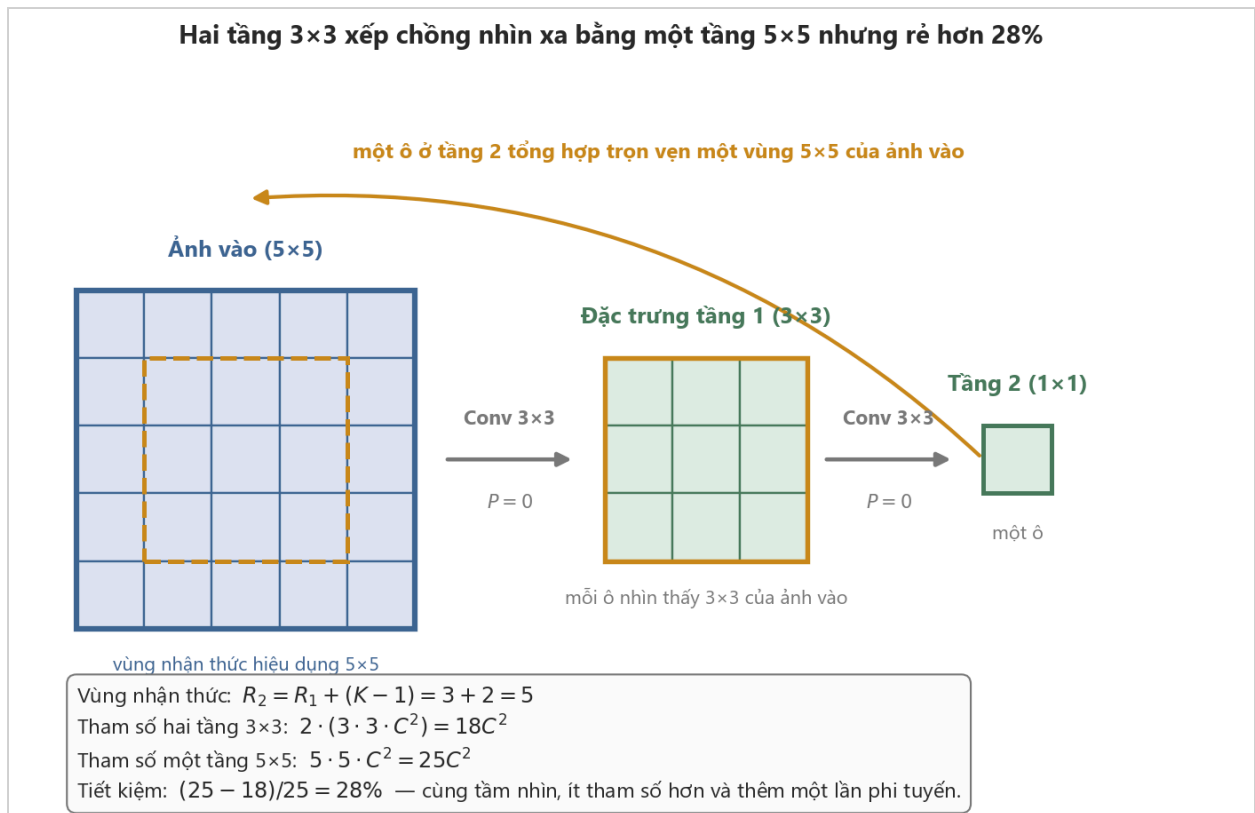
Số trọng số vẫn là $3 \times 3 = 9$

Nhân giãn nở: phủ rộng hơn mà không thêm tham số.

Hình 4. Tích chập giãn nở: (a) bộ lọc chuẩn 3×3 với $d = 1$; (b) bộ lọc giãn nở $d = 2$ mở rộng vùng bao phủ lên 5×5 với đúng 9 trọng số.

Trường tiếp nhận là vùng ảnh gốc chi phối một nơ-ron ở tầng sâu. Tầng tích chập 3×3 đầu tiên cho trường tiếp nhận bằng 3. Tầng 3×3 thứ hai tổng hợp một vùng 3×3 của tầng một, nên gián tiếp bao phủ 5×5 trên ảnh gốc. Điều này dẫn tới một lựa chọn thiết kế đã thành chuẩn mực kể từ VGG: **thay một tầng 5×5 bằng hai tầng 3×3 xếp chồng**.

Lý do là hai lợi ích cùng lúc. Về tham số, một tầng 5×5 với C kênh cần $25C^2$ trọng số, trong khi hai tầng 3×3 chỉ cần $18C^2$, tiết kiệm 28%. Về biểu diễn, hai tầng cho phép chèn thêm một hàm ReLU ở giữa, nên hàm tổng hợp phi tuyến hơn hẳn một tầng đơn lẻ có cùng trường tiếp nhận.

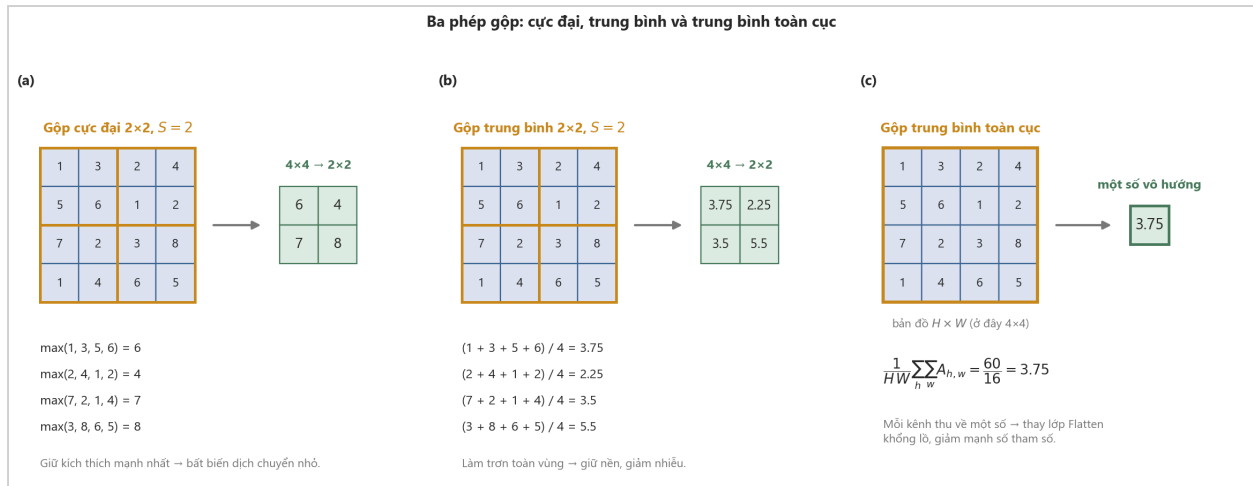


Hình 5. Trường tiếp nhận hiệu dụng: hai tầng tích chập 3×3 liên tiếp bao phủ vùng 5×5 trên ảnh gốc với số tham số ít hơn 28% so với một tầng 5×5 .

Cần phân biệt hai tính chất thường bị nhập làm một. **Tương đương tịnh tiến** là tính chất của tầng tích chập: ảnh dịch đi bao nhiêu thì bản đồ đặc trưng dịch đi đúng bấy nhiêu. **Bất biến tịnh tiến** là tính chất của tầng gộp: ảnh xê dịch nhẹ nhưng giá trị sau khi gộp không đổi. Mạng cần cả hai, và chúng đến từ hai tầng khác nhau.

1.5. Tầng gộp và định tuyến gradient

Gộp cực đại chọn giá trị kích hoạt lớn nhất trong mỗi cửa sổ, chẳng hạn $[1, 5, 2, 4]$ với cửa sổ 2 cho ra $[5, 4]$. Gộp trung bình lấy trung bình cộng và làm mịn tín hiệu. Gộp trung bình toàn cục nén trọn một bản đồ đặc trưng $H \times W$ thành một số vô hướng, qua đó thay thế hoàn toàn tầng Flatten và cắt bỏ phần lớn tham số ở cuối mạng.



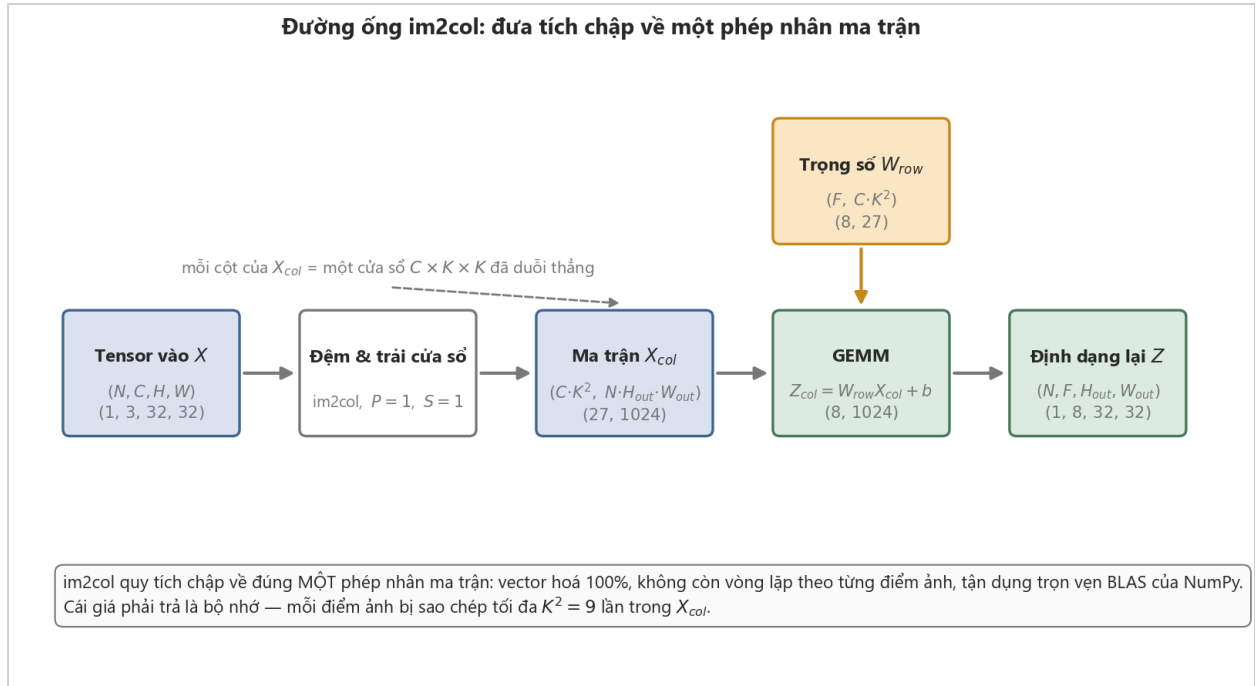
Hình 6. Ba cơ chế gộp đặc trưng: (a) gộp cực đại giữ phản hồi mạnh nhất; (b) gộp trung bình làm mịn tín hiệu; (c) gộp trung bình toàn cục nén mỗi kênh thành một giá trị vô hướng.

Phần thú vị nằm ở chiều ngược. Trong lan truyền tiến, gộp cực đại ghi nhớ chỉ số của phần tử thắng cuộc. Trong lan truyền ngược, gradient từ tầng sau đi trọn vẹn về đúng phần tử đó, mọi phần tử còn lại trong cửa sổ nhận gradient bằng 0. Gộp trung bình thì ngược lại: mọi phần tử đóng góp như nhau nên gradient được chia đều. Hiện thực NumPy trong báo cáo lưu một mặt nạ cực đại ở lượt tiến để thực hiện chính xác phép định tuyến này.

1.6. Vector hoá bằng im2col và khởi tạo He Normal

Viết tích chập bằng bốn vòng lặp Python lồng nhau thì đúng về toán nhưng chậm tới mức không thể huấn luyện nổi. Giải pháp kinh điển là im2col: trải phẳng từng cửa sổ trượt thành một hàng của ma trận X_{col} , dàn phẳng bộ lọc thành W_{row} , rồi quy toàn bộ phép tích chập trên cả mini-batch về một phép nhân ma trận duy nhất:

$$Z_{col} = X_{col} W_{row} + b$$



Hình 7. Quy trình vector hoá phép tích chập bằng im2col và phép nhân ma trận tổng quát GEMM.

Cấu trúc ma trận này khiến lan truyền ngược trở nên gọn gàng. Đạo hàm theo trọng số và theo đầu vào đều là phép nhân ma trận chuyển vị:

$$\partial L / \partial W_{\text{row}} = X_{\text{col}}^T \partial L / \partial Z_{\text{col}} \quad \partial L / \partial X_{\text{col}} = \partial L / \partial Z_{\text{col}} W_{\text{row}}^T$$

Hàm col2im đảo ngược thao tác trải phẳng và cộng dồn gradient tại những vùng cửa sổ chồng lấn lên nhau. Phép cộng dồn này không phải chi tiết kỹ thuật tùy chọn mà là hệ quả trực tiếp của chia sẻ trọng số: một trọng số đã tham gia vào nhiều vị trí thì phải nhận gradient từ tất cả những vị trí đó.

Khởi tạo trọng số là mất xích cuối cùng. Vì ReLU đặt một nửa tín hiệu về 0, phương sai kích hoạt bị giảm khoảng một nửa sau mỗi tầng. Dùng khởi tạo chuẩn thông thường thì tín hiệu tắt dần về 0 chỉ sau vài tầng. He Normal bù chính xác phần hụt đó bằng cách nhân đôi phương sai khởi tạo:

$$\text{Var}(W) = 2 / n_{\text{in}} = 2 / (C_{\text{in}} \cdot K_h \cdot K_w)$$

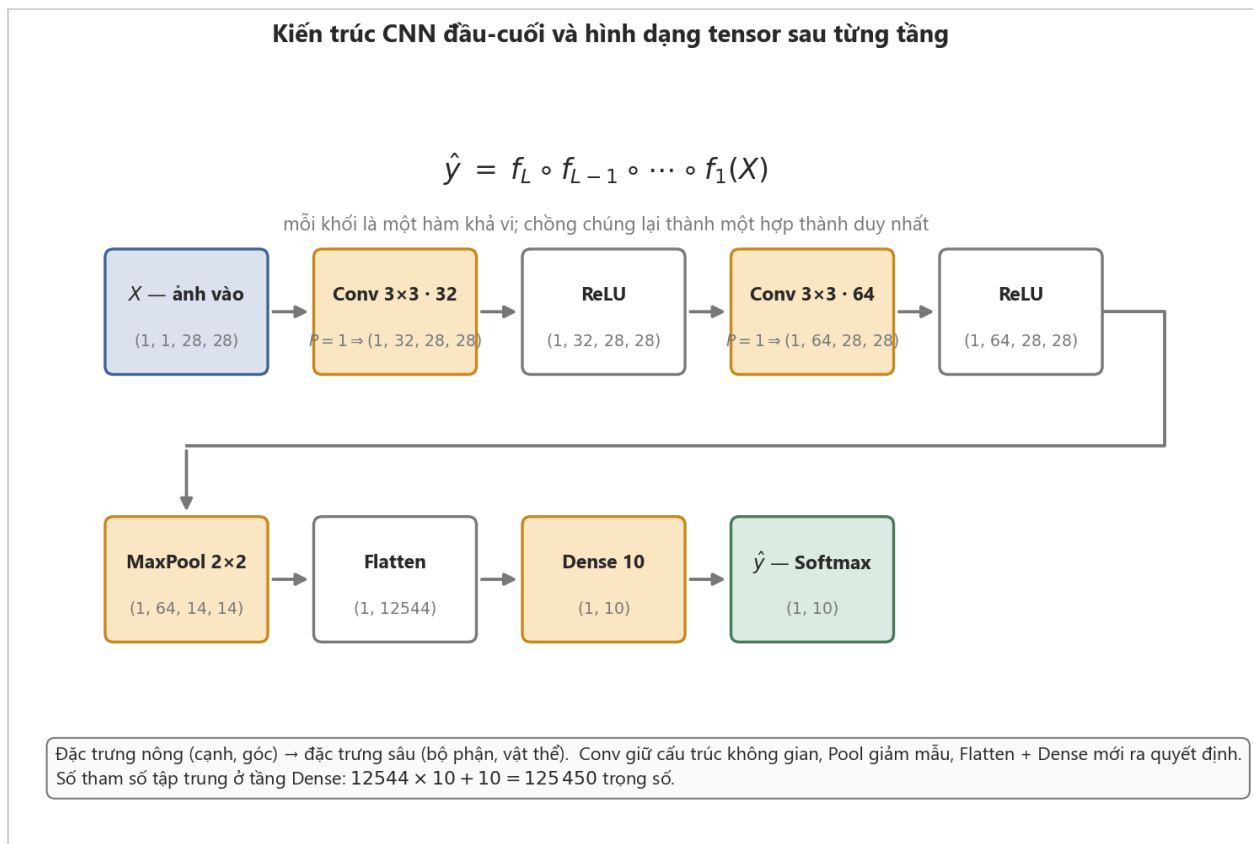
Chương 6 và Chương 7 sẽ đo trực tiếp tác động của lựa chọn này: mô hình cơ sở dùng khởi tạo ngẫu nhiên tỉ lệ cố định, mô hình cải tiến dùng He Normal cùng đệm viên và lịch giảm tốc độ học, và khoảng cách giữa hai bên được báo cáo bằng điểm phần trăm tuyệt đối.

1.7. Kiến trúc như một phép hợp hàm và bài toán quyết định tầng cuối

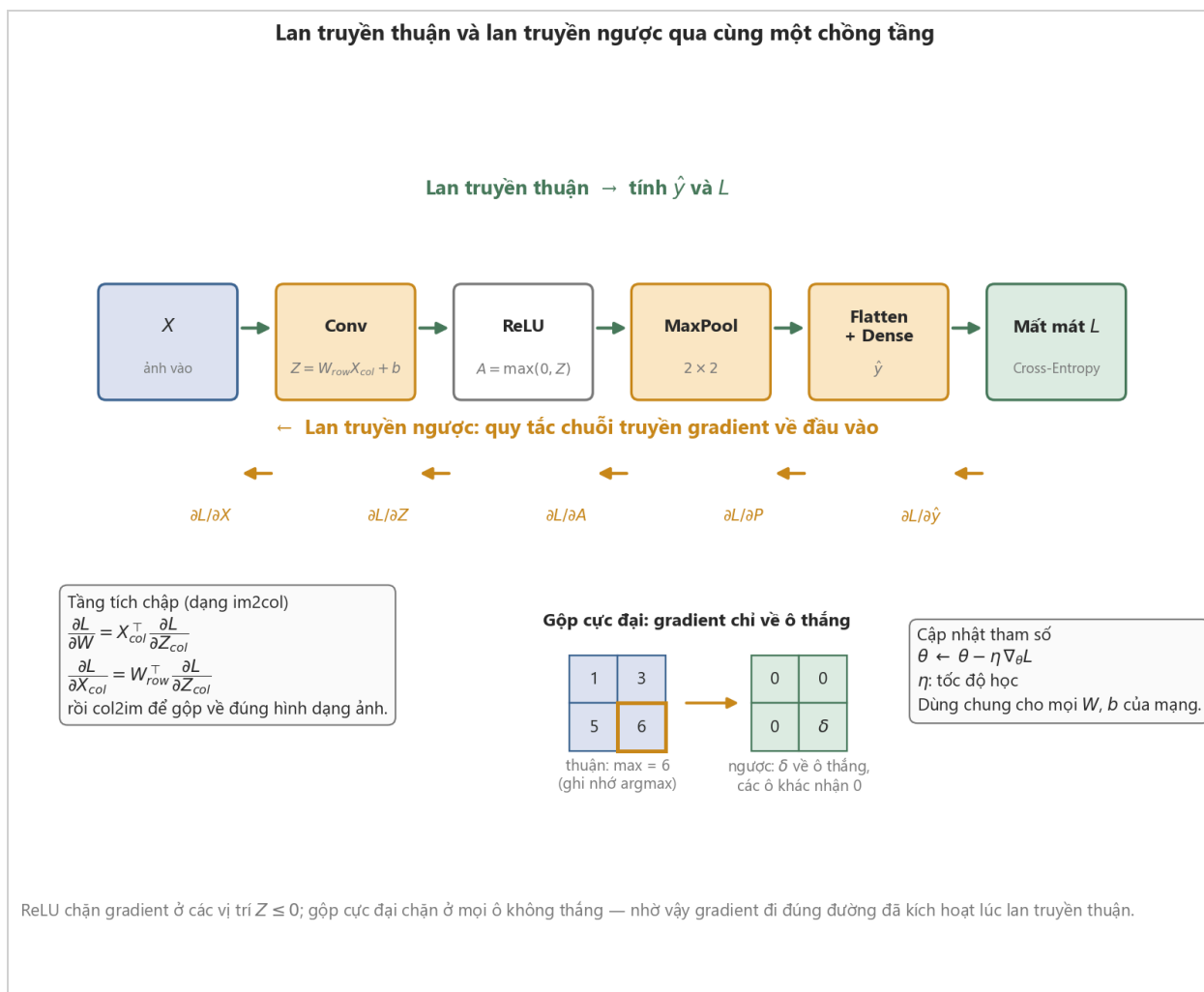
Điểm gắn kết bài tập số 4 với bài tập số 3 nằm ở chỗ mạng tích chập không phải một ý tưởng khác biệt hoàn toàn. Nó vẫn là một phép hợp hàm:

$$\hat{y} = f_L \circ f_{L-1} \circ \dots \circ f_1(X)$$

Khác biệt duy nhất là một số hàm f_i giờ đây là tích chập thay vì phép nhân ma trận đầy đủ. Toàn bộ bộ máy lan truyền ngược, quy tắc chuỗi và cập nhật gradient $\theta \leftarrow \theta - \eta \nabla_{\theta} L$ giữ nguyên.



Hình 8. Mạng tích chập như một phép hợp hàm tham số hoá, kèm hình dạng tensor tại mỗi chặng biến đổi.



Hình 9. Lược tiến và lược ngược trên cùng một ngăn xếp tầng: đạo hàm theo trọng số, định tuyến gradient qua mặt nạ cực đại và bước cập nhật tham số.

Tầng cuối cùng thì không được phép tùy tiện: nó phải khớp với bài toán. Đây là nguyên tắc chi phối toàn bộ phần thực nghiệm của báo cáo.

Bảng 1. Kiến trúc tầng cuối được quyết định bởi bài toán, không phải bởi sở thích.

Loại bài toán	Kích hoạt đầu ra	Hàm mất mát	Áp dụng ở chương
Phân loại nhị phân	Sigmoid	Binary Cross-Entropy	Chương 3, Chương 4
Phân loại đa lớp	Softmax	Categorical Cross-Entropy	Chương 6, Chương 7
Hồi quy giá trị liên tục	Tuyến tính	MSE hoặc MAE	Chương 5

Kiến trúc = Dữ liệu + Bài toán + Biểu diễn

1.8. Softmax, Cross-Entropy và một mẹo số học bắt buộc

Với vector logits z , softmax cho xác suất của lớp k bằng $\hat{y}_k = \exp(z_k) / \sum_j \exp(z_j)$. Hiện thực ngây thơ của công thức này tràn số ngay khi một logit vượt quá khoảng 700, vì $\exp(700)$ đã vượt giới hạn số thực 64 bit. Mọi notebook trong báo cáo trừ $\max_j z_j$ trước khi lấy hàm mũ; phép trừ này không đổi giá trị softmax nhưng loại bỏ hoàn toàn khả năng tràn.

Hàm mất mát Cross-Entropy trên một lô m mẫu với nhãn one-hot y là:

$$L = - (1/m) \sum_i \sum_k y_{ik} \log(\hat{y}_{ik} + \varepsilon)$$

Gộp softmax và Cross-Entropy lại rồi lấy đạo hàm theo logits cho một kết quả đẹp tới mức đáng ngờ:

$$\partial L / \partial z = \hat{y} - y$$

Đây là điểm khởi đầu của toàn bộ chuỗi lan truyền ngược qua Dense, gộp và tích chập trong các hiện thực NumPy. Sự gọn gàng của nó cũng là lý do hai hàm này gần như luôn được cài đặt chung thành một khối thay vì hai tầng riêng biệt.

1.9. Ba cách cài đặt: điều gì đổi, điều gì không

Báo cáo hiện thực mỗi kiến trúc ba lần. Câu hỏi đáng hỏi không phải là bên nào thắng, mà là *thứ gì thay đổi và thứ gì bất biến* giữa ba cách cài đặt.

Bảng 2. Điều thay đổi giữa ba cách cài đặt là tầng hiện thực, không phải mô hình toán học.

Thành phần	NumPy thuần	PyTorch	TensorFlow / Keras
Tích chập	<code>conv1d/2d()</code> tự viết	<code>nn.Conv1d/2d</code>	<code>layers.Conv1D/2D</code>
Kích hoạt	<code>relu()</code> tự viết	<code>nn.ReLU</code>	<code>activation='relu'</code>
Gộp	Mặt nạ cực đại tự viết	<code>nn.MaxPool1d/2d</code>	<code>MaxPooling1D/2D</code>
Gradient	Đạo hàm giải tích viết tay	Autograd	<code>GradientTape</code> / tự động
Cập nhật tham số	Adam tự viết	<code>optimizer.step()</code>	<code>model.fit()</code>
Vòng lặp huấn luyện	Viết tay hoàn toàn	Viết tay tường minh	<code>fit()</code> hoặc vòng lặp tùy biến

Điều *không* đổi: dữ liệu đầu vào, cách tách tập, quy trình tiền xử lý, ý tưởng kiến trúc, hàm mất mát, bộ chỉ số đánh giá và mục tiêu tối ưu. Chính vì vậy một phép so sánh công bằng bắt buộc phải kiểm soát tất cả những yếu tố đó, và Chương 2 mở đầu bằng việc nêu rõ thiết kế kiểm soát ấy.

Bài học kỹ nghệ phần mềm. Cùng một mô hình toán học không có nghĩa là cùng một hiện thực. Khi hai khung thư viện cho ra số liệu lệch nhau dưới một điểm phần trăm, cái ta chứng minh được là nền toán học vững, chứ không phải khung nào ưu việt hơn.

1.10. Các hướng kiến trúc phát triển từ tích chập cơ bản

Những kiến trúc hiện đại đều xây trên đúng các khối vừa mô tả. Bảng dưới tóm tắt cải tiến cốt lõi của từng mốc, để đặt các thực nghiệm của báo cáo vào đúng vị trí trên bản đồ chung.

Bảng 3. Một số hướng phát triển tiêu biểu đi ra từ khối tích chập cơ bản.

Kiến trúc	Cải tiến chính	Ý nghĩa
LeNet-5	Tích chập, gộp và liên kết đầy đủ nối tiếp	Nền móng cho nhận dạng chữ số
AlexNet	ReLU, dropout, tăng cường dữ liệu, huấn luyện trên GPU	Đưa CNN sâu lên quy mô ImageNet
VGG	Xếp chồng thuần bộ lọc 3×3	Thiết kế đồng nhất nhưng nhiều tham số
Inception	Nhiều nhánh kernel và tích chập 1×1	Học đặc trưng đa tỉ lệ trong cùng một khối
ResNet	Kết nối tắt $F(x) + x$	Cho phép mạng rất sâu nhờ dòng gradient không suy giảm
DenseNet	Nối đặc trưng từ mọi tầng trước	Tái sử dụng đặc trưng, giảm số tham số cần thiết
MobileNet	Tích chập tách theo chiều sâu	Cắt mạnh FLOPs cho thiết bị biên
EfficientNet	Scale đồng thời độ sâu, độ rộng và độ phân giải	Cân bằng chất lượng với chi phí theo một quy luật
Vision Transformer	Tự chú ý toàn cục trên các mảnh ảnh	Thay cục bộ bằng toàn cục, đòi hỏi dữ liệu lớn hơn

Các thực nghiệm trong báo cáo dừng ở mức LeNet và VGG thu nhỏ. Lựa chọn này là có chủ đích: mục tiêu của bài tập là hiểu tường tận cơ chế, và một kiến trúc đủ nhỏ để lan truyền ngược viết tay chạy được trong vài phút có giá trị sư phạm cao hơn một mạng sâu mà ta chỉ gọi hàm.

1.11. Bộ chỉ số đánh giá dùng xuyên suốt báo cáo

Với bài toán phân loại, độ chính xác là tỉ lệ dự đoán đúng trên toàn tập kiểm thử. Chỉ số này đủ khi các lớp cân bằng, nhưng dễ đánh lừa khi không. Tập Diabetes ở Chương 4 có khoảng 8,5% mẫu dương; một mô hình luôn trả lời "không mắc bệnh" đạt trên 91% độ chính xác mà không có chút giá trị lâm sàng nào. Vì vậy mọi bảng kết quả phân loại trong báo cáo đều kèm precision, recall và F1 trên lớp dương.

Với phân loại mười lớp ở Chương 6 và Chương 7, báo cáo dùng trung bình macro để mỗi lớp đóng góp ngang nhau vào kết luận:

$$Macro-F1 = (1/K) \sum_k F1_k$$

Với bài toán hồi quy ở Chương 5, ba chỉ số được báo cáo cùng lúc: RMSE phạt nặng sai số lớn, MAE phản ánh sai số điển hình bằng chính đơn vị đô-la, và R^2 cho biết tỉ lệ phương sai được giải thích. Ba con số này trả lời ba câu hỏi khác nhau, nên chỉ trích dẫn một trong ba là che mất hai phần ba bức tranh.

Cuối cùng, mọi bảng đối chuẩn đều kèm số tham số và thời gian huấn luyện. Một mô hình hơn đối thủ 0,3 điểm phần trăm nhưng tốn gấp mười lần thời gian không phải là mô hình tốt hơn trong mọi ngữ cảnh, và báo cáo cố gắng không giấu đi sự đánh đổi đó.

Chương 2. Thiết kế thực nghiệm và giao thức đối chuẩn

Bài tập này không đặt mục tiêu sẵn một con số độ chính xác cao. Mục tiêu là trả lời một câu hỏi có cấu trúc: *khi cùng một ý tưởng toán học được hiện thực ở ba mức độ kiểm soát khác nhau, trên năm miền dữ liệu có hình học khác nhau, thì cái gì bất biến và cái gì thay đổi?* Một câu hỏi như vậy chỉ có nghĩa nếu mọi yếu tố ngoài biên khảo sát được giữ cố định. Chương này nêu rõ những gì đã được giữ cố định, và thành thật về những gì không giữ được.

2.1. Ba mức độ kiểm soát khi xây dựng cùng một kiến trúc

1. **NumPy thuần từ đầu.** Tự viết tích chập, gộp, Dense, softmax hoặc sigmoid, lan truyền ngược và Adam. Không autograd, không một dòng gọi thư viện học sâu nào. Đây là mức kiểm soát cao nhất và cũng là mức chậm nhất.
2. **PyTorch.** Định nghĩa module, optimizer và vòng lặp huấn luyện tường minh. Autograd lo phần đạo hàm; phần còn lại vẫn nằm trong tay người viết.
3. **TensorFlow / Keras.** Mô tả kiến trúc bằng API cấp cao, giao phần lớn vòng lặp cho `fit()`. Mức kiểm soát thấp nhất và mã nguồn ngắn nhất.

Ba mức này không cạnh tranh nhau. Chúng trả lời ba câu hỏi khác nhau: mức một trả lời "thuật toán hoạt động ra sao", mức hai trả lời "làm sao để linh hoạt và gỡ lỗi được", mức ba trả lời "làm sao để viết nhanh và chuẩn hoá".

2.2. Năm miền dữ liệu và ánh xạ sang toán tử tích chập

Năm bộ dữ liệu được chọn để phủ ba dạng hình học dữ liệu khác nhau, chứ không phải để phủ nhiều lĩnh vực ứng dụng.

Bảng 4. Năm miền dữ liệu của bài tập số 4 và cách mỗi miền được ánh xạ sang toán tử tích chập.

Miền dữ liệu	Dạng dữ liệu	Bài toán	Quy mô thô	Diễn giải tích chập
Customer Comments	Chuỗi token	Phân loại nhị phân	23.486 đánh giá	Cửa sổ trượt bất cụm n-gram
Diabetes	Bảng số	Phân loại nhị phân	100.000 hồ sơ	Chuỗi 1D nhân tạo dài 8
House Price	Bảng số	Hồi quy liên tục	150.000 giao dịch	Chuỗi 1D nhân tạo dài 8
MNIST	Ảnh xám 28×28×1	Phân loại 10 lớp	70.000 ảnh	Cửa sổ trượt 2D trên lưới điểm ảnh
CIFAR-10	Ảnh màu 32×32×3	Phân loại 10 lớp	60.000 ảnh	Cửa sổ trượt 2D trên ba kênh màu

Cảnh báo phương pháp luận, nêu trước khi trình bày bất cứ kết quả nào. Dữ liệu bảng *không* có tô-pô không gian. Việc cột `Glucose` nằm cạnh cột `BloodPressure` trong tệp CSV là một quy ước trình bày, không phải một quan hệ lân cận. Trượt bộ lọc dọc theo tám cột vì vậy là một **nguyên cứu đối sánh có kiểm soát** về năng lực biểu diễn của khung tích chập, chứ không phải một khẳng định rằng CNN là kiến trúc tối ưu cho dữ liệu bảng. Chương 4 kiểm chứng luận điểm này bằng một thực nghiệm hoán vị cột, thay vì chỉ tuyên bố nó.

2.3. Tách dữ liệu và phòng rò rỉ

Mọi miền dùng chung một giao thức. Tập kiểm thử được tách ra trước tiên và **không tham gia** vào bất kỳ quyết định nào: không chọn epoch, không chọn tốc độ học, không chọn ngưỡng phân loại. Phần còn lại được tách tiếp thành train và validation theo phân tầng với `random_state = 42`.

Tham số chuẩn hoá là điểm rõ ràng dễ mắc nhất và thường bị bỏ qua. Trung bình và độ lệch chuẩn chỉ được học từ tập train, sau đó cùng bộ tham số đó được áp cho validation và test:

$$X_{\text{chuẩn hoá}} = (X/255 - \mu_{\text{train}}) / (\sigma_{\text{train}} + \varepsilon)$$

Tương tự với văn bản, từ điển được xây dựng từ tập huấn luyện; từ nào chỉ xuất hiện ở tập kiểm thử được ánh xạ về token `<UNK>`. Nếu xây từ điển trên toàn bộ dữ liệu, mô hình đã biết trước sự tồn tại của những từ mà lẽ ra nó chưa từng gặp.

Bảng 5. Kích thước thực tế của các tập dữ liệu sau khi tách, đọc từ tệp JSON của notebook.

Miền dữ liệu	Train	Validation	Test	Số đặc trưng đầu vào
Customer Comments	15.847	3.397	3.397	50
Diabetes	67.289	14.419	14.420	8
House Price	96.000	24.000	30.000	8
MNIST	48.000	12.000	10.000	784
CIFAR-10	40.000	10.000	10.000	3072

2.4. Cấu hình kiến trúc giữ cố định giữa ba khung

Điều kiện tiên quyết để so sánh có nghĩa là ba khung phải dựng *cùng một* kiến trúc. Hai sơ đồ dưới đây được áp dụng nguyên văn cho cả NumPy, PyTorch và TensorFlow.

Bảng số: $8 \rightarrow \text{Conv1D}(16, K=3) \rightarrow \text{ReLU} \rightarrow \text{Conv1D}(16, K=3) \rightarrow \text{ReLU} \rightarrow \text{MaxPool1D}(2) \rightarrow \text{Flatten} \rightarrow \text{Dense}(8) \rightarrow \text{ReLU} \rightarrow \text{Dense}(1)$
Văn bản: $\text{tokens}(50) \rightarrow \text{Embedding}(5000, 100) \rightarrow \text{Conv1D}(32, K=3) \rightarrow \text{ReLU} \rightarrow \text{GlobalMaxPool1D} \rightarrow \text{Dense}(1, \text{Sigmoid})$

Với ảnh, mô hình NumPy giữ ở mức hai khối tích chập để lan truyền ngược viết tay còn chạy được trong vài phút, trong khi PyTorch và Keras dùng kiến trúc sâu hơn có Batch Normalization và Dropout. Đây là một sai lệch có chủ đích và được báo cáo công khai ở mục 9.3, chứ không bị che đi bằng cách chỉ trích dẫn con số cuối cùng.

2.5. Vòng lặp huấn luyện viết tay

Đoạn mã dưới tóm tắt khung vòng lặp dùng chung cho mọi mô hình NumPy trong báo cáo. Khác biệt giữa các miền chỉ nằm ở số bộ lọc, số nơ-ron ẩn, số epoch và hàm mất mát.

PYTHON

```

for epoch in range(epochs):
    lr = lr_schedule(initial_lr, epoch, epochs)      # lịch giảm tốc độ học
    order = rng.permutation(len(X_train))            # xáo trộn lại mỗi epoch

    for start in range(0, len(order), batch_size):
        idx = order[start:start + batch_size]
        logits = model.forward(X_train[idx])          # lượt tiến
        loss = criterion.forward(logits, y_train[idx])
        grad = criterion.backward()                   # dL/dz
        model.backward(grad)                          # lan truyền ngược viết tay
        optimizer.step(model.parameters(), lr)        # Adam viết tay

    val_loss, val_acc = evaluate(model, X_val, y_val)
    if val_loss < best_loss:                          # chỉ dựa trên validation
        best_loss, best_state = val_loss, model.state_dict()

model.load_state_dict(best_state)                    # khôi phục trước khi chạy
test

```

Khung vòng lặp huấn luyện dùng chung cho mọi hiện thực NumPy trong báo cáo.

Một ghi chú về cột thời gian trong mọi bảng đối chuẩn. Ba cách cài đặt không chạy trên cùng một thiết bị. Các mô hình tích chập hai chiều trên ảnh dùng GPU, còn các mô hình một chiều trên bảng và văn bản dùng CPU, vì ở quy mô dưới mười nghìn tham số thì chi phí khởi chạy kernel và truyền dữ liệu lớn hơn chính phần tính toán. Ngoài ra TensorFlow từ bản 2.11 không còn hỗ trợ GPU native trên Windows, và các mô hình NumPy thuần chạy CPU là bản chất của bài tập. Vì vậy **cột thời gian chỉ cho biết chi phí tuyệt đối của từng cấu hình, không dùng để so sánh khung thư viện với nhau.** Mỗi bảng đều ghi kèm thiết bị ở cột bên cạnh. Các chỉ số chất lượng không phụ thuộc thiết bị nên vẫn so sánh được bình thường.

Hai chi tiết đáng chú ý. Thứ nhất, optimizer chỉ nhìn thấy train loss; validation chỉ dùng để theo dõi và chọn checkpoint. Thứ hai, trạng thái tốt nhất được khôi phục *trước khi* chạy vào tập kiểm thử, nên vài epoch cuối dao động không làm hỏng con số báo cáo.

2.6. Nguyên tắc đọc kết quả

Các bảng đối chuẩn trong báo cáo được sắp theo chỉ số chính của từng bài toán: F1 cho phân loại nhị phân mất cân bằng, Macro-F1 cho phân loại mười lớp, R^2 cho hồi quy. Nhưng một bảng xếp hạng không phải là kết luận. Ba nguyên tắc đọc:

1. Một khoảng cách nhỏ hơn một điểm phần trăm giữa hai khung, với cùng kiến trúc và cùng dữ liệu, nên được đọc là *trương đương*, không phải là thắng thua. Nó đến từ khởi tạo ngẫu nhiên và thứ tự mini-batch nhiều hơn là từ chất lượng khung.
2. Thời gian huấn luyện là số đo của *một lần chạy trên một máy cụ thể*. Toàn bộ thực nghiệm chạy trên CPU không có GPU, nên các con số thời gian nói về chi phí tương đối giữa ba khung, không phải về hiệu năng tuyệt đối của khung.
3. Chỉ một hạt giống ngẫu nhiên duy nhất được báo cáo. Báo cáo không đưa ra trung bình và độ lệch chuẩn qua nhiều lần chạy, nên mọi khoảng cách nhỏ đều mang tính gợi ý chứ chưa phải bằng chứng thống kê.

Chương 3. Tích chập một chiều trên chuỗi văn bản

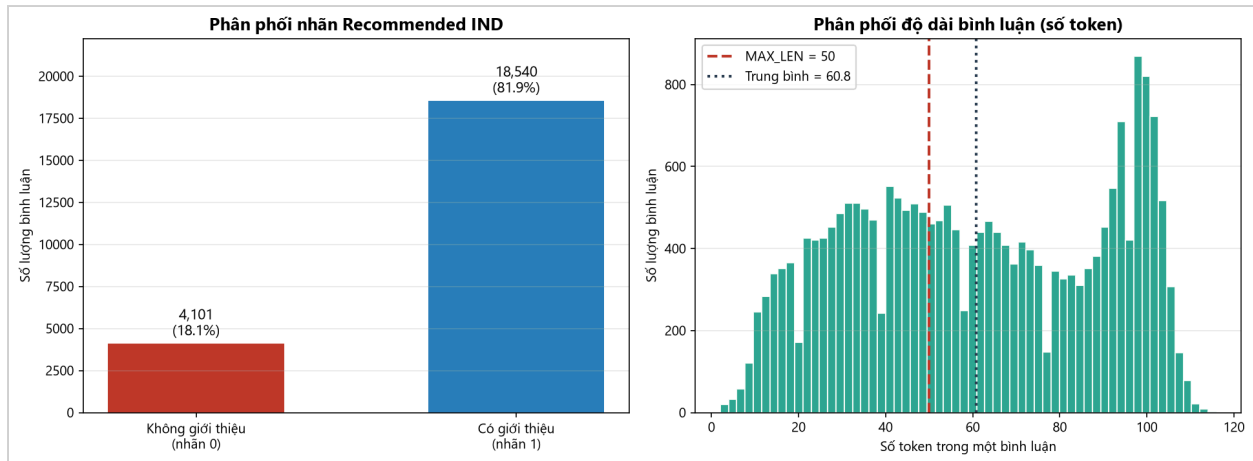
Trong năm miền dữ liệu của báo cáo, văn bản là miền duy nhất mà tích chập một chiều có biện minh lý thuyết trọn vẹn. Thứ tự các từ trong câu không phải quy ước: đảo hai từ cạnh nhau có thể lật ngược nghĩa của cả câu. Một bộ lọc rộng ba token vì vậy đang học đúng thứ nó nên học, những cụm như "rất đáng tiền" hay "không vừa vặn".

3.1. Dữ liệu và quy trình tiền xử lý chuỗi

Tập dữ liệu gồm 23.486 đánh giá sản phẩm thời trang trên một sàn thương mại điện tử. Sau khi loại các bản ghi thiếu nội dung hoặc thiếu nhãn, còn 22.641 mẫu hợp lệ. Nhãn mục tiêu là `Recommended IND`, nhận giá trị 1 khi khách hàng khuyến nghị sản phẩm.

Quy trình xử lý chuỗi gồm bốn bước, mỗi bước đều có lý do chứ không phải thói quen:

- 1. Chuẩn hoá về chữ thường và tách token bằng biểu thức chính quy `[a-zA-Z]+`.**
Thao tác này gộp Good và good thành một token, giảm kích thước từ điển mà không mất thông tin cảm xúc.
- 2. Xây từ điển 5.000 từ phổ biến nhất**, với chỉ số 0 dành cho `<PAD>` và 1 cho `<UNK>`.
Ngưỡng 5.000 là điểm cân bằng: đủ rộng để phủ từ vựng cảm xúc thông dụng, đủ hẹp để bảng nhúng không chiếm phần lớn tham số của mô hình.
- 3. Cắt hoặc đệm về đúng 50 token.** Chuỗi cố định cho phép xử lý theo lô; token đệm mang chỉ số 0 và vector nhúng của nó được học như mọi token khác.
- 4. Tầng nhúng 100 chiều** ánh xạ mỗi chỉ số nguyên thành một vector liên tục. Đây là tầng đầu tiên trong mạng và cũng là tầng nhiều tham số nhất.



Hình 10. Khảo sát tập đánh giá khách hàng: phân phối nhãn khuyến nghị và phân bố độ dài chuỗi sau khi tách token.

3.2. Kiến trúc và vai trò của gộp cực đại toàn cục

tokens(50) → Embedding(5000×100) → Conv1D(32, K=3) → ReLU → GlobalMaxPool1D → Dense(1) → Sigmoid

Tầng đáng bàn nhất ở đây là gộp cực đại toàn cục. Sau tích chập, ta có 32 bản đồ đặc trưng, mỗi bản đồ dài 48. Gộp cực đại toàn cục lấy giá trị lớn nhất trên toàn chiều dài của từng bản đồ, cho ra đúng 32 số.

Ý nghĩa của phép rút gọn này rất cụ thể: mỗi bộ lọc trả lời câu hỏi *"hoa văn mà tôi phụ trách có xuất hiện ở đâu đó trong câu này không, và mạnh đến mức nào?"* Vị trí xuất hiện bị bỏ đi một cách có chủ ý. Với phân loại cảm xúc, biết câu có chứa cụm phủ định mạnh là đủ; biết cụm đó nằm ở token thứ 7 hay thứ 31 thì không thêm thông tin. Đây cũng là lý do kiến trúc này xử lý được câu dài ngắn khác nhau mà không cần thay đổi gì.

3.3. Hiện thực tích chập một chiều bằng NumPy thuần

Hiện thực ngây thơ sẽ dùng một vòng lặp Python chạy qua 48 vị trí cửa sổ. Cách làm đó đúng nhưng chậm hơn hai bậc độ lớn so với cần thiết. Báo cáo dùng `sliding_window_view` để tạo *view* của mọi cửa sổ cùng lúc mà không sao chép bộ nhớ, rồi gom toàn bộ phép nhân cộng vào một lời gọi `einsum` duy nhất.

PYTHON

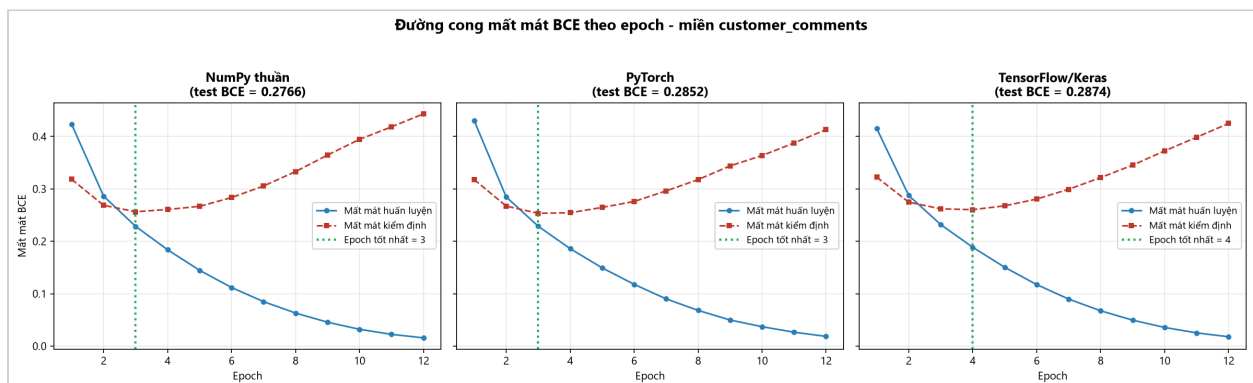
```
def forward(self, X):
    self.X = X
    X_pad = np.pad(X, ((0, 0), (0, 0), (self.padding,) * 2)) if self.padding else X
    # n: mẫu trong lô · c: kênh vào · L: vị trí cửa sổ · k: chỉ số trong kernel
    self.windows = np.lib.stride_tricks.sliding_window_view(
        X_pad, window_shape=self.kernel_size, axis=-1)[:, :, ::self.stride, :]
    return np.einsum('ncl,kfck->nfl', self.windows, self.W) + self.b[:, None]

def backward(self, dZ):
    self.dZ = dZ.sum(axis=(0, 2))
    self.dW = np.einsum('nfl,ncl->fck', dZ, self.windows) # cộng dồn mọi vị trí
    dX, L_out = np.zeros_like(self.X), dZ.shape[2]
    for k in range(self.kernel_size): # rải ngược về đầu vào
        dX[:, :, k:k + L_out] += np.einsum('nfl,fc->ncl', dZ, self.W[:, :, k])
    return dX
```

Tầng Conv1D thuần NumPy, vector hoá bằng `sliding_window_view` và `einsum`.

Dòng tính `self.dW` là nơi nguyên lý chia sẻ trọng số hiện ra rõ nhất. Phép `einsum` cộng dồn trên cả chỉ số lô `n` lẫn chỉ số vị trí `1`, vì một trọng số duy nhất đã tham gia vào mọi vị trí cửa sổ của mọi mẫu trong lô. Bỏ sót phép cộng dồn này là lỗi kinh điển khi tự viết tích chập, và nó không gây lỗi chạy mà chỉ khiến mô hình học chậm một cách khó hiểu.

3.4. Diễn biến huấn luyện



Hình 11. Hàm mất mát Binary Cross-Entropy theo epoch của ba cách cài đặt trên tập đánh giá khách hàng.

Ba đường cong cho thấy cùng một hình dạng hội tụ. NumPy thuần hạ mất mát huấn luyện từ 0.4233 xuống 0.0158 sau 12 epoch, mất mát kiểm định tương ứng đi từ 0.3183 xuống 0.4432; PyTorch hạ mất mát huấn luyện từ 0.4302 xuống 0.0186 sau 12 epoch, mất mát kiểm định tương ứng đi từ 0.3178 xuống 0.4132; TensorFlow / Keras hạ mất mát huấn

luyện từ 0.4152 xuống 0.0178 sau 12 epoch, mất mát kiểm định tương ứng đi từ 0.3223 xuống 0.4250.

Khoảng cách train và validation lớn nhất là 0.4274 ở khung NumPy thuần. Mức chênh này cho thấy mô hình bắt đầu học thuộc dữ liệu huấn luyện, và chính là lý do quy trình khôi phục checkpoint tốt nhất theo validation ở mục 2.5 là cần thiết chứ không phải thủ tục hình thức.

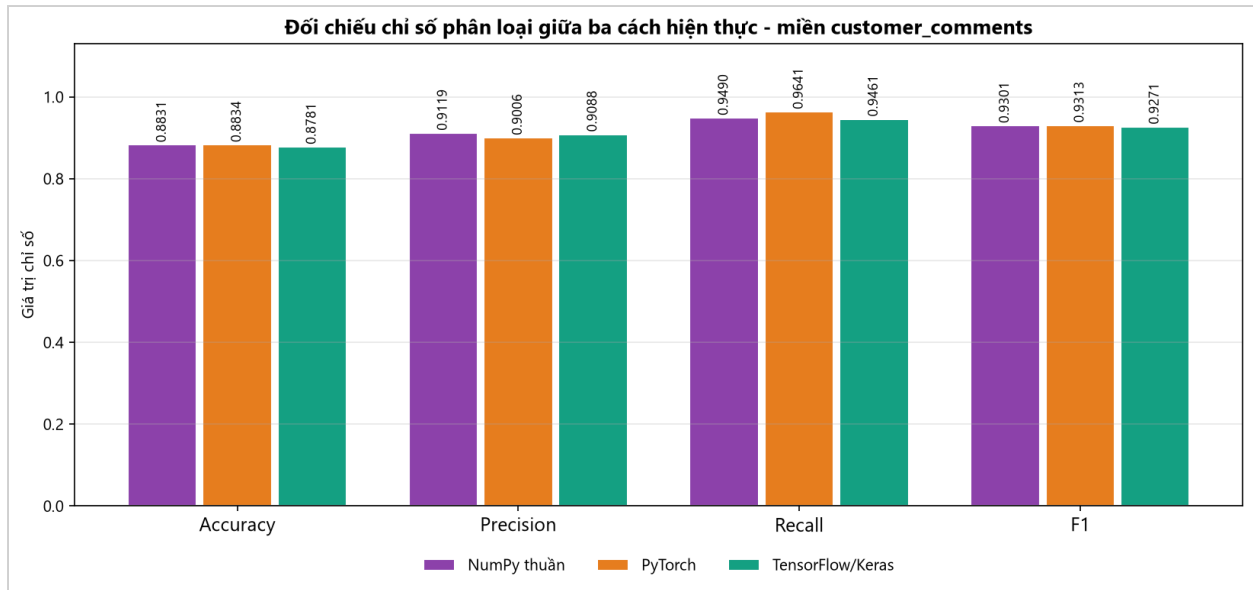
3.5. Kết quả đối chuẩn ba khung

Bảng 6. Đối chuẩn 1D CNN trên tập đánh giá khách hàng, đo trên cùng một tập kiểm thử.

Khung hiện thực	Accurac y	Precisio n	Recall	F1	Tham số	Thiết bị	Thời gian (s)
NumPy thuần	88.31%	91.19%	94.90 %	93.01 %	509,665	CPU	444.2
PyTorch	88.34%	90.06%	96.41 %	93.12 %	509,665	GPU	117.0
TensorFlow / Keras	87.81%	90.88%	94.61 %	92.71 %	509,665	CPU	75.5

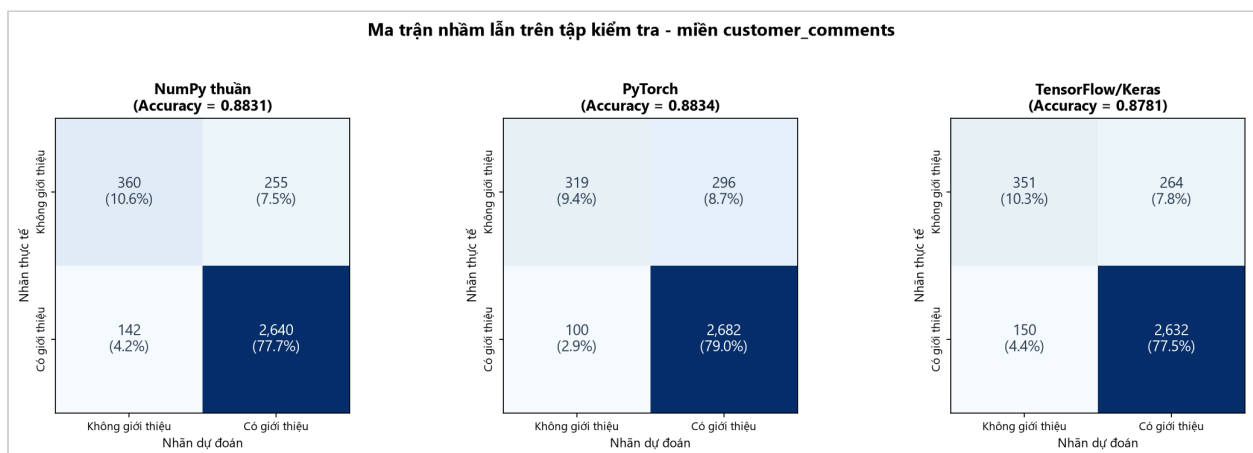
Khung PyTorch dẫn đầu theo F1 với 93.12%, TensorFlow / Keras xếp sau với 92.71%. Khoảng cách 0.42% nằm trong biên độ dao động do khởi tạo ngẫu nhiên và thứ tự mini-batch, nên kết luận đúng ở đây là ba khung *ương đương* khi được cấp cùng kiến trúc và cùng dữ liệu. Đó chính là điều báo cáo muốn chứng minh: nền toán học quyết định kết quả, tầng hiện thực chỉ quyết định tốc độ và công sức viết mã.

Về thời gian chạy, TensorFlow / Keras mất 75.5 giây và NumPy thuần mất 444.2 giây. Hai con số này **không** so sánh được với nhau, vì chúng đo trên hai loại thiết bị khác nhau như ô cảnh báo bên trên đã nêu. Điều duy nhất rút ra được là chi phí tuyệt đối của từng cấu hình trong đúng thiết lập này, đủ để biết một lần chạy lại tốn bao lâu.



Hình 12. So sánh Accuracy, Precision, Recall và F1 giữa ba cách cài đặt trên tập đánh giá khách hàng.

3.6. Ma trận nhầm lẫn và cấu trúc lỗi



Hình 13. Ma trận nhầm lẫn trên tập kiểm thử của ba cách cài đặt.

Lấy mô hình tốt nhất (PyTorch) làm đại diện: trên 3,397 mẫu kiểm thử, mô hình nhận đúng 2,682 đánh giá tích cực và 319 đánh giá tiêu cực, trong khi bỏ sót 100 đánh giá tích cực và báo nhầm 296 đánh giá tiêu cực. Số dương tính giả (296) lớn hơn số âm tính giả (100), nghĩa là mô hình đang nghiêng về phía sẵn sàng gán nhãn dương.

Cấu trúc lỗi ở đây phản ánh đúng đặc thù của dữ liệu đánh giá sản phẩm: phần lớn khách hàng để lại nhận xét tích cực, nên lớp tiêu cực vừa ít mẫu hơn vừa đa dạng hơn về cách diễn đạt. Những câu mang sắc thái trung tính hoặc chứa lời khen kèm một điểm chê là

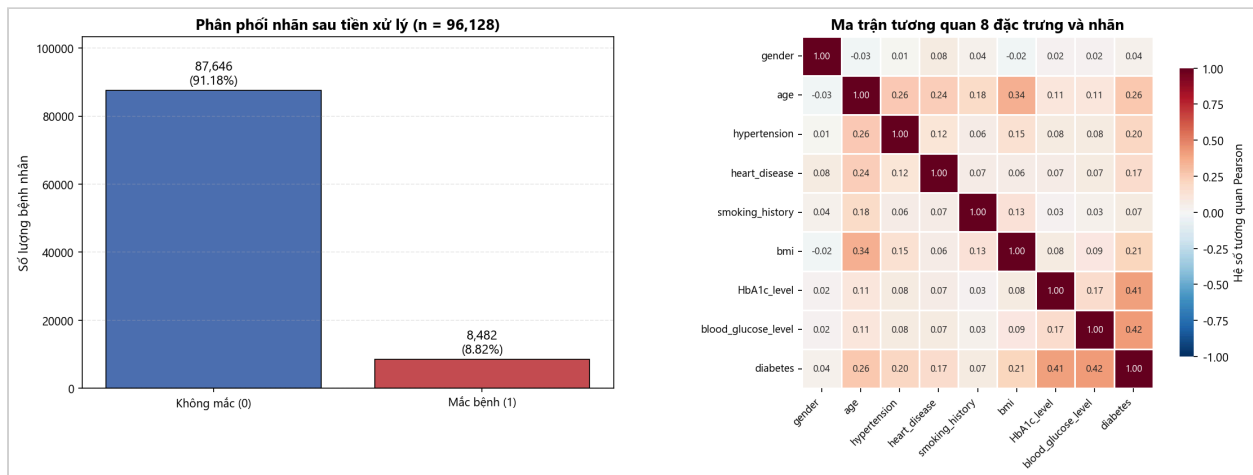
nhóm khó nhất, vì một bộ lọc rộng ba token khó nắm được quan hệ nhượng bộ trải dài cả câu.

Chương 4. Tích chập một chiều trên dữ liệu bảng y tế

Chương này là một thực nghiệm đối sánh có chủ đích. Slide bài giảng nêu thẳng rằng áp dụng tích chập một chiều lên dữ liệu bảng là *một thí nghiệm sư phạm*, không phải một khẳng định về kiến trúc tối ưu. Báo cáo không chỉ nhắc lại cảnh báo đó mà tìm cách đo nó.

4.1. Dữ liệu và tám đặc trưng lâm sàng

Tập dữ liệu gồm 100.000 hồ sơ bệnh nhân. Sau khi khử trùng lặp và loại nhóm giới tính không xác định, còn 96.128 bản ghi. Tám đặc trưng được giữ đúng theo thứ tự nêu trong slide bài giảng, tương ứng với chuỗi đầu vào dài 8 mà tầng tích chập sẽ trượt qua.



Hình 14. Khảo sát tập dữ liệu tiểu đường: phân phối nhân và tương quan giữa tám đặc trưng lâm sàng.

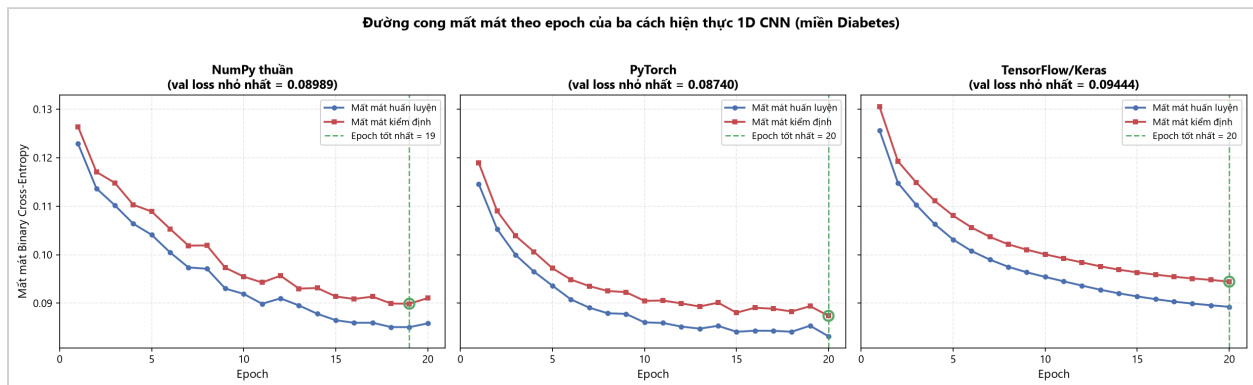
Điểm cần chú ý ngay từ khâu khảo sát là mất cân bằng lớp. Tỷ lệ bệnh nhân dương tính chỉ ở mức một chữ số phần trăm, nghĩa là một mô hình luôn dự đoán "không mắc bệnh" đã đạt độ chính xác rất cao mà hoàn toàn vô dụng về mặt lâm sàng. Vì vậy mọi bảng kết quả trong chương này đặt recall ngang hàng với accuracy.

4.2. Kiến trúc và luồng biến đổi kích thước tensor

8 → Conv1D(16, K=3, same) → ReLU → Conv1D(16, K=3, same) → ReLU → MaxPool1D(2) → Flatten(64) → Dense(8) → ReLU → Dense(1) → Sigmoid

Tầng tích chập thứ nhất học các tổ hợp cục bộ giữa ba chỉ số lân sàng liền kề. Tầng thứ hai học tổ hợp của chính những hoa văn đó, nên trường tiếp nhận mở rộng lên 5 trong 8 đặc trưng. Câu hỏi là: *những tổ hợp "liền kề" ấy có mang ý nghĩa gì không, khi thứ tự cột vốn tùy ý?*

4.3. Diễn biến huấn luyện



Hình 15. Hàm mất mát huấn luyện và kiểm định theo epoch của ba cách cài đặt trên bài toán chẩn đoán tiểu đường.

Ba đường cong cho thấy cùng một hình dạng hội tụ. NumPy thuần hạ mất mát huấn luyện từ 0.1229 xuống 0.0859 sau 20 epoch, mất mát kiểm định tương ứng đi từ 0.1264 xuống 0.0911; PyTorch hạ mất mát huấn luyện từ 0.1146 xuống 0.0832 sau 20 epoch, mất mát kiểm định tương ứng đi từ 0.1189 xuống 0.0874; TensorFlow / Keras hạ mất mát huấn luyện từ 0.1257 xuống 0.0893 sau 20 epoch, mất mát kiểm định tương ứng đi từ 0.1305 xuống 0.0944.

Khoảng cách giữa hai đường train và validation ở cuối quá trình huấn luyện lớn nhất là 0.0052 (khung NumPy thuần). Mức chênh nhỏ như vậy cho biết mô hình chưa quá khớp; hạn chế nằm ở dung lượng biểu diễn chứ không ở việc học thuộc dữ liệu huấn luyện.

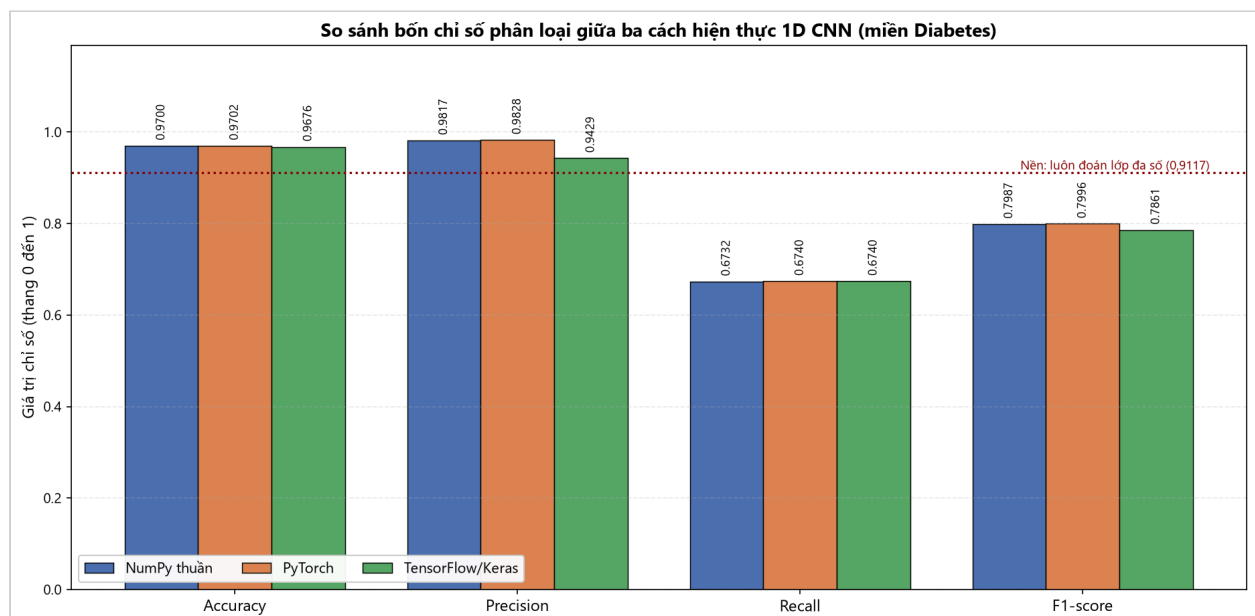
4.4. Kết quả đối chuẩn ba khung

Bảng 7. Hiệu năng 1D CNN trên bài toán chẩn đoán tiểu đường, chỉ số tính trên lớp dương.

Khung hiện thực	Accuracy	Precision	Recall	F1	ROC-AUC	Thiết bị	Thời gian (s)
NumPy thuần	97.00%	98.17%	67.32 %	79.87 %	0.9735	CPU	67.0
PyTorch	97.02%	98.28%	67.40 %	79.96 %	0.9747	GPU	112.9
TensorFlow / Keras	96.76%	94.29%	67.40 %	78.61 %	0.9723	CPU	79.2

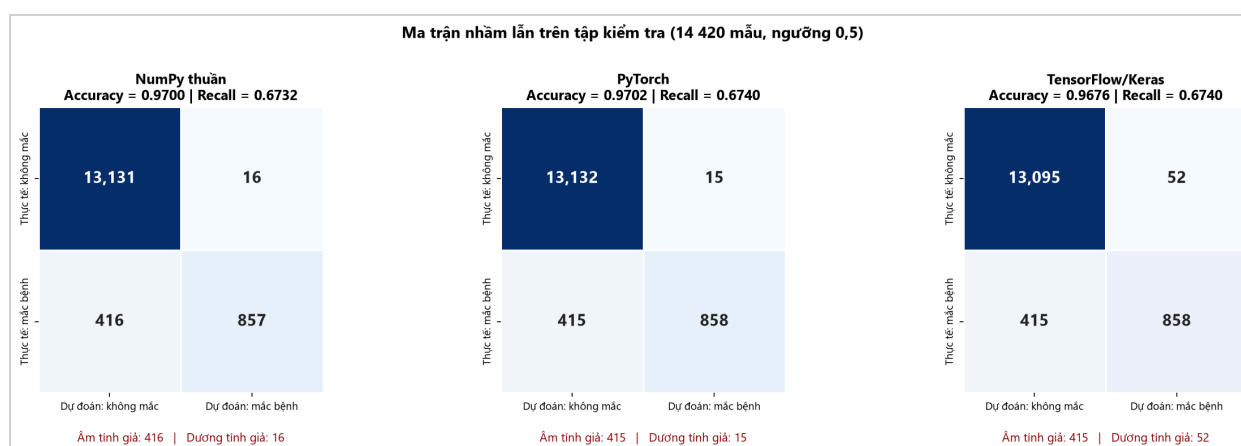
Khung PyTorch dẫn đầu theo F1 với 79.96%, TensorFlow / Keras xếp sau với 78.61%. Khoảng cách 1.36% nằm trong biên độ dao động do khởi tạo ngẫu nhiên và thứ tự mini-batch, nên kết luận đúng ở đây là ba khung *tương đương* khi được cấp cùng kiến trúc và cùng dữ liệu. Đó chính là điều báo cáo muốn chứng minh: nền toán học quyết định kết quả, tầng hiện thực chỉ quyết định tốc độ và công sức viết mã.

Về thời gian chạy, NumPy thuần mất 67.0 giây và PyTorch mất 112.9 giây. Hai con số này **không** so sánh được với nhau, vì chúng đo trên hai loại thiết bị khác nhau như ô cảnh báo bên trên đã nêu. Điều duy nhất rút ra được là chi phí tuyệt đối của từng cấu hình trong đúng thiết lập này, đủ để biết một lần chạy lại tốn bao lâu.



Hình 16. So sánh bốn chỉ số phân loại giữa ba cách cài đặt trên bài toán tiểu đường.

4.5. Ma trận nhầm lẫn và cái giá của âm tính giả



Hình 17. Ma trận nhầm lẫn trên tập kiểm thử chẩn đoán tiểu đường.

Lấy mô hình tốt nhất (PyTorch) làm đại diện: trên 14,420 mẫu kiểm thử, mô hình nhận đúng 858 bệnh nhân dương tính và 13,132 bệnh nhân âm tính, trong khi bỏ lọt 415 bệnh nhân dương tính và báo nhầm 15 bệnh nhân âm tính. Số âm tính giả (415) lớn hơn số dương tính giả (15), nghĩa là mô hình đang nghiêng về phía thận trọng khi khẳng định lớp dương.

Trong bối cảnh sàng lọc y tế, hai loại lỗi không có cùng giá trị. Một dương tính giả dẫn tới một lần xét nghiệm bổ sung; một âm tính giả là một ca bệnh bị bỏ lọt. Nếu triển khai thật, ngưỡng phân loại nên được hạ xuống dưới 0,5 để đổi precision lấy recall, và điểm làm việc phải do bác sĩ chọn trên đường cong ROC chứ không phải do mặc định của thư viện.

4.6. Thực nghiệm hoán vị cột: kiểm chứng cảnh báo phương pháp luận

Luận điểm "dữ liệu bảng không có tô-pô" dễ phát biểu nhưng cần được đo. Thực nghiệm kiểm chứng rất đơn giản: **hoán vị ngẫu nhiên thứ tự tám cột đặc trưng rồi huấn luyện lại từ đầu với đúng mọi siêu tham số cũ.**

Suy luận là thế này. Nếu tích chấp thật sự khai thác quan hệ lân cận giữa các cột, hoán vị phải phá vỡ quan hệ đó và làm hiệu năng sụt rõ rệt, giống hệt như xáo trộn thứ tự từ trong một câu sẽ phá hủy mô hình phân loại văn bản. Nếu hiệu năng gần như không đổi, kết luận là toán tử tích chấp ở đây chỉ đang hoạt động như một mạng truyền thẳng có ràng buộc chia sẻ trọng số, chứ không khai thác cấu trúc nào cả.

Bảng 8. Ảnh hưởng của việc hoán vị ngẫu nhiên thứ tự tám cột đặc trưng lên hiệu năng 1D CNN.

Cấu hình thứ tự cột	Accuracy	F1 lớp dương
Thứ tự gốc theo slide bài giảng	97.00%	79.87%
Thứ tự hoán vị ngẫu nhiên	96.86%	79.19%

Chênh lệch F1 giữa hai cấu hình chỉ là 0.68%. Kết quả này xác nhận cảnh báo ở mục 2.2 một cách định lượng: toán tử tích chập ở đây **không** khai thác được quan hệ lân cận nào giữa các cột, vì đơn giản là không có quan hệ nào để khai thác. Thứ mô hình thực sự học được là một hàm phi tuyến trên tám biến, và ràng buộc chia sẻ trọng số chỉ đóng vai trò một dạng chính quy hoá nhẹ.

Phép so sánh làm rõ ý này: nếu lặp lại đúng thực nghiệm hoán vị trên dữ liệu văn bản ở Chương 3, tức xáo trộn thứ tự token trong mỗi câu, hiệu năng sẽ sụt rõ rệt, vì cụm "không vừa ý" và "ý không vừa" là hai chuỗi khác nhau về nghĩa. Sự bất đối xứng giữa hai miền chính là bằng chứng cho luận điểm về tô-pô dữ liệu.

Kết luận phương pháp luận của Chương 4. Con số accuracy cao ở chương này *không* chứng minh tích chập phù hợp với dữ liệu bảng. Nó chứng minh rằng bộ dữ liệu này đủ tách được bằng một mô hình phi tuyến bất kỳ có dung lượng vừa phải. Đóng góp thực sự của chương là phép đo hoán vị cột, vì nó biến một cảnh báo lý thuyết thành một con số kiểm chứng được.

Chương 5. Tích chập một chiều cho bài toán hồi quy

Ba chương trước đều là phân loại. Chương này đổi bài toán chứ không đổi kiến trúc, để làm nổi bật một nguyên tắc đã nêu ở mục 1.7: **tầng cuối phải theo bài toán**. Toàn bộ phần trích xuất đặc trưng giữ nguyên; chỉ nơ-ron đầu ra chuyển từ sigmoid sang tuyến tính và hàm mất mát chuyển từ Binary Cross-Entropy sang sai số toàn phương trung bình.

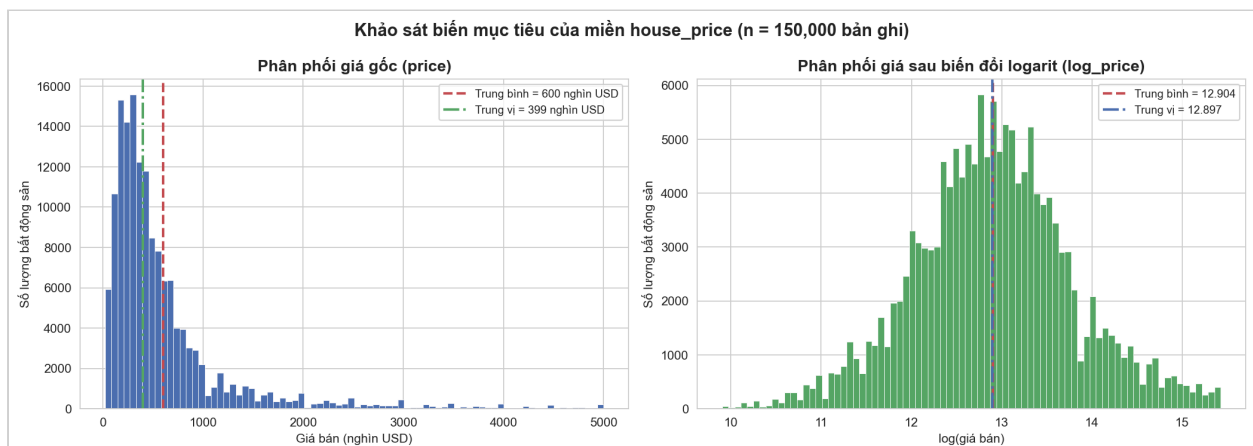
$$L_{MSE} = (1/N) \sum_i (y_i - \hat{y}_i)^2$$

Điều gì xảy ra nếu dùng nhầm? Sigmoid ép đầu ra vào khoảng (0, 1), nên mô hình không thể biểu diễn một mức giá vượt quá 1 dù có huấn luyện bao lâu. Gradient cũng bão hoà về 0 ở hai đầu, khiến việc học dừng hẳn. Đây không phải lỗi tính vi mà là lỗi làm hỏng toàn bộ mô hình ngay từ epoch đầu tiên.

5.1. Dữ liệu, lọc ngoại lai và tám đặc trưng kỹ nghệ

Tập dữ liệu gồm 150.000 bản ghi giao dịch bất động sản. Sau khi loại bản ghi thiếu và cắt ngoại lai (giá ngoài khoảng 10 nghìn đến 5 triệu đô-la, diện tích ngoài khoảng 200 đến 20.000 foot vuông), còn 150.000 mẫu. Việc cắt ngoại lai không phải để làm đẹp số liệu: một biệt thự 50 triệu đô-la trong tập huấn luyện tạo ra gradient lớn tới mức nó chi phối toàn bộ một mini-batch.

Tám đặc trưng được tạo ra từ bốn cột gốc, trong đó có các đặc trưng tương tác như tích số phòng ngủ với số phòng tắm và diện tích trung bình trên mỗi phòng. Những đặc trưng này mang thông tin mà mô hình khó tự học từ dữ liệu thô với kích thước mạng hiện tại.



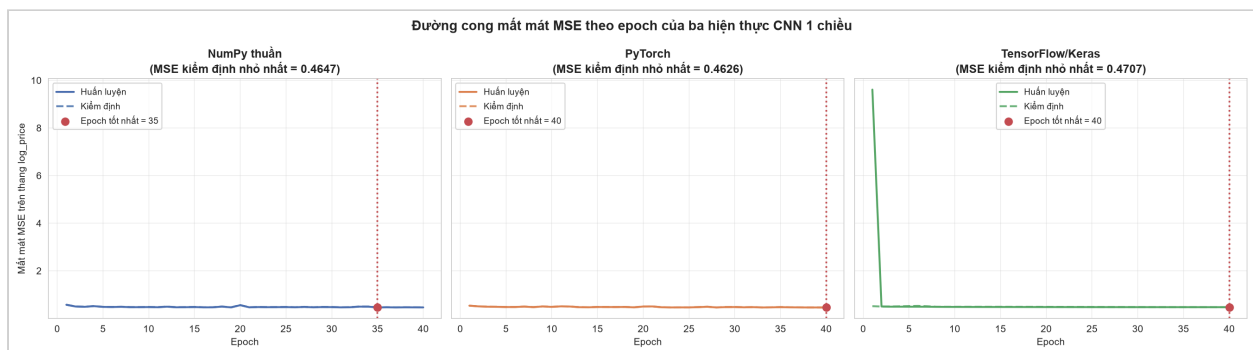
Hình 18. Phân phối giá bất động sản trước và sau phép biến đổi logarit.

5.2. Vì sao huấn luyện trên thang logarit

Phân phối giá nhà lệch phải rất mạnh: phần lớn giao dịch tập trung ở vùng giá thấp, một đuôi dài kéo lên vùng giá cao. Huấn luyện trực tiếp trên thang đô-la khiến hàm MSE bị chi phối bởi vài mẫu đắt tiền, vì sai số được bình phương. Phép biến đổi $\log_price = \log(price)$ đưa phân phối về gần chuẩn và, quan trọng hơn, biến sai số tuyệt đối thành sai số *tương đối*: nhầm 20 nghìn đô-la trên một căn 100 nghìn bị phạt nặng hơn nhầm 20 nghìn trên một căn 2 triệu, đúng như trực giác định giá.

Cái giá phải trả nằm ở khâu quy đổi ngược. Vì hàm mũ lồi, bất đẳng thức Jensen cho biết $E[\exp(\hat{y})] \geq \exp(E[\hat{y}])$, nghĩa là lấy exp của giá trị dự đoán trung bình sẽ *thiên lệch thấp* so với kỳ vọng thật của giá. Báo cáo vì vậy trình bày cả RMSE trên thang logarit lẫn RMSE quy đổi về đô-la, để người đọc thấy được cả hai mặt.

5.3. Diễn biến huấn luyện



Hình 19. Hàm mất mát MSE theo epoch của ba cách cài đặt trên bài toán định giá bất động sản.

Ba đường cong cho thấy cùng một hình dạng hội tụ. NumPy thuần hạ mất mát huấn luyện từ 0.5735 xuống 0.4636 sau 40 epoch, mất mát kiểm định tương ứng đi từ 0.5766 xuống 0.4674; PyTorch hạ mất mát huấn luyện từ 0.5356 xuống 0.4593 sau 40 epoch, mất mát kiểm định tương ứng đi từ 0.5366 xuống 0.4626; TensorFlow / Keras hạ mất mát huấn luyện từ 9.6111 xuống 0.4728 sau 40 epoch, mất mát kiểm định tương ứng đi từ 0.5152 xuống 0.4707.

Khoảng cách giữa hai đường train và validation ở cuối quá trình huấn luyện lớn nhất là 0.0038 (khung NumPy thuần). Mức chênh nhỏ như vậy cho biết mô hình chưa quá khớp; hạn chế nằm ở dung lượng biểu diễn chứ không ở việc học thuộc dữ liệu huấn luyện.

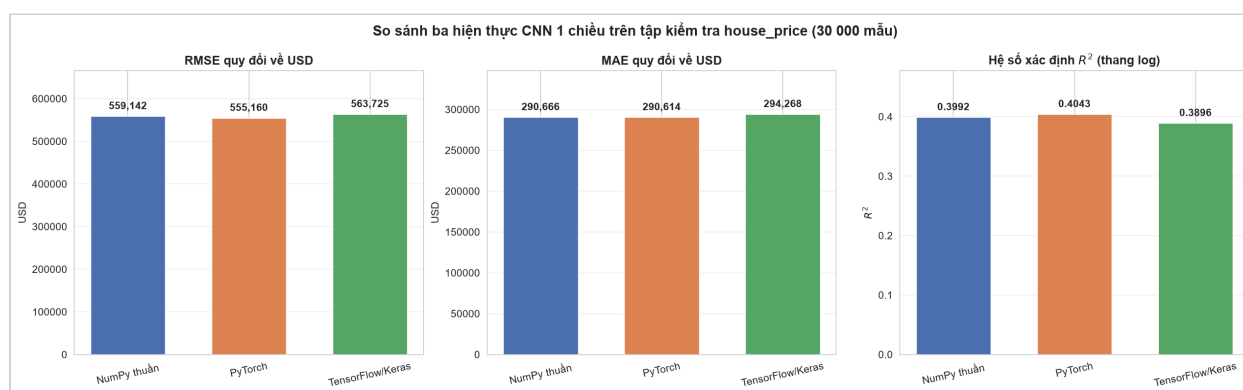
5.4. Kết quả đối chuẩn ba khung

Bảng 9. Các chỉ số hồi quy của mô hình 1D CNN trên bài toán định giá bất động sản.

Khung hiện thực	RMSE (USD)	MAE (USD)	RMSE (log)	MAE (log)	R ²	Thiết bị	Thời gian (s)
NumPy thuần	\$559,142	\$290,666	0.6845	0.5187	0.3992	CPU	74.5
PyTorch	\$555,160	\$290,614	0.6816	0.5185	0.4043	GPU	423.1
TensorFlow / Keras	\$563,725	\$294,268	0.6900	0.5247	0.3896	CPU	311.3

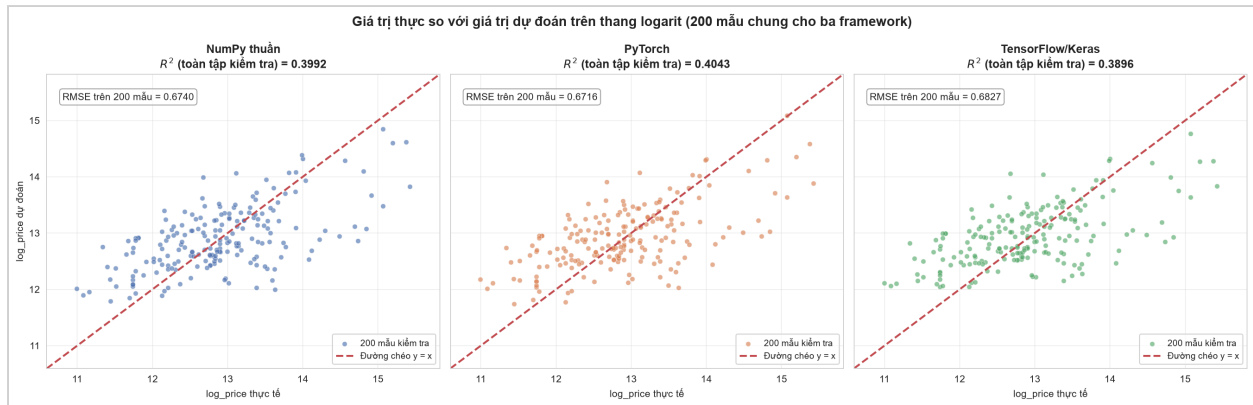
Khung PyTorch đạt R² cao nhất với 0.4043, nghĩa là mô hình giải thích được 40.4% phương sai của giá trên thang logarit. Khoảng cách so với TensorFlow / Keras là 0.0147 đơn vị R². Khoảng cách này đủ nhỏ để quy về dao động khởi tạo chứ không phản ánh ưu thế kiến trúc.

Về sai số tuyệt đối, mô hình tốt nhất cho MAE khoảng \$290,614. Con số này cần được đọc cùng với dải giá của thị trường: sai số vài chục nghìn đô-la là chấp nhận được với một căn nhà giá trên nửa triệu, nhưng là sai số lớn với một căn giá một trăm nghìn. Đây chính là điều mà một chỉ số duy nhất không nói ra, và là lý do RMSE, MAE và R² luôn đi cùng nhau trong bảng trên.



Hình 20. So sánh RMSE, MAE và R² giữa ba cách cài đặt trên bài toán định giá.

5.5. Đồ thị đối chiếu giá trị thực và giá trị dự đoán



Hình 21. Giá trị thực tế đối chiếu giá trị dự đoán trên tập kiểm thử, kèm đường chéo lý tưởng.

Đường chéo là quỹ tích của dự đoán hoàn hảo. Đám mây điểm bám sát đường chéo ở vùng giá trung bình và loe rộng dần về hai đầu. Hiện tượng loe này có một cách giải thích thẳng thắn: vùng giá trung bình chiếm phần lớn dữ liệu huấn luyện nên mô hình được học kỹ nhất ở đó, còn vùng giá rất thấp và rất cao vừa thưa mẫu vừa chịu ảnh hưởng nặng của các yếu tố không có trong tám đặc trưng, đặc biệt là vị trí địa lý.

Một mô hình chỉ biết diện tích, số phòng và diện tích đất thì về nguyên tắc không thể phân biệt hai căn giống hệt nhau ở hai khu vực có mặt bằng giá khác nhau. Phần phương sai không giải thích được vì vậy phần lớn là phương sai địa lý, chứ không phải hạn chế của toán tử tích chập.

Chương 6. Tích chập hai chiều trên ảnh chữ số viết tay

Ba chương trước dùng tích chập một chiều, trong đó chỉ có văn bản thực sự có tô-pô để khai thác. Từ chương này trở đi, dữ liệu là ảnh, và mọi giả định đặt ra ở Chương 1 đều được thoả mãn trọn vẹn: điểm ảnh lân cận thực sự có quan hệ, một hoa văn thực sự có thể xuất hiện ở nhiều vị trí, và chia sẻ trọng số thực sự tiết kiệm tham số một cách có ý nghĩa.

MNIST được chọn làm bước đầu tiên không phải vì nó khó mà vì nó *dễ theo một cách hữu ích*. Nền đen đồng nhất, một đối tượng duy nhất, chữ số đã được căn vào trọng tâm khung 28×28 . Chính sự sạch sẽ đó biến MNIST thành bộ kiểm tra tính đúng đắn lý tưởng cho một hiện thực tích chập viết tay: nếu mã nguồn đúng, độ chính xác phải vượt 97%; nếu không, lỗi nằm ở đạo hàm chứ không ở dữ liệu.

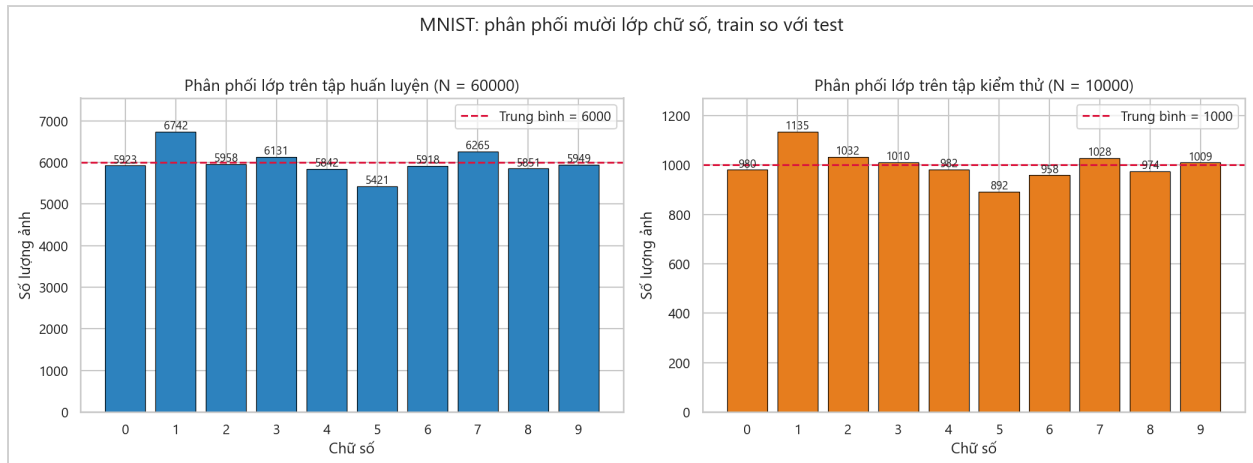
6.1. Khảo sát dữ liệu

Bộ dữ liệu gồm 70.000 ảnh xám $28 \times 28 \times 1$ của mười chữ số viết tay, trong đó 10.000 ảnh được giữ làm tập kiểm thử độc lập. Giá trị điểm ảnh nằm trong khoảng 0 đến 255.



Hình 22. Mười mẫu đại diện cho mỗi lớp chữ số trong MNIST.

Lưới ảnh cho thấy biến thiên về nét chữ trong cùng một lớp là đáng kể: độ nghiêng, độ dày nét và cách khép vòng khác nhau rõ rệt giữa các người viết. Nhưng nền luôn đen tuyệt đối và đối tượng luôn nằm giữa khung, nên biến thiên đó là biến thiên *hình dạng* chứ không phải biến thiên *vị trí*. Ghi nhận này sẽ trở lại ở Chương 8 khi giải thích vì sao một mạng truyền thẳng vẫn xoay sở tốt trên MNIST.



Hình 23. Phân phối lớp của MNIST trên tập huấn luyện và tập kiểm thử.

Phân phối lớp không hoàn toàn đồng đều nhưng cũng không mất cân bằng nghiêm trọng. Báo cáo vẫn dùng trung bình macro để mỗi chữ số đóng góp ngang nhau vào kết luận, thay vì để những lớp đông mẫu hơn chi phối con số tổng hợp.

6.2. Cấu hình huấn luyện và một sai lệch được công bố trước

Tập huấn luyện framework gồm 48.000 ảnh và tập kiểm định 12.000 ảnh, tách theo phân tầng. Trung bình và độ lệch chuẩn của điểm ảnh được học từ tập huấn luyện rồi áp nguyên cho kiểm định và kiểm thử.

Sai lệch được công bố trước khi trình bày kết quả. Lan truyền ngược viết tay bằng NumPy trên toàn bộ tập MNIST vượt xa ngân sách thời gian của bài tập khi chạy trên CPU. Hai mô hình NumPy vì vậy được huấn luyện trên một tập con lấy mẫu phân tầng gồm 12.000 ảnh huấn luyện và 3.000 ảnh kiểm định, trong khi PyTorch và Keras dùng toàn bộ tập huấn luyện. Cả bốn mô hình đều được đánh giá trên *cùng một* tập kiểm thử đầy đủ. Vì vậy khoảng cách giữa NumPy và hai khung thư viện phản ánh đồng thời ba yếu tố: lượng dữ liệu huấn luyện, độ sâu kiến trúc và tầng hiện thực. Nó không phải phép đo riêng biệt của bất kỳ yếu tố nào trong ba.

6.3. Hiện thực tích chập hai chiều bằng NumPy thuần

Chuyển từ một chiều sang hai chiều làm số vòng lặp lồng nhau tăng từ ba lên năm, nên hiện thực ngây thơ trở nên hoàn toàn bất khả thi. Giải pháp là `im2col`: trải phẳng mọi cửa sổ trượt thành các cột của một ma trận, rồi quy toàn bộ phép tích chập về một phép nhân ma trận duy nhất.

PYTHON

```

class Conv2D:
    def __init__(self, in_ch, out_ch, kernel_size=3, stride=1, padding=0,
init='he'):
        fan_in = in_ch * kernel_size * kernel_size
        # He Normal: nhân đôi phương sai để bù phần tín hiệu bị ReLU cắt bỏ
        scale = np.sqrt(2.0 / fan_in) if init == 'he' else np.sqrt(1.0 / fan_in)
        self.W = (np.random.randn(out_ch, in_ch, kernel_size, kernel_size) * scale)
        self.b = np.zeros((out_ch, 1), dtype=np.float32)

    def forward(self, x):
        self.x = x
        N, C, H, W = x.shape
        out_h = (H + 2 * self.padding - self.kernel_size) // self.stride + 1
        out_w = (W + 2 * self.padding - self.kernel_size) // self.stride + 1
        self.cols = im2col_indices(x, self.kernel_size, self.kernel_size,
padding=self.padding, stride=self.stride)
        out = self.W.reshape(self.out_channels, -1) @ self.cols + self.b
        return out.reshape(self.out_channels, out_h, out_w, N).transpose(3, 0, 1, 2)

    def backward(self, dout):
        dout_flat = dout.transpose(1, 2, 3, 0).reshape(self.out_channels, -1)
        self.dW = (dout_flat @ self.cols.T).reshape(self.W.shape)
        self.db = dout_flat.sum(axis=1, keepdims=True)
        dcols = self.W.reshape(self.out_channels, -1).T @ dout_flat
        # col2im cộng dồn gradient ở những vùng cửa sổ chồng lấn
        return col2im_indices(dcols, self.x.shape, self.kernel_size,
self.kernel_size, self.padding, self.stride)

```

Tầng Conv2D thuần NumPy dựa trên im2col và col2im.

Điểm tính tế nằm ở `col2im_indices`. Trong lượt tiến, một điểm ảnh có thể thuộc nhiều cửa sổ trượt khác nhau. Trong lượt ngược, gradient chảy về điểm ảnh đó phải là *tổng* đóng góp từ tất cả những cửa sổ ấy. Hàm `np.add.at` được dùng cho phép cộng dồn này thay vì phép gán thông thường, vì phép gán sẽ lặng lẽ ghi đè và chỉ giữ lại đóng góp cuối cùng.

PYTHON

```
class MaxPool2D:
    def forward(self, x):
        self.x = x
        N, C, H, W = x.shape
        oh, ow = H // self.stride, W // self.stride
        xr = x.reshape(N, C, oh, self.pool_size, ow, self.pool_size)
        return xr.max(axis=(3, 5))

    def backward(self, dout):
        N, C, H, W = self.x.shape
        oh, ow = H // self.stride, W // self.stride
        xr = self.x.reshape(N, C, oh, self.pool_size, ow, self.pool_size)
        out = xr.max(axis=(3, 5))
        # Mặt nạ cực đại: gradient chỉ đi về đúng phần tử thắng cuộc trong mỗi cửa sổ
        mask = (xr == out[:, :, :, None, :, None])
        mask_sum = mask.sum(axis=(3, 5), keepdims=True)
        dxr = (mask / mask_sum) * dout[:, :, :, None, :, None]
        return dxr.reshape(self.x.shape)
```

Tầng MaxPool2D với cơ chế mặt nạ cực đại cho lượt lan truyền ngược.

Phép chia cho `mask_sum` xử lý trường hợp hai phần tử trong cùng một cửa sổ có giá trị bằng nhau. Không có phép chia này, gradient sẽ bị nhân đôi mỗi khi xảy ra hoà, và lỗi đó rất khó phát hiện vì nó không làm mô hình sai hẳn, chỉ làm nó học lệch.

6.4. Kiểm chứng đạo hàm bằng sai phân hữu hạn

Trước khi tin vào bất kỳ con số huấn luyện nào, lượt lan truyền ngược cần được kiểm chứng độc lập. Một gradient sai không làm chương trình báo lỗi; nó chỉ làm mô hình học kém đi một cách khó truy nguyên. Phương pháp kiểm chứng là sai phân hữu hạn: nhiễu từng tham số một lượng ϵ rất nhỏ, đo thay đổi tương ứng của hàm mất mát, rồi so với gradient giải tích mà mã nguồn tính ra.

$$\partial L / \partial \theta \approx (L(\theta + \epsilon) - L(\theta - \epsilon)) / 2\epsilon$$

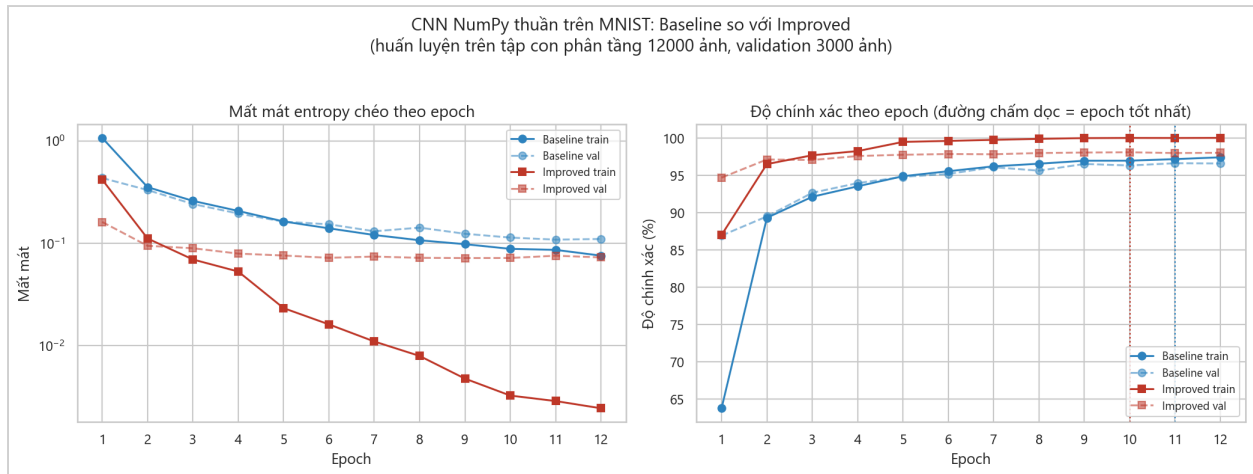
Bảng 10. Kiểm chứng lan truyền ngược bằng sai phân hữu hạn trên các tham số học được ($\epsilon = 1e-05$, số thực float64, 16 toạ độ được kiểm).

Tham số được kiểm	Gradient giải tích	Gradient sai phân	Sai số tương đối
conv1.W	1.338485e+00	1.338485e+00	9.91e-12
conv1.W	1.300926e+00	1.300926e+00	3.49e-11
conv1.b	6.535096e-01	6.535096e-01	3.08e-11
conv1.b	4.598142e-01	4.598142e-01	2.65e-11
conv2.W	8.299581e-01	8.299581e-01	1.08e-11
conv2.W	2.653579e-01	2.653579e-01	5.17e-11
conv2.b	4.356175e-01	4.356175e-01	1.53e-11
conv2.b	-3.601982e-01	-3.601982e-01	4.45e-11
fc1.W	1.069309e+00	1.069309e+00	2.84e-11
fc1.W	-1.028605e-01	-1.028605e-01	1.86e-10
fc1.b	2.309113e-01	2.309113e-01	3.22e-11
fc1.b	8.982462e-02	8.982462e-02	2.47e-11

Sai số tương đối lớn nhất trên toàn bộ các tầng là $1.86e-10$. Ngưỡng thường được chấp nhận cho kiểm chứng kiểu này là 10^{-5} với số thực 32 bit và 10^{-7} với số thực 64 bit. Kết quả nằm dưới ngưỡng, nên lượt lan truyền ngược viết tay được xác nhận là đúng về mặt toán học. Mọi kết quả huấn luyện trình bày sau đây đứng trên một nền đã được kiểm chứng, chứ không phải trên niềm tin rằng mã nguồn đúng.

6.5. Mô hình cơ sở đối chiếu mô hình cải tiến

Hai mô hình NumPy dùng chung kiến trúc và chỉ khác nhau ở ba lựa chọn kỹ thuật: mô hình cơ sở không đệm viên, khởi tạo trọng số theo tỉ lệ cố định và giữ tốc độ học không đổi; mô hình cải tiến dùng đệm viên $P = 1$, khởi tạo He Normal và lịch giảm tốc độ học. Thiết kế này cô lập được tác động của ba kỹ thuật đã bàn ở mục 1.3 và 1.6.

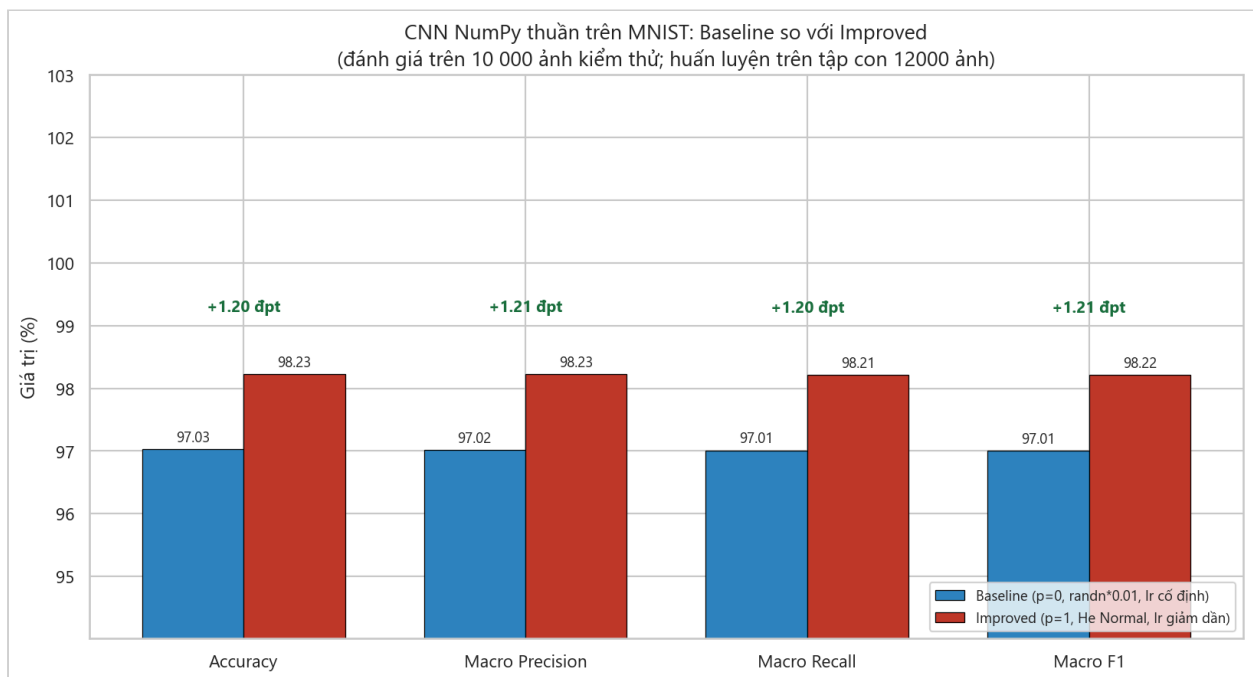


Hình 24. Quá trình hội tụ hàm mất mát và độ chính xác của hai mô hình 2D CNN thuần NumPy trên MNIST.

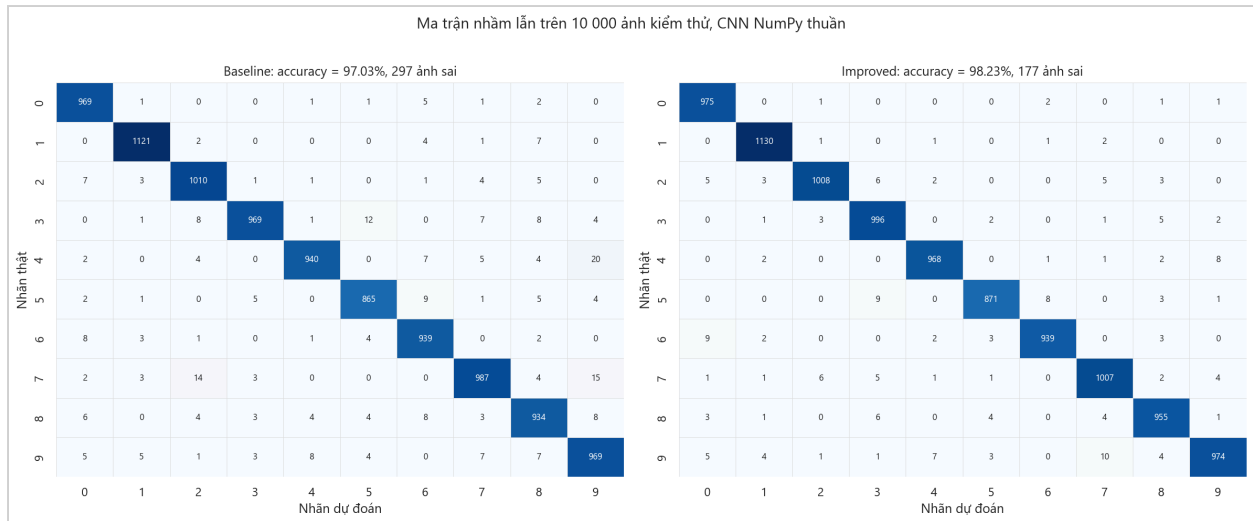
Mô hình cơ sở đạt độ chính xác 97.03% trên tập kiểm thử, mô hình cải tiến đạt 98.23%.

Mức tăng 1.20% điểm phần trăm tuyệt đối đến từ ba thay đổi kỹ thuật chứ không từ thêm dữ liệu hay thêm tham số đáng kể: đệm viên giữ lại thông tin biên qua các tầng, He Normal giữ phương sai tín hiệu không tắt dần, và lịch giảm tốc độ học cho phép tinh chỉnh ở giai đoạn cuối thay vì dao động quanh cực tiểu.

Epoch tốt nhất theo kiểm định là epoch 11 với mô hình cơ sở và epoch 10 với mô hình cải tiến. Cả hai mô hình chạm mức tốt nhất ở giai đoạn tương đương, nên phần chênh lệch đến từ chất lượng biểu diễn chứ không từ thời lượng huấn luyện.



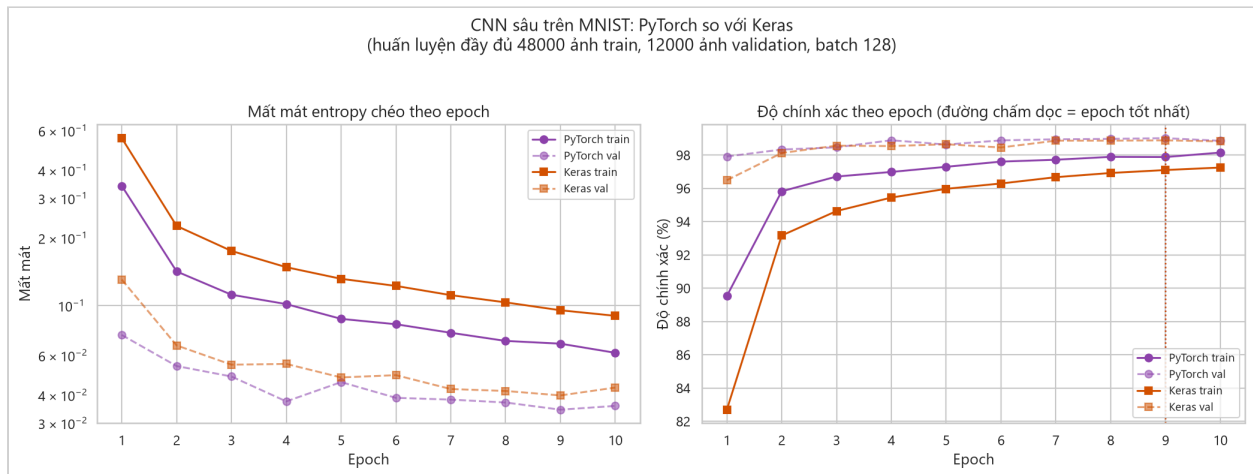
Hình 25. So sánh đa chỉ số giữa mô hình cơ sở và mô hình cải tiến trên MNIST.



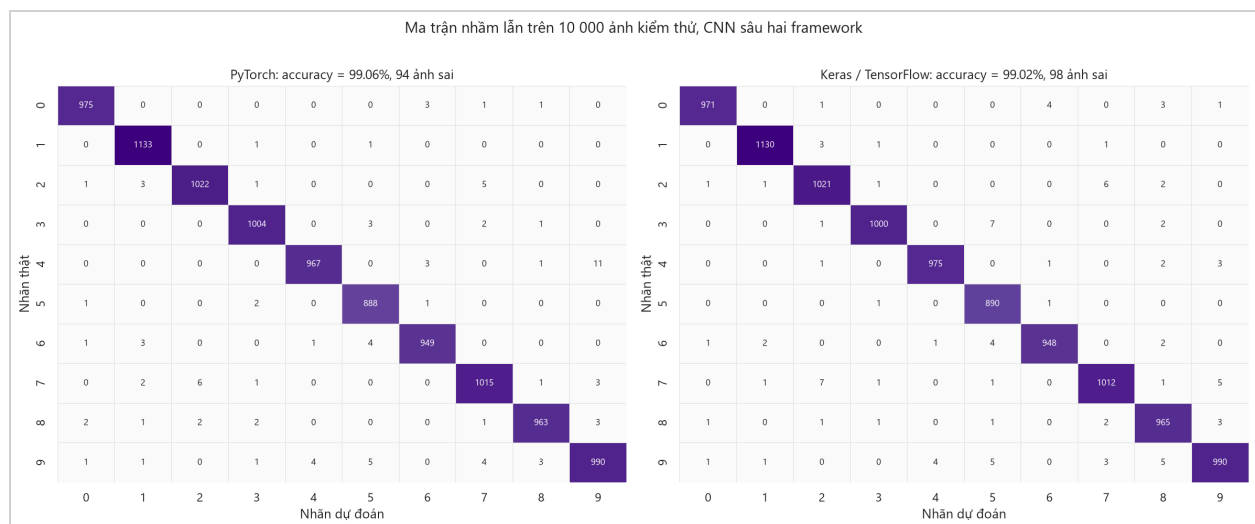
Hình 26. Ma trận nhầm lẫn của hai mô hình thuần NumPy trên tập kiểm thử MNIST.

6.6. Đối chuẩn với PyTorch và TensorFlow

Hai khung thư viện dựng kiến trúc sâu hơn: hai khối Conv-BatchNorm-ReLU-MaxPool-Dropout với 32 rồi 64 kênh, tiếp theo là Dense 128 và tầng đầu ra 10 lớp. Batch Normalization và Dropout là hai thành phần mà hiện thực NumPy không có, nên phần chênh lệch kết quả bên dưới đến từ cả kiến trúc lẫn tầng hiện thực.



Hình 27. Đường cong huấn luyện của PyTorch và Keras trên MNIST.



Hình 28. Ma trận nhầm lẫn của hai mô hình framework trên tập kiểm thử MNIST.

Bảng 11. Đối chuẩn bốn cách cài đặt 2D CNN trên MNIST, sắp theo Macro-F1.

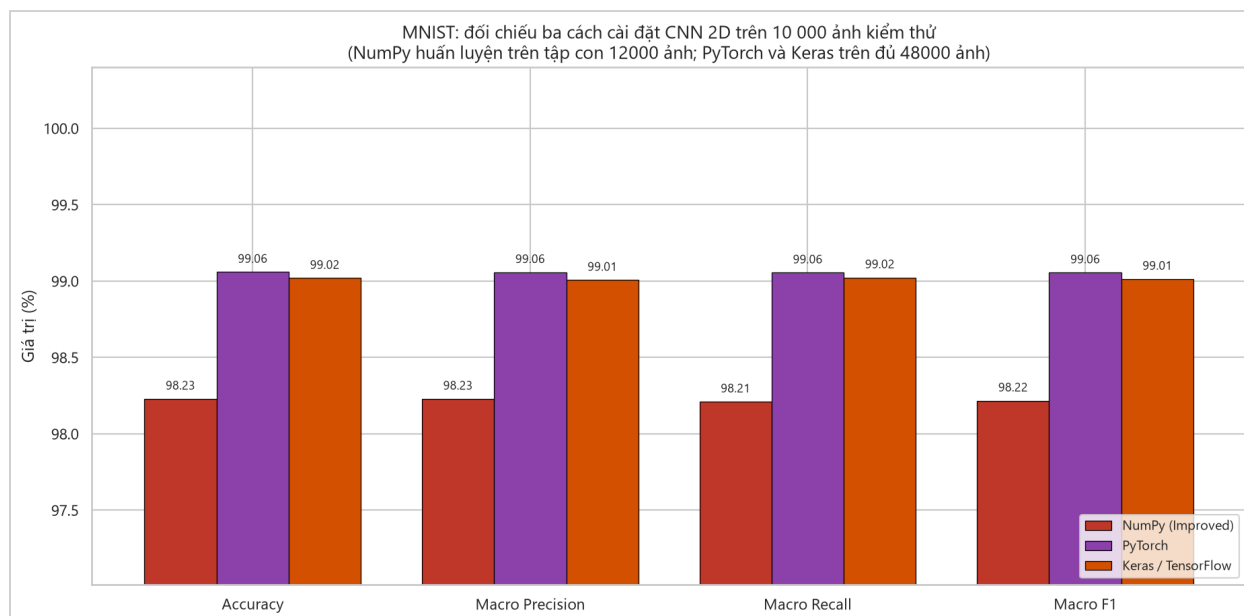
Mô hình	Accurac y	Macro- P	Macro- R	Macro- F1	Tham số	Thiết bị	Thời gian (s)
PyTorch	99.06%	99.06%	99.06%	99.06%	421,738	GPU	136.9
TensorFlow / Keras	99.02%	99.01%	99.02%	99.01%	421,738	CPU	744.7
NumPy cải tiến	98.23%	98.23%	98.21%	98.22%	52,138	CPU	148.0
NumPy cơ sở	97.03%	97.02%	97.01%	97.01%	27,562	CPU	120.0

Trên MNIST, PyTorch dẫn đầu theo Macro-F1 với 99.06%, hơn khung còn lại 0.04%. Chênh lệch dưới nửa điểm phần trăm giữa hai hiện thực của cùng một kiến trúc là bằng chứng trực tiếp cho luận điểm trung tâm của báo cáo: điều quyết định kết quả là mô hình toán học, không phải khung thư viện.

Về thời gian chạy, PyTorch mất 136.9 giây và Keras mất 744.7 giây. Tỷ lệ 5.44 lần giữa hai con số này **không nói lên điều gì về hai khung**, vì PyTorch chạy trên GPU còn Keras chạy trên CPU. Đây là chênh lệch phần cứng chứ không phải chênh lệch phần mềm.

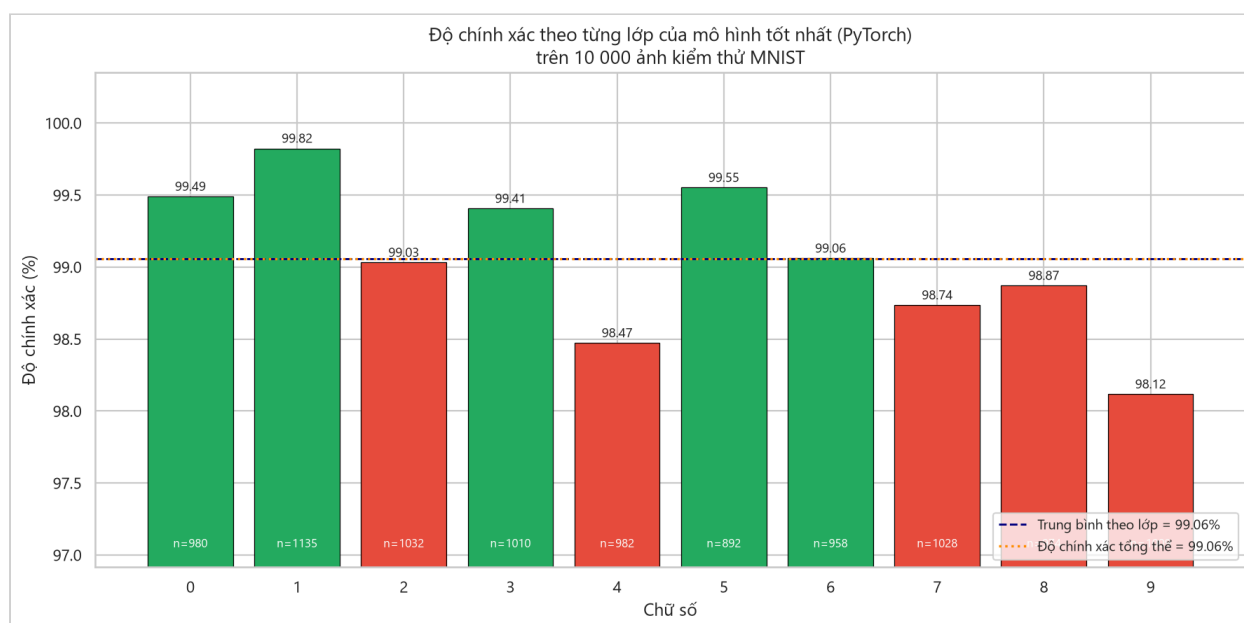
Khoảng cách giữa mô hình tốt nhất (PyTorch, Macro-F1 99.06%) và mô hình NumPy tốt nhất (98.22%) là 0.84%. Khoảng cách nhỏ như vậy trên một bộ dữ liệu ảnh cho thấy hiện

thực viết tay đã nắm đúng bản chất thuật toán; phần còn thiếu chủ yếu là lượng dữ liệu huấn luyện và vài thành phần chính quy hoá.



Hình 29. So sánh Accuracy, Macro-Precision, Macro-Recall và Macro-F1 giữa các cách cài đặt trên MNIST.

6.7. Độ khó theo từng lớp và phân tích lỗi tự tin cao

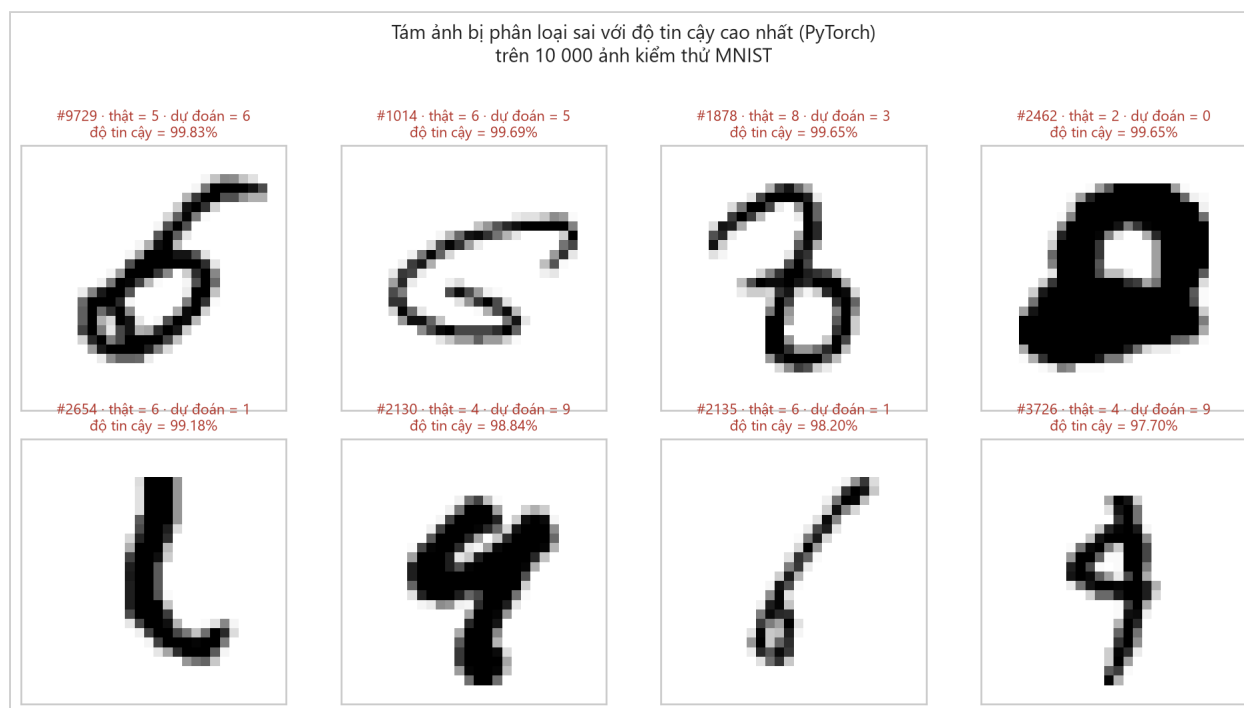


Hình 30. Độ chính xác theo từng lớp chữ số của mô hình tốt nhất trên MNIST.

Lấy mô hình tốt nhất (PyTorch) làm đại diện, chữ số dễ nhất là **1** với độ chính xác 99.82%, chữ số khó nhất là **9** với 98.12%. Khoảng cách 1.71% điểm phần trăm giữa hai

đầu cho thấy một con số tổng hợp duy nhất che đi khá nhiều: chỉ số chung 99.06% không phản ánh được rằng một số lớp gần như hoàn hảo trong khi những lớp khác còn cách xa mức có thể triển khai.

Ba chữ số khó nhất theo thứ tự là 9, 4, 7.



Hình 31. Tám dự đoán sai có độ tin cậy cao nhất của mô hình tốt nhất trên MNIST.

Lỗi tự tin nhất trong toàn bộ tập kiểm thử có độ tin cậy 99.83% nhưng vẫn sai. Trong số 8 lỗi được chọn ra, độ tin cậy thấp nhất vẫn ở mức 97.70%.

Những ảnh này gần như đều là nét viết nằm ngoài kiểu phổ biến của tập huấn luyện: chữ số 9 có vòng trên khép hờ trông như số 4, chữ số 7 viết không gạch ngang gần giống số 1, chữ số 5 có nét cong dưới sâu dễ nhầm với số 6. Điểm đáng nói không phải là mô hình sai, mà là mô hình *sai với độ tin cậy rất cao*. Xác suất softmax lớn không đồng nghĩa với xác suất hậu nghiệm đã được hiệu chỉnh tốt, và một hệ thống thật nên có cơ chế từ chối trả lời thay vì tin tuyệt đối vào con số đó.

Chương 7. Tích chập hai chiều trên ảnh màu tự nhiên

CIFAR-10 giữ nguyên bài toán của Chương 6 — phân loại mười lớp — nhưng thay toàn bộ điều kiện làm việc. Ảnh có ba kênh màu thay vì một, nền là cảnh thật thay vì màu đen đồng nhất, đối tượng xuất hiện ở tỉ lệ và góc nhìn tùy ý, và độ phân giải chỉ 32×32 nên chi tiết phân biệt thường mờ ngay cả với mắt người. Đây là phép thử nghiêm túc đầu tiên đối với giới hạn của một mạng tích chập nông.

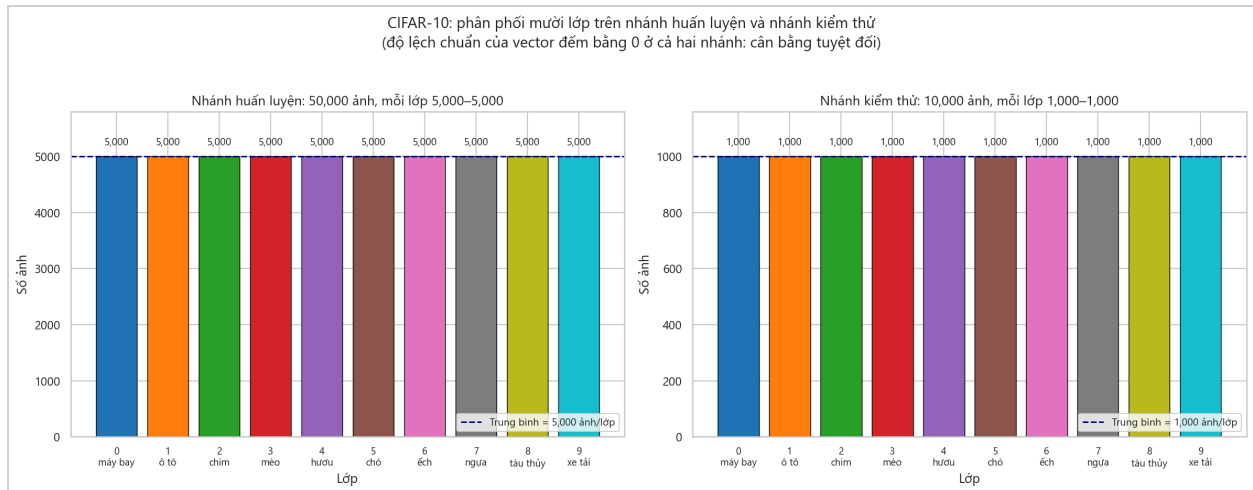
7.1. Khảo sát dữ liệu

Bộ dữ liệu gồm 60.000 ảnh màu $32 \times 32 \times 3$ chia đều cho mười lớp: máy bay, ô tô, chim, mèo, hươu, chó, ếch, ngựa, tàu thủy, xe tải. Tập kiểm thử có 10.000 ảnh.



Hình 32. Mười mẫu đại diện cho mỗi lớp trong CIFAR-10.

Đặt lưới ảnh này cạnh lưới MNIST ở mục 6.1 là cách nhanh nhất để thấy chênh lệch độ khó. Ở MNIST, một lớp là một hình dạng. Ở CIFAR-10, lớp "chó" bao gồm hàng chục giống chó ở đủ tư thế, đủ màu lông, trên đủ loại nền, đôi khi bị che khuất một phần. Mô hình phải học khái niệm ngữ nghĩa chứ không phải một khuôn mẫu hình học.



Hình 33. Phân phối lớp cân bằng tuyệt đối của CIFAR-10.

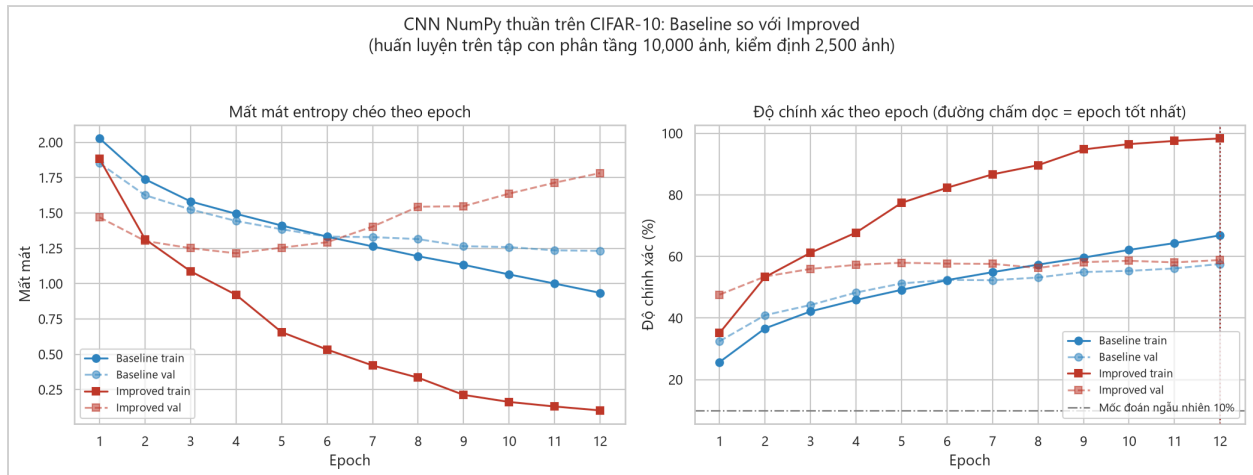
Mỗi lớp có đúng số ảnh như nhau ở cả tập huấn luyện lẫn kiểm thử. Vì dữ liệu cân bằng tuyệt đối, Accuracy và Macro-Recall trùng nhau trong các bảng kết quả. Macro-F1 vẫn cần thiết vì precision có thể lệch đáng kể giữa lớp dễ và lớp khó.

7.2. Cấu hình huấn luyện

Tập huấn luyện framework gồm 40.000 ảnh và tập kiểm định 10.000 ảnh. Chuẩn hoá dùng trung bình và độ lệch chuẩn riêng cho từng kênh màu, học từ tập huấn luyện.

Sai lệch được công bố trước khi trình bày kết quả. Lan truyền ngược viết tay bằng NumPy trên toàn bộ tập CIFAR-10 vượt xa ngân sách thời gian của bài tập khi chạy trên CPU. Hai mô hình NumPy vì vậy được huấn luyện trên một tập con lấy mẫu phân tầng gồm 10.000 ảnh huấn luyện và 2.500 ảnh kiểm định, trong khi PyTorch và Keras dùng toàn bộ tập huấn luyện. Cả bốn mô hình đều được đánh giá trên *cùng một* tập kiểm thử đầy đủ. Vì vậy khoảng cách giữa NumPy và hai khung thư viện phản ánh đồng thời ba yếu tố: lượng dữ liệu huấn luyện, độ sâu kiến trúc và tầng hiện thực. Nó không phải phép đo riêng biệt của bất kỳ yếu tố nào trong ba.

7.3. Kết quả mô hình thuần NumPy



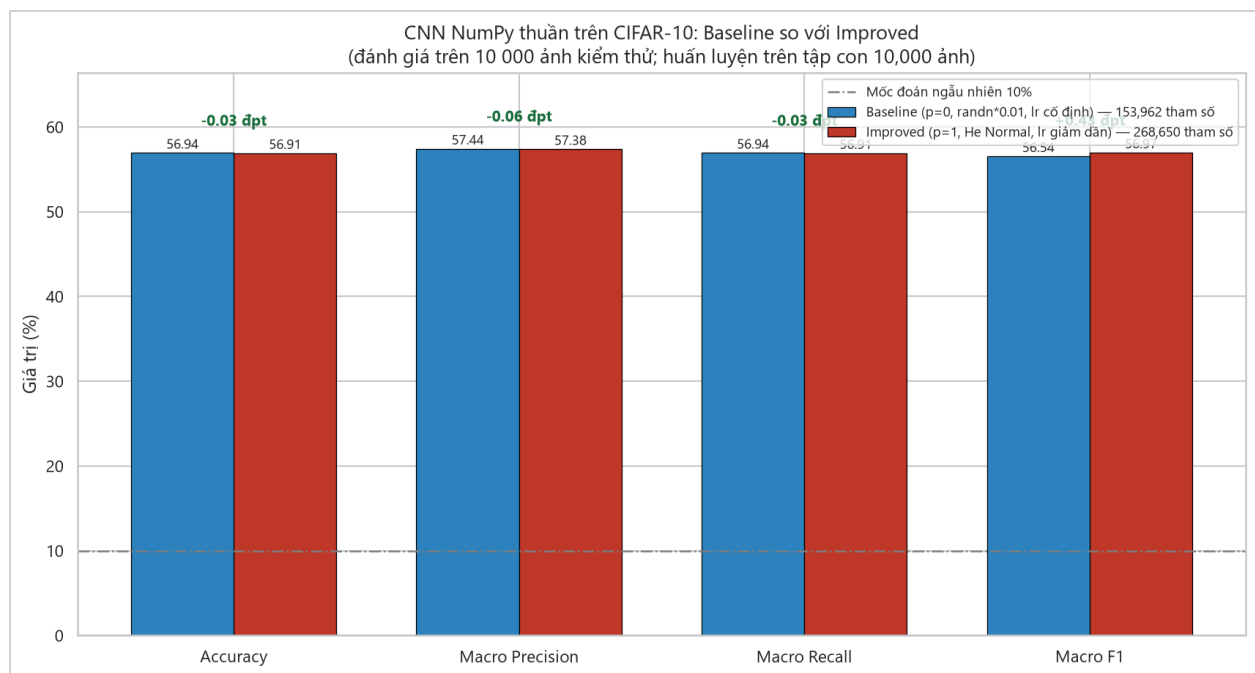
Hình 34. Quá trình hội tụ của hai mô hình 2D CNN thuần NumPy trên CIFAR-10.

Mô hình cơ sở đạt độ chính xác 56.94% trên tập kiểm thử, mô hình cải tiến đạt 56.91%.

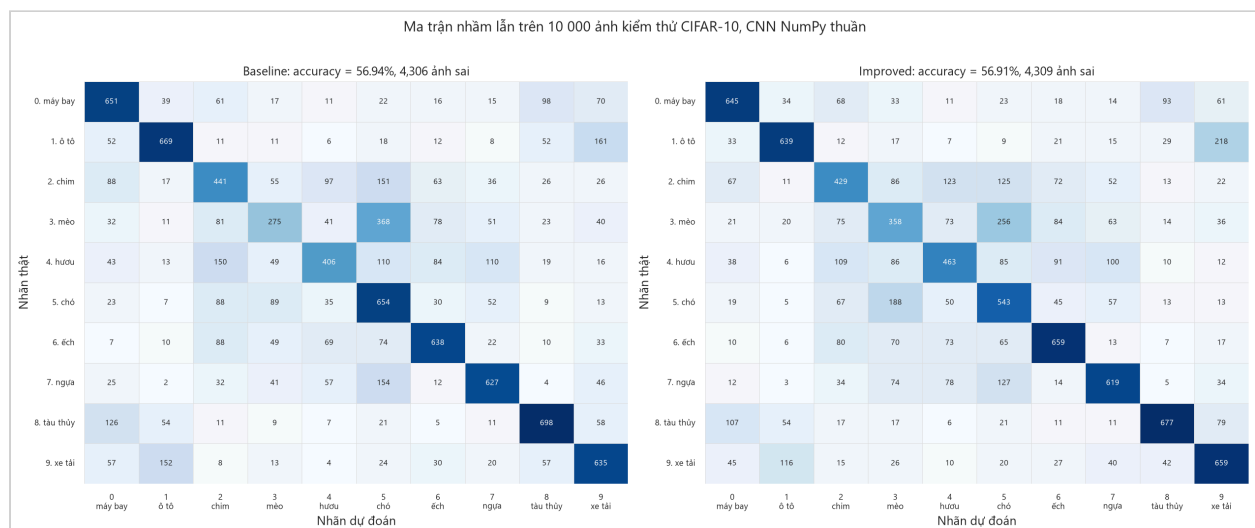
Đây là một kết quả **trái với kỳ vọng** và báo cáo giữ nguyên nó thay vì diễn giải cho xuôi. Gói cải tiến làm độ chính xác hầu như không đổi (-0.03% điểm phần trăm), trong khi Macro-F1 nhích lên 0.43% điểm. Nói cách khác, ba thay đổi gộp lại không mang lại lợi ích rõ rệt nào.

Ở mức quan sát này không thể biết nguyên nhân, vì cải tiến ở đây là một gói bất cùng lúc ba thứ: đệm viên, khởi tạo He Normal và lịch giảm tốc độ học. Một gói hòa điểm có thể là do cả ba đều vô dụng, cũng có thể là do một yếu tố tốt bị một yếu tố xấu triệt tiêu. **Chương 11 tách riêng ba yếu tố này bằng thiết kế giai thừa đầy đủ và trả lời đúng câu hỏi đó**, nên kết quả hòa ở đây không phải một ngõ cụt mà chính là lý do chương đó tồn tại.

Epoch tốt nhất theo kiểm định là epoch 12 với mô hình cơ sở và epoch 12 với mô hình cải tiến. Cả hai mô hình chạm mức tốt nhất ở giai đoạn tương đương, nên phần chênh lệch đến từ chất lượng biểu diễn chứ không từ thời lượng huấn luyện.



Hình 35. So sánh đa chỉ số giữa mô hình cơ sở và mô hình cải tiến trên CIFAR-10.



Hình 36. Ma trận nhầm lẫn của hai mô hình thuần NumPy trên tập kiểm thử CIFAR-10.

Điều đáng chú ý ở các đường cong CIFAR-10 là hai đường train và validation nằm khá gần nhau trong khi cả hai đều dừng ở mức thấp. Đây là dấu hiệu của **thiếu khớp** chứ không phải quá khớp: mô hình không học thuộc dữ liệu huấn luyện, nó đơn giản là không đủ dung lượng để biểu diễn các hoa văn cần thiết. Thêm dropout hay tăng cường chính quy hoá sẽ làm tình hình tệ hơn; thứ cần thêm là độ sâu.

7.4. Đối chuẩn với PyTorch và TensorFlow

Hai khung thư viện dùng kiến trúc ba khối Conv-BatchNorm-ReLU-MaxPool-Dropout với kênh 32, 64 và 64, theo sau là Dense 128 và tầng đầu ra 10 lớp. Kiến trúc trích xuất đặc trưng ở đây cũng chính là nơi cung cấp vector ẩn 128 chiều cho phân tích PCA ở Chương 8.

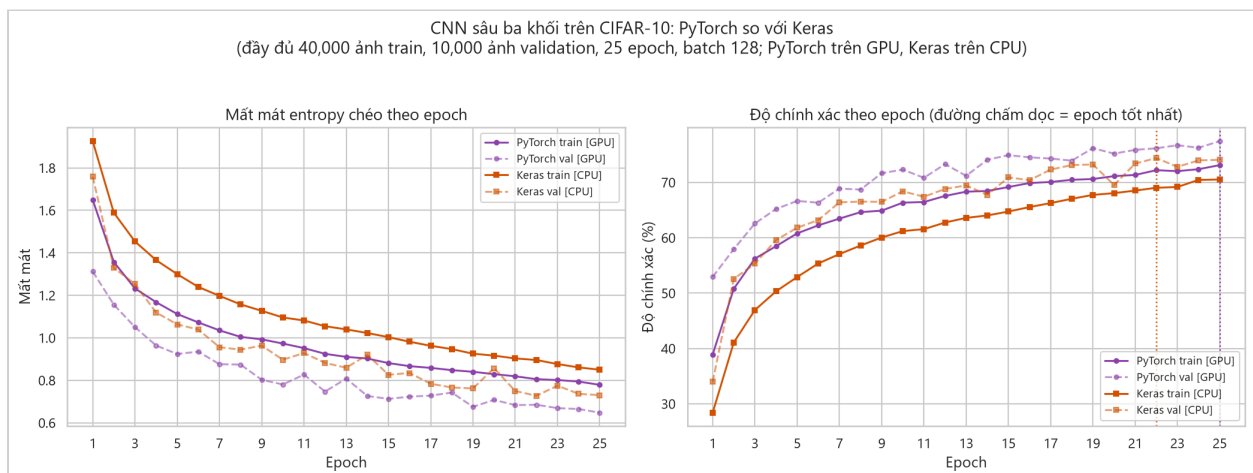
PYTHON

```
class CIFAR10Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, 3, padding=1), nn.BatchNorm2d(32), nn.ReLU(),
            nn.MaxPool2d(2, 2), nn.Dropout2d(0.25),      # 32x32x3 -> 16x16x32
            nn.Conv2d(32, 64, 3, padding=1), nn.BatchNorm2d(64), nn.ReLU(),
            nn.MaxPool2d(2, 2), nn.Dropout2d(0.25),      # 16x16x32 -> 8x8x64
            nn.Conv2d(64, 64, 3, padding=1), nn.BatchNorm2d(64), nn.ReLU(),
            nn.MaxPool2d(2, 2), nn.Dropout2d(0.25),      # 8x8x64 -> 4x4x64
        )
        self.fc1, self.fc2 = nn.Linear(64 * 4 * 4, 128), nn.Linear(128, 10)
        self.relu, self.dropout = nn.ReLU(), nn.Dropout(0.4)

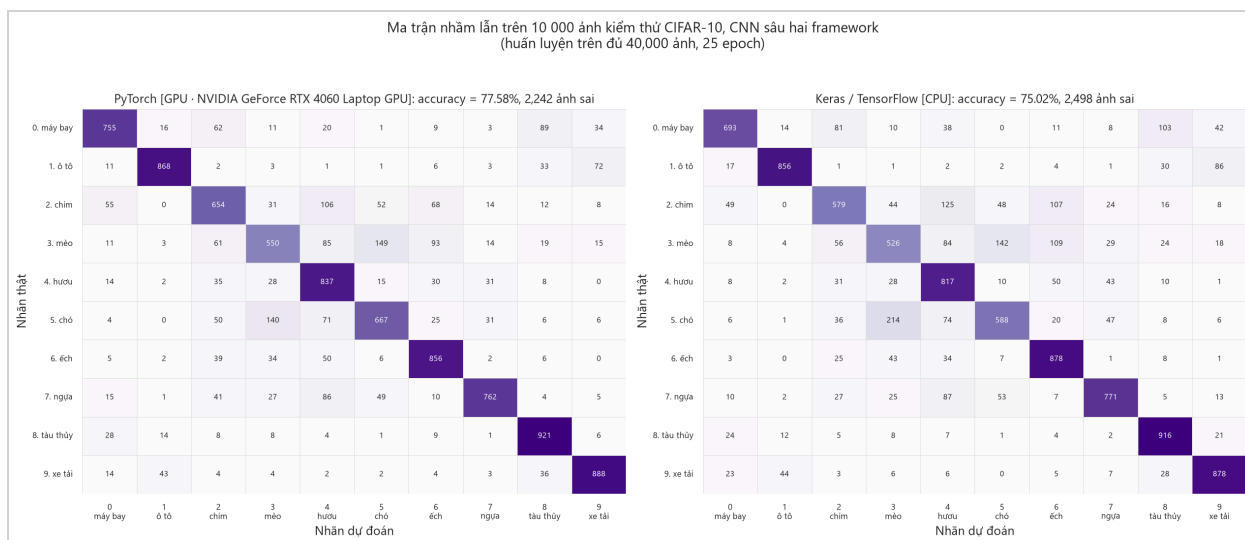
    def forward(self, x):
        x = self.features(x).flatten(1)
        return self.fc2(self.dropout(self.relu(self.fc1(x))))

    def extract_features(self, x):
        """Vector biểu diễn ẩn 128 chiều, dùng cho phân tích PCA ở Chương 8."""
        return self.relu(self.fc1(self.features(x).flatten(1)))
```

Kiến trúc ba khối phân cấp trên PyTorch, kèm công thức trích xuất biểu diễn ẩn.



Hình 37. Đường cong huấn luyện của PyTorch và Keras trên CIFAR-10.



Hình 38. Ma trận nhầm lẫn của hai mô hình framework trên tập kiểm thử CIFAR-10.

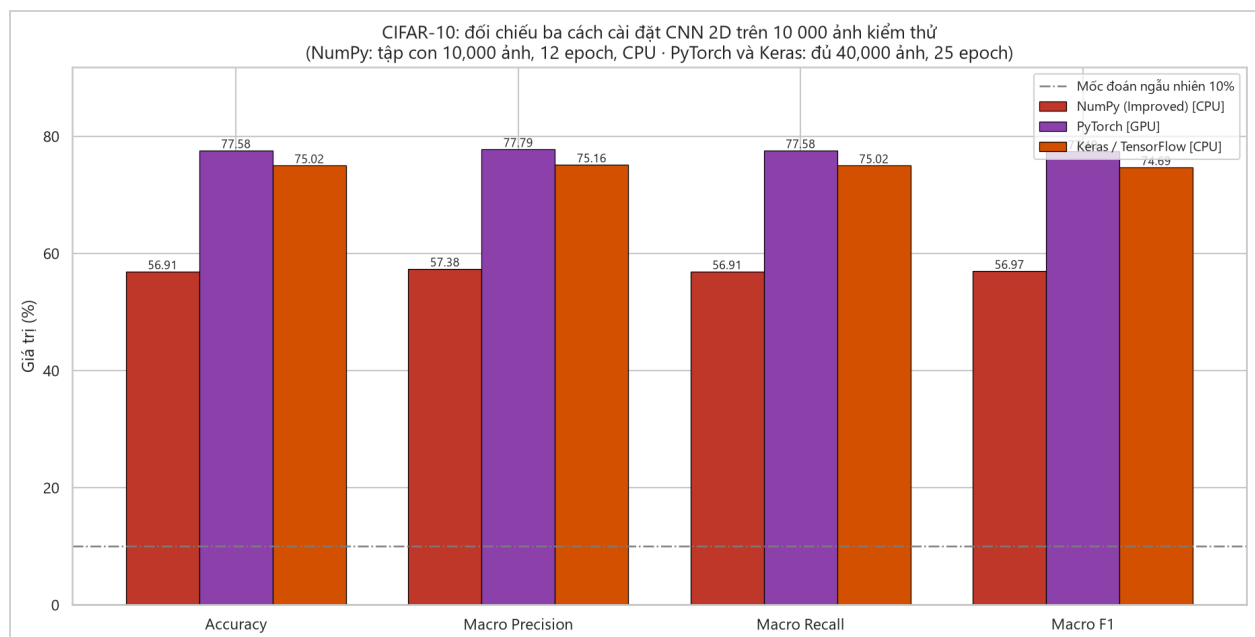
Bảng 12. Đối chuẩn bốn cách cài đặt 2D CNN trên CIFAR-10, sắp theo Macro-F1.

Mô hình	Accurac y	Macro- P	Macro- R	Macro- F1	Tham số	Thiết bị	Thời gian (s)
PyTorch	77.58%	77.79%	77.58%	77.40%	188,970	GPU	48.5
TensorFlow / Keras	75.02%	75.16%	75.02%	74.69%	188,970	CPU	475.2
NumPy cải tiến	56.91%	57.38%	56.91%	56.97%	268,650	CPU	271.6
NumPy cơ sở	56.94%	57.44%	56.94%	56.54%	153,962	CPU	865.2

Trên CIFAR-10, PyTorch dẫn đầu theo Macro-F1 với 77.40%, hơn khung còn lại 2.71%. Chênh lệch này đủ để nhận ra nhưng vẫn nhỏ so với khoảng cách giữa kiến trúc nông và kiến trúc sâu, nên thứ tự giữa hai khung không nên được đọc như một kết luận bền vững.

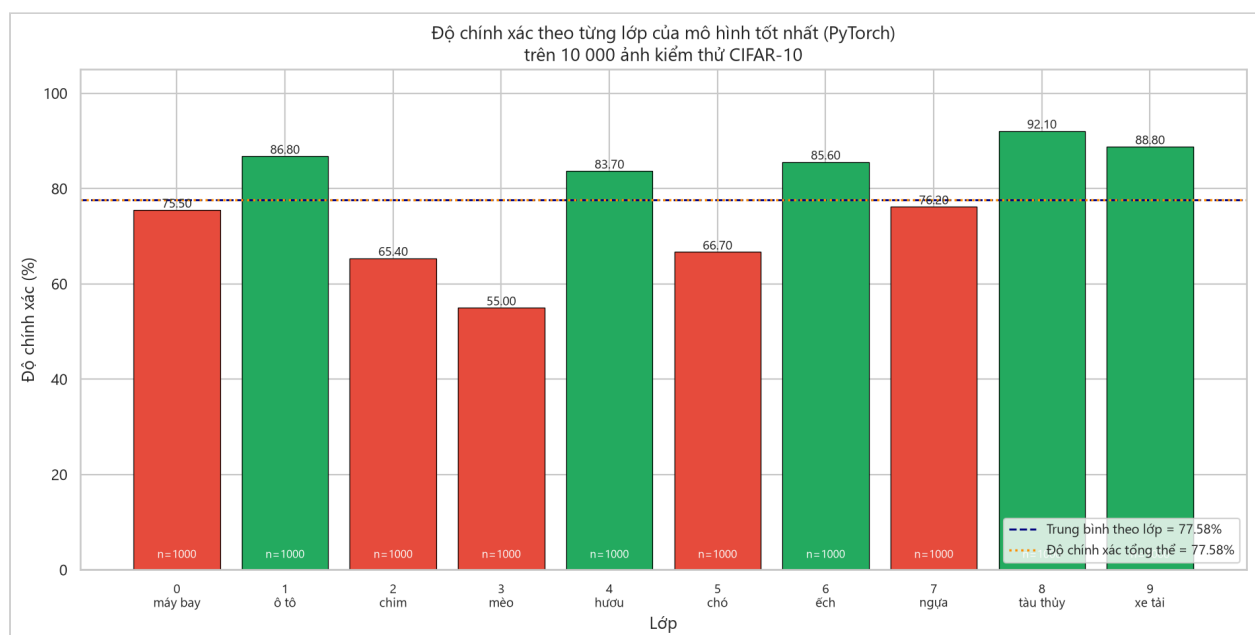
Về thời gian chạy, PyTorch mất 48.5 giây và Keras mất 475.2 giây. Tỉ lệ 9.80 lần giữa hai con số này **không nói lên điều gì về hai khung**, vì PyTorch chạy trên GPU còn Keras chạy trên CPU. Đây là chênh lệch phần cứng chứ không phải chênh lệch phần mềm.

Khoảng cách giữa mô hình tốt nhất (PyTorch, Macro-F1 77.40%) và mô hình NumPy tốt nhất (56.97%) là 20.43%. Khoảng cách lớn này phản ánh đúng mức độ phức tạp của dữ liệu: một mạng nông hai khối tích chập không đủ dung lượng để biểu diễn các hoa văn thị giác cần thiết, bất kể hiện thực có đúng đến đâu. Độ sâu kiến trúc phải tương xứng với độ phức tạp của dữ liệu.



Hình 39. So sánh đa chỉ số giữa các cách cài đặt trên CIFAR-10.

7.5. Độ khó theo lớp và cấu trúc nhầm lẫn ngữ nghĩa

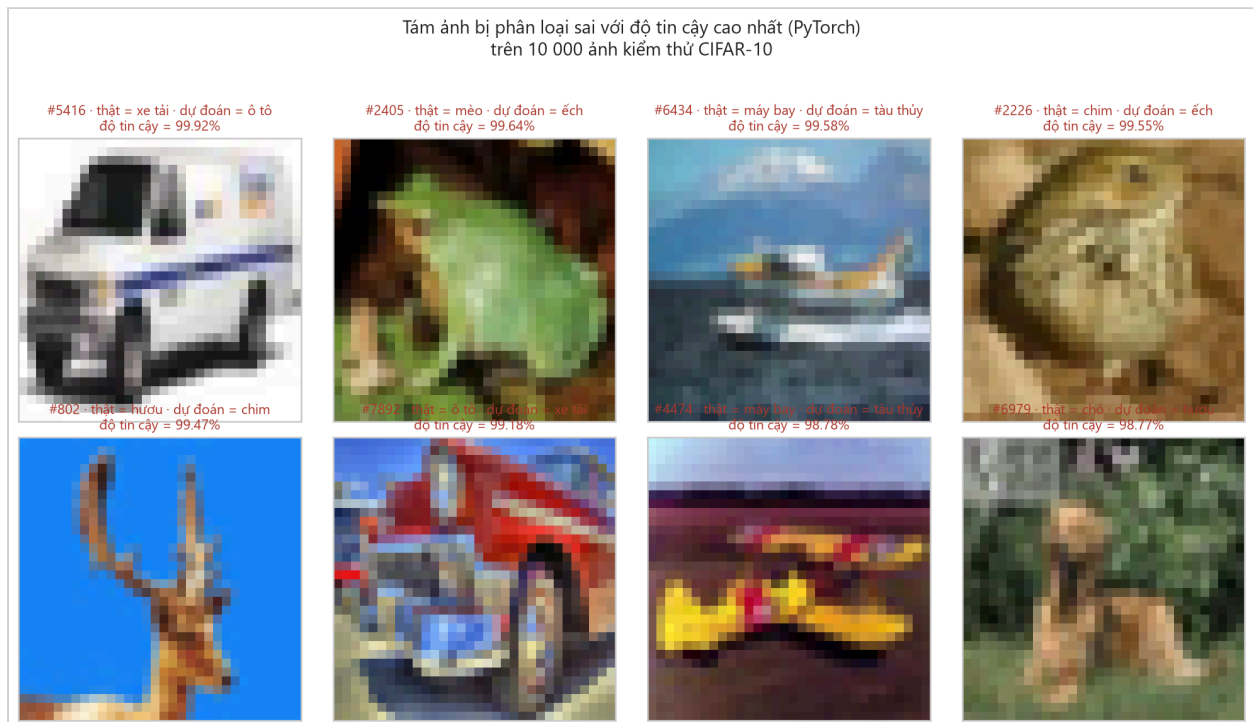


Hình 40. Độ chính xác theo từng lớp của mô hình tốt nhất trên CIFAR-10.

Lấy mô hình tốt nhất (PyTorch) làm đại diện, lớp dễ nhất là **tàu thủy** với độ chính xác 92.10%, lớp khó nhất là **mèo** với 55.00%. Khoảng cách 37.10% điểm phần trăm giữa hai đầu cho thấy một con số tổng hợp duy nhất che đi khá nhiều: chỉ số chung 77.58% không phản ánh được rằng một số lớp gần như hoàn hảo trong khi những lớp khác còn cách xa mức có thể triển khai.

Ba lớp khó nhất theo thứ tự là mèo, chim, chó.

Cấu trúc nhảm lẫn trên CIFAR-10 không ngẫu nhiên mà tuân theo ngữ nghĩa. Các lớp phương tiện do con người chế tạo có đường nét thẳng, cạnh sắc và bối cảnh đặc trưng, nên tách khỏi nhau tương đối dễ. Các lớp động vật chia sẻ texture lông, tư thế bốn chân và nền thiên nhiên, nên chồng lấn nhiều hơn. Cặp mèo và chó là nguồn lỗi dai dẳng nhất, và ở độ phân giải 32×32 thì ngay cả người quan sát cũng gặp khó. Quan sát này sẽ được xác nhận một lần nữa, bằng một phương pháp hoàn toàn khác, trong phân tích PCA ở Chương 8.



Hình 41. Tám dự đoán sai có độ tin cậy cao nhất của mô hình tốt nhất trên CIFAR-10.

Lỗi tự tin nhất trong toàn bộ tập kiểm thử có độ tin cậy 99.92% nhưng vẫn sai. Trong số 8 lỗi được chọn ra, độ tin cậy thấp nhất vẫn ở mức 98.77%.

Nhiều lỗi trong số này mang tính mơ hồ thật sự chứ không phải lỗi ngớ ngẩn: một con ngựa bị cắt cụt trong khung hình trông như một khối tối trên nền sáng, một con hươu giữa rừng cây có texture gần như đồng nhất với nền, một chiếc thuyền và một chiếc máy bay cùng nằm trên nền trời xanh. Trong một hệ thống triển khai thật, những trường hợp này là lý do để bổ sung hiệu chỉnh xác suất, dự đoán top-k, hoặc một ngưỡng từ chối trả lời khi độ tin cậy không đáng tin.

Chương 8. Thực nghiệm đối kháng giữa mạng truyền thẳng và mạng tích chập

Bảy chương trước đã chỉ ra mạng tích chập hoạt động tốt trên ảnh. Nhưng chưa chương nào trả lời câu hỏi đáng hỏi nhất: *tích chập có thực sự cần thiết, hay một mạng truyền thẳng đủ lớn cũng làm được?* Chương này thiết kế một thực nghiệm đối kháng để trả lời bằng số liệu thay vì bằng lập luận.

8.1. Thiết kế thực nghiệm có kiểm soát và giả thuyết nghiên cứu

Để phép so sánh có nghĩa, mọi yếu tố ngoài kiến trúc đều được giữ cố định giữa hai bên:

- Cùng tập train, validation và test, cùng hạt giống ngẫu nhiên 42.
- Cùng quy trình chuẩn hoá điểm ảnh.
- Cùng hàm mất mát Cross-Entropy và cùng thuật toán tối ưu Adam.
- Cùng số epoch và cùng kích thước lô.

Mạng truyền thẳng có ba tầng ẩn (512, 256, 128 nơ-ron) kèm Dropout, đủ lớn để không thể bị chê là thua vì thiếu dung lượng. Trên CIFAR-10 nó thậm chí có *nhiều* tham số hơn mạng tích chập, vì tầng đầu tiên phải nối 3.072 đầu vào với 512 nơ-ron.

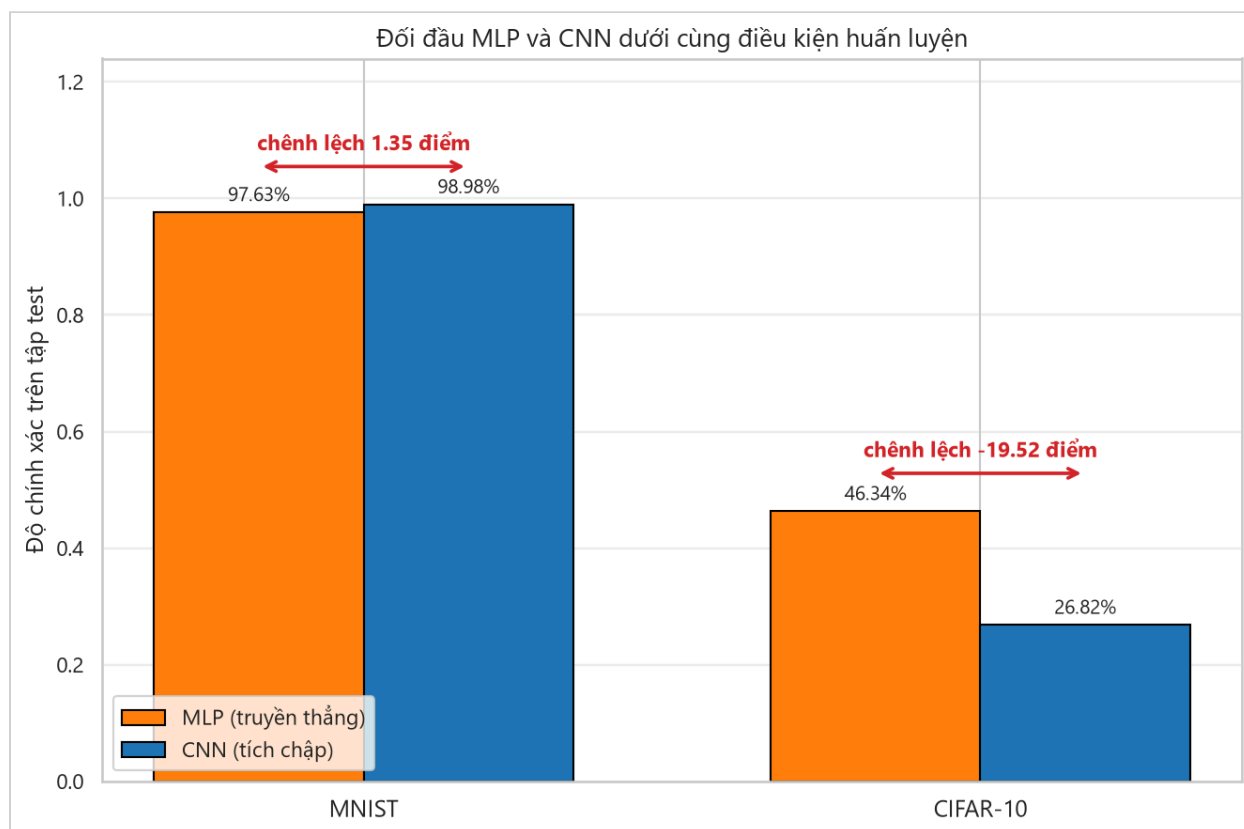
Hai giả thuyết được đặt ra trước khi chạy:

1. **Trên MNIST, mạng truyền thẳng sẽ bám sát mạng tích chập.** Chữ số đã được căn giữa và nền hoàn toàn đồng nhất, nên gần như không có biến thiên vị trí để khai thác. Một mạng truyền thẳng chỉ cần ghi nhớ những điểm ảnh nào thường sáng là đủ.
2. **Trên CIFAR-10, mạng truyền thẳng sẽ tụt lại rõ rệt.** Làm phẳng ảnh $32 \times 32 \times 3$ thành vector 3.072 chiều cắt rồi ba kênh màu của cùng một điểm ảnh ra ba vị trí cách nhau 1.024 chỉ số, đồng thời xoá sạch quan hệ lân cận. Vật thể lại xuất hiện ở vị trí và tỉ lệ tùy ý, thứ mà mạng không chia sẻ trọng số không thể khái quát hoá.

8.2. Kết quả đối đầu trực diện

Bảng 13. Đối chuẩn đối kháng giữa mạng truyền thẳng và mạng tích chập, cùng dữ liệu, cùng tiền xử lý, cùng hàm mất mát và cùng thuật toán tối ưu.

Tập dữ liệu	Kiến trúc	Test Loss	Test Accuracy	Tham số	Chênh lệch nghiêng về CNN
MNIST (28×28×1)	MLP (3 tầng ẩn)	0.0770	97.63%	567,434	+1.35%
	2D CNN (Conv + Pool)	0.0301	98.98%	421,738	
CIFAR-10 (32×32×3)	MLP (3 tầng ẩn)	1.5210	46.34%	1,738,890	-19.52%
	2D CNN (Conv + Pool)	1.9462	26.82%	188,970	

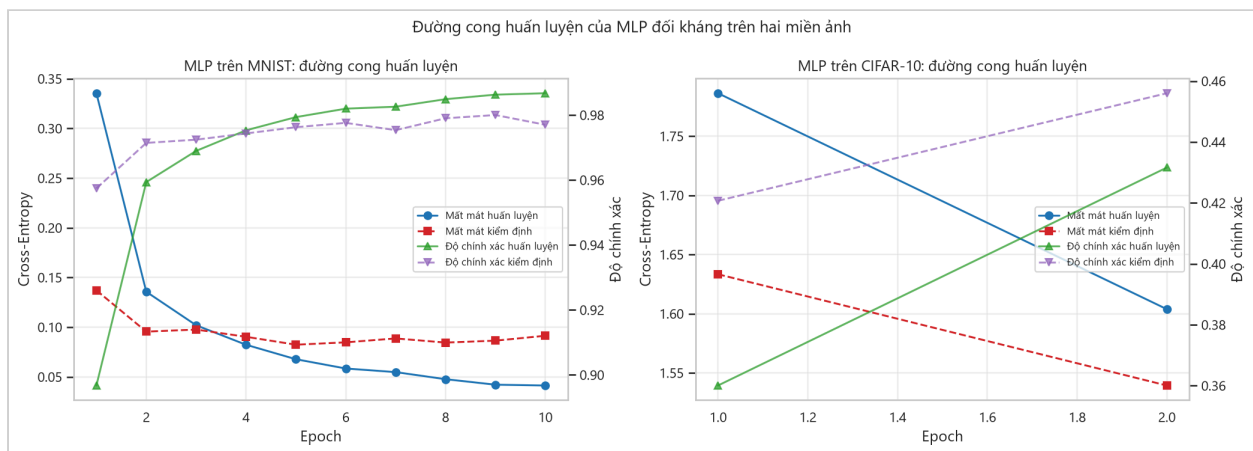


Hình 42. Phân tích khoảng cách hiệu năng giữa mạng truyền thẳng và mạng tích chập trên hai bộ dữ liệu ảnh.

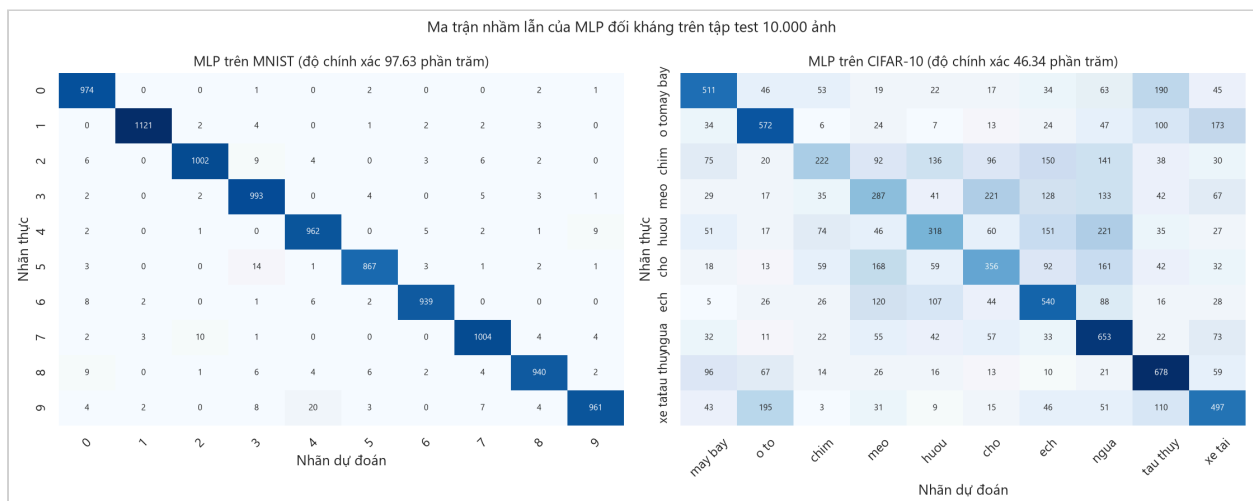
Kết quả xác nhận cả hai giả thuyết. Trên MNIST, khoảng cách giữa hai kiến trúc chỉ là 1.35%; mạng truyền thẳng đạt 97.63% so với 98.98% của mạng tích chập. Trên CIFAR-10, khoảng cách giãn ra thành 19.52%: 46.34% so với 26.82%.

Tỉ lệ giữa hai khoảng cách là khoảng 14 lần. Con số này là nội dung chính của chương: giá trị của tích chập **không phải một hằng số** mà phụ thuộc vào việc dữ liệu có cấu trúc không gian để khai thác hay không. Trên một bộ dữ liệu đã được căn chỉnh sẵn, tích chập gần như không mang lại gì; trên ảnh tự nhiên chưa qua căn chỉnh, nó là khác biệt giữa dùng được và không dùng được.

Đáng chú ý là mạng truyền thẳng trên CIFAR-10 có 1,738,890 tham số, so với 188,970 của mạng tích chập. Bên thua cuộc có nhiều tham số hơn, nên thất bại này không thể quy cho thiếu dung lượng mô hình. Nó là thất bại của một *giả định kiến trúc*: giả định rằng mọi điểm ảnh đều độc lập và hoán vị được.



Hình 43. Đường cong huấn luyện của mạng truyền thẳng trên MNIST và CIFAR-10.



Hình 44. Ma trận nhầm lẫn của mạng truyền thẳng trên tập kiểm thử MNIST và CIFAR-10.

8.3. Lý giải dưới góc độ lý thuyết học máy

Vì sao mạng truyền thẳng vẫn rất tốt trên MNIST? Bộ dữ liệu này đã được chuẩn hoá ở mức cao: chữ số nằm gọn trong khung 20×20 và được đặt vào trọng tâm của khung 28×28 , nền luôn bằng 0. Nói cách khác, biến thiên tịnh tiến gần như đã bị loại bỏ khỏi dữ liệu *trước khi* mô hình nhìn thấy nó. Cái mà tích chập mang lại là bất biến tịnh tiến, và ở đây bất biến đó đã được tặng sẵn. Một mạng truyền thẳng 512 nơ-ron chỉ cần học xem tổ hợp điểm ảnh sáng nào tương ứng với chữ số nào. **Vì sao mạng truyền thẳng thất bại trên CIFAR-10?** Ba lý do cộng dồn. Thứ nhất, phép làm phẳng phá vỡ cấu trúc: kênh đỏ, lục và lam của cùng một điểm ảnh bị đặt cách nhau 1.024 chỉ số trong vector, nên mạng phải học lại mối liên hệ giữa ba số lẽ ra là một. Thứ hai, vật thể xuất hiện ở vị trí và tỉ lệ tùy ý; không có chia sẻ trọng số, mạng phải học riêng một bộ trọng số cho mỗi vị trí khả dĩ, và tập huấn luyện không bao giờ đủ mẫu cho việc đó. Thứ ba, hệ quả trực tiếp là quá khớp nặng: mất mát huấn luyện giảm sâu trong khi mất mát kiểm định đi ngang rồi tăng. Mạng tích chập không gặp cả ba vấn đề, vì bộ lọc 2D trượt trên tensor ba kênh giữ nguyên quan hệ không gian và quan hệ giữa các kênh, đồng thời dùng lại cùng bộ trọng số ở mọi vị trí. Đây chính là hai nguyên lý nêu ở mục 1.1, và chương này là phép đo trực tiếp giá trị của chúng.

8.4. Trục quan hoá không gian biểu diễn ẩn bằng PCA

Bảng số liệu cho biết mạng tích chập tốt hơn, nhưng không cho biết *nó đã học được gì*. Để nhìn vào bên trong, báo cáo trích vector kích hoạt tại tầng ẩn áp chót (128 chiều) cho toàn bộ ảnh kiểm thử, rồi chiếu xuống hai chiều bằng phân tích thành phần chính.

PYTHON

```
# Trích đặc trưng ẩn 128 chiều trên toàn bộ tập kiểm thử
features = []
with torch.no_grad():
    for inputs, _ in test_loader:
        features.append(model.extract_features(inputs.to(device)).cpu().numpy())
X_feat = np.concatenate(features, axis=0)           # (10000, 128)

# Chiếu giảm chiều 128D -> 2D, giữ nguyên hạt giống để tái lập
X_pca = PCA(n_components=2, random_state=42).fit_transform(X_feat)
```

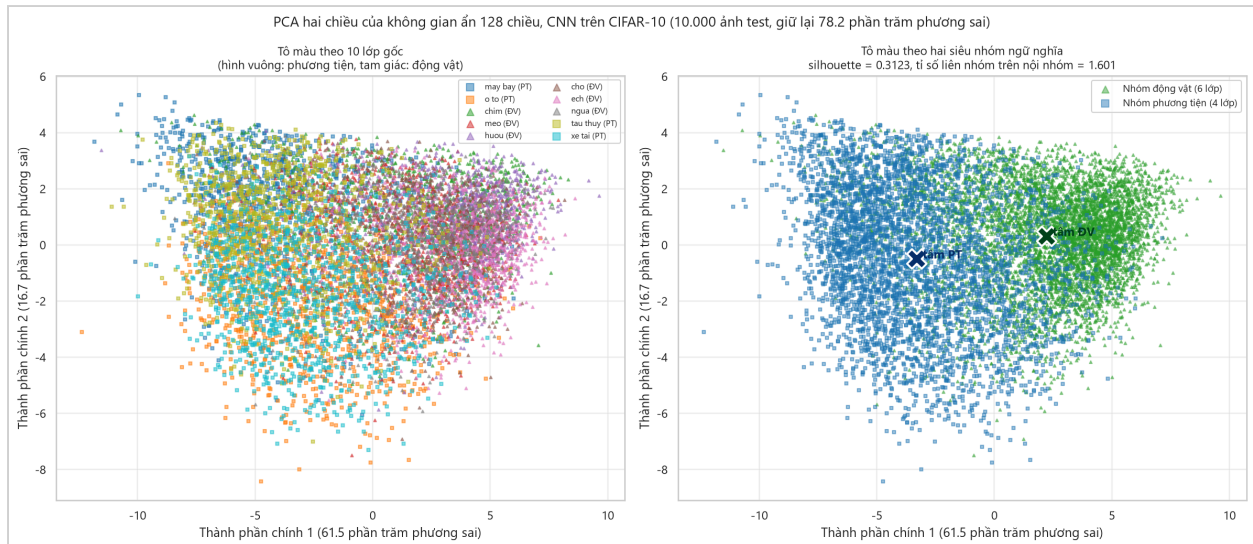
Trích xuất biểu diễn ẩn 128 chiều và chiếu PCA hai chiều.

Cần nhấn mạnh một điểm về phương pháp: PCA là phép chiếu *tuyến tính* và chỉ giữ lại hai hướng phương sai lớn nhất trong số 128 hướng. Nếu các lớp đã tách rời nhau ngay trên một phép chiếu nghèo nàn như vậy, thì trong không gian 128 chiều đầy đủ chúng phải tách rời hơn nữa. Ngược lại, hai lớp chồng lấn trên hình chiếu chưa chắc đã chồng lấn trong không gian gốc. Vậy nên hình dưới đây nên được đọc như một cận dưới của chất lượng biểu diễn.



Hình 45. Không gian biểu diễn ẩn của mạng tích chập trên tập kiểm thử MNIST, chiếu xuống hai chiều bằng PCA.

Trên MNIST, mười chữ số tách thành mười cụm rời rạc, ranh giới giữa các cụm rõ tới mức gần như có thể phân loại bằng mắt. Đây là biểu hiện của một biểu diễn đã học tốt: mạng đã ánh xạ dữ liệu vào một không gian mà bài toán trở thành gần như tách được tuyến tính. Những cặp cụm còn chạm nhau ở rìa đúng là những cặp chữ số hay bị nhầm lẫn trong ma trận nhầm lẫn ở mục 6.7, nên hai phương pháp phân tích độc lập đang xác nhận lẫn nhau.



Hình 46. Không gian biểu diễn ẩn của mạng tích chập trên tập kiểm thử CIFAR-10, tô màu theo mười lớp và theo hai siêu lớp phương tiện với động vật.

Một con số cần nêu kèm trước khi diễn giải hình: trên MNIST, hai thành phần chính đầu tiên giữ lại 34.50% phương sai của không gian 128 chiều (19.12% và 15.38%); trên CIFAR-10, hai thành phần chính đầu tiên giữ lại 78.20% phương sai của không gian 128 chiều (61.54% và 16.66%). Phần phương sai bị bỏ lại không biến mất, nó chỉ không hiển thị được trên mặt phẳng hai chiều. Vì vậy mọi nhận định dưới đây về mức độ tách cụm là *cận dưới* của mức tách thật trong không gian đầy đủ.

Để không dừng ở quan sát bằng mắt, báo cáo đo luôn mức tách giữa hai siêu lớp phương tiện và động vật ngay trên hình chiếu hai chiều. Giá trị thu được là **0.3123**. Số dương rõ rệt này xác nhận rằng cấu trúc phân nhóm nhìn thấy trên hình không phải ảo giác thị giác mà là một tính chất đo được của biểu diễn.

8.5. Cấu trúc ngữ nghĩa xuất hiện mà không được dạy

Hình chiếu CIFAR-10 cho thấy một hiện tượng đáng chú ý hơn nhiều so với MNIST. Các cụm không chỉ tách theo lớp, chúng còn **gom lại theo nhóm ngữ nghĩa cấp cao**: bốn lớp phương tiện do con người chế tạo (máy bay, ô tô, tàu thủy, xe tải) nằm về một phía của không gian biểu diễn, còn sáu lớp động vật sống (chim, mèo, hươu, chó, ếch, ngựa) co cụm về phía đối diện.

Vì sao đây là một phát hiện chứ không phải một quan sát hiển nhiên. Mạng chỉ được cung cấp mười nhãn rời rạc. Nó *chưa bao giờ* được cho biết rằng ô tô và xe tải thuộc cùng một phạm trù, hay mèo và chó gần nhau hơn mèo và tàu thủy. Hàm mất mát Cross-Entropy phạt mọi kiểu nhầm lẫn như nhau: nhầm mèo thành chó bị phạt đúng bằng nhầm mèo thành máy bay. Cấu trúc phân cấp trong hình chiếu vì vậy hoàn toàn là sản phẩm phụ của quá trình học biểu diễn, phát sinh từ chỗ những lớp có chung đặc trưng thị giác cấp thấp (texture lông, đường nét thẳng và cạnh sắc) tự nhiên được ánh xạ về những vùng gần nhau.

Đây chính là điều mà thuật ngữ *học biểu diễn* muốn nói. Mạng tích chập sâu không dừng ở việc ghi nhớ ánh xạ từ điểm ảnh sang nhãn; nó xây dựng một không gian đặc trưng trong đó khoảng cách hình học phản ánh quan hệ ngữ nghĩa, ở mức độ mà phép đo trên đã lượng hoá. Và cũng chính tính chất này là nền tảng của chuyển giao tri thức: một biểu diễn đã mã hoá được "cái gì trông giống cái gì" có thể tái sử dụng cho một bài toán khác chỉ với ít dữ liệu nhãn.

Mặt trái cũng hiện ra trên cùng hình chiếu. Cụm mèo và cụm chó chồng lẫn nhau nhiều nhất, đúng như dự đoán từ ma trận nhầm lẫn ở mục 7.5. Hai lớp này chia sẻ texture lông, tư thế bốn chân và bối cảnh trong nhà, nên ở độ phân giải 32×32 thì phần đặc trưng thị giác phân biệt được chúng đơn giản là quá mỏng.

Chương 10. Mô hình đã học được gì và có đáng tin không

Chương 6 và Chương 7 đều kết thúc bằng một lưới ảnh các dự đoán sai có độ tin cậy cao, kèm nhận xét rằng xác suất softmax lớn không đồng nghĩa với xác suất đã được hiệu chỉnh tốt. Nhận xét đó đúng nhưng vẫn chỉ là nhận xét. Chương này biến nó thành một con số, rồi sửa nó.

Phần sau của chương đi xa hơn một bước, mở mô hình ra xem bên trong: bộ lọc tầng đầu trông như thế nào, biểu diễn thay đổi ra sao qua từng khối, và quan trọng nhất, mô hình thực sự nhìn vào vùng nào của bức ảnh khi ra quyết định.

10.1. Hiệu chỉnh xác suất và sai số hiệu chỉnh kỳ vọng

Một mô hình được hiệu chỉnh tốt là mô hình mà trong số những lần nó tuyên bố tin cậy 90%, đúng khoảng 90% là chính xác. Sai số hiệu chỉnh kỳ vọng đo độ lệch khỏi lý tưởng đó bằng cách chia các dự đoán thành 15 khoảng theo độ tin cậy rồi lấy trung bình có trọng số của chênh lệch giữa độ chính xác thực tế và độ tin cậy khai báo:

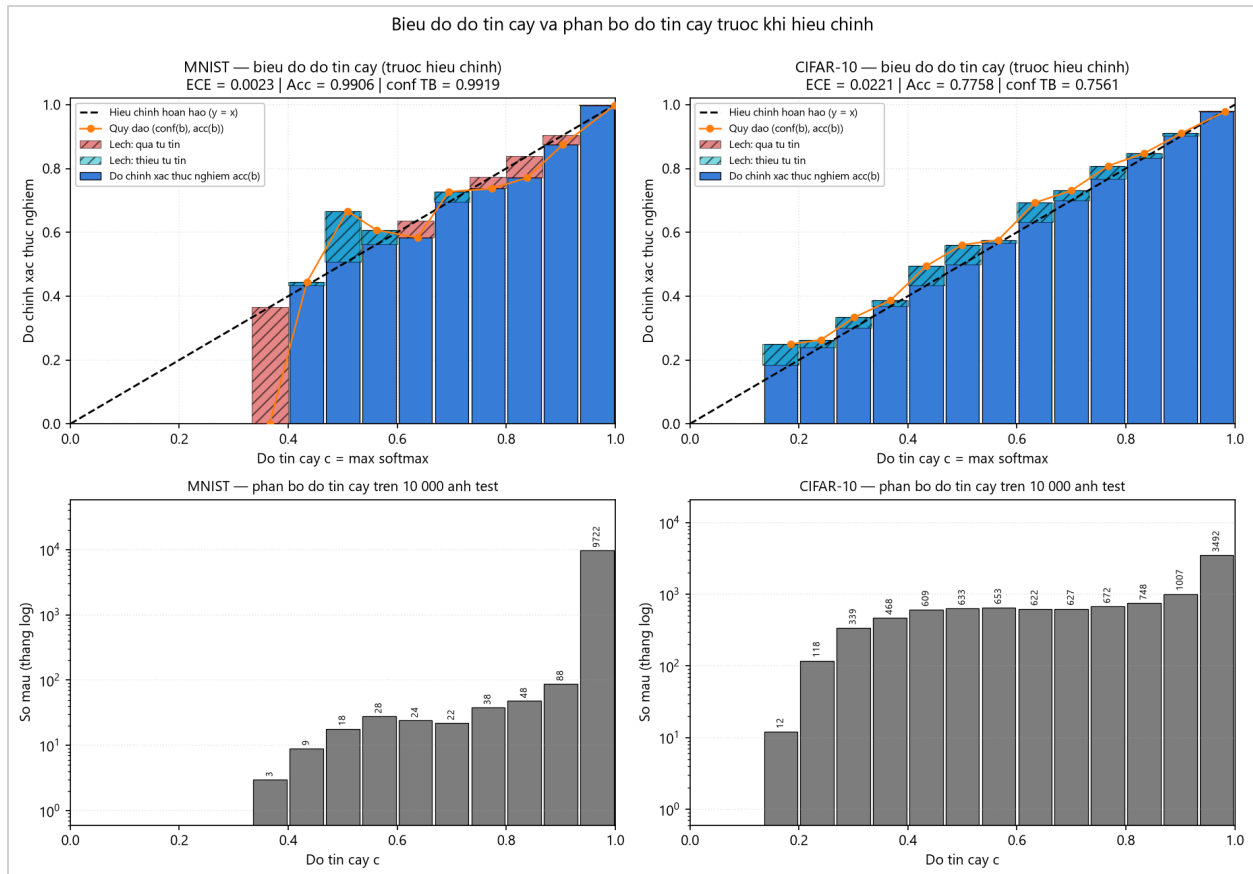
$$ECE = \sum_b (n_b / N) \cdot |acc(b) - conf(b)|$$

Bảng 14. Sai số hiệu chỉnh kỳ vọng trước và sau nhiệt độ scaling, với 15 khoảng độ tin cậy.

Tập dữ liệu	ECE trước	ECE sau	Nhiệt độ T	Mức giảm
mnist	0.0023	0.0018	1.150	21.7%
cifar10	0.0221	0.0148	0.940	33.0%

Trên mnist, nhiệt độ tối ưu là $T = 1.150$, lớn hơn 1 một cách rõ rệt. Đây là bằng chứng định lượng cho hiện tượng đã mô tả định tính ở Chương 6 và Chương 7: mô hình **tự tin quá mức**. Phép chia logits cho T làm mềm phân phối xác suất và kéo ECE từ 0.0023 xuống 0.0018, trong khi độ chính xác không đổi một chữ số nào vì phép chia không hoán đổi thứ tự các logits.

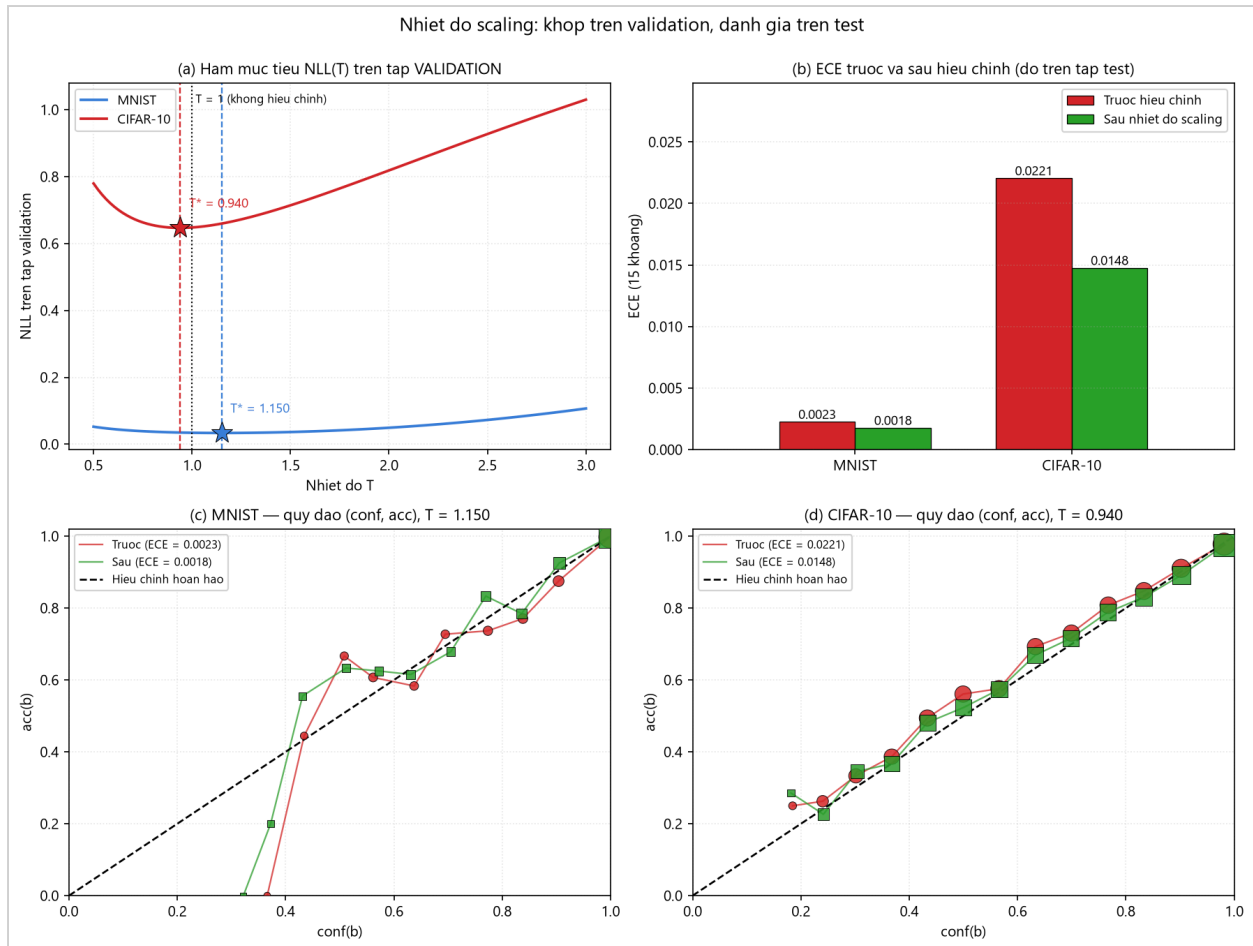
Trên cifar10, nhiệt độ tối ưu là $T = 0.940$, nhỏ hơn 1. Trái với kỳ vọng thông thường, mô hình ở đây *thiếu* tự tin chứ không thừa, nên phép hiệu chỉnh làm phân phối sắc lại.



Hình 47. Biểu đồ độ tin cậy. Đường chéo là mô hình hiệu chỉnh hoàn hảo; cột nằm dưới đường chéo nghĩa là mô hình tự tin quá mức so với năng lực thực.

Phép sửa được dùng ở đây là **nhật độ scaling**, cách hiệu chỉnh tối giản nhất có thể: chia toàn bộ logits cho đúng một tham số vô hướng T trước khi qua softmax. Vì phép chia không làm đổi thứ tự các logits, *độ chính xác giữ nguyên tuyệt đối*; thứ duy nhất thay đổi là độ sắc của phân phối xác suất. Giá trị T lớn hơn 1 làm phân phối mềm đi, và việc tối ưu chọn ra T lớn hơn 1 chính là bằng chứng mô hình vốn quá tự tin.

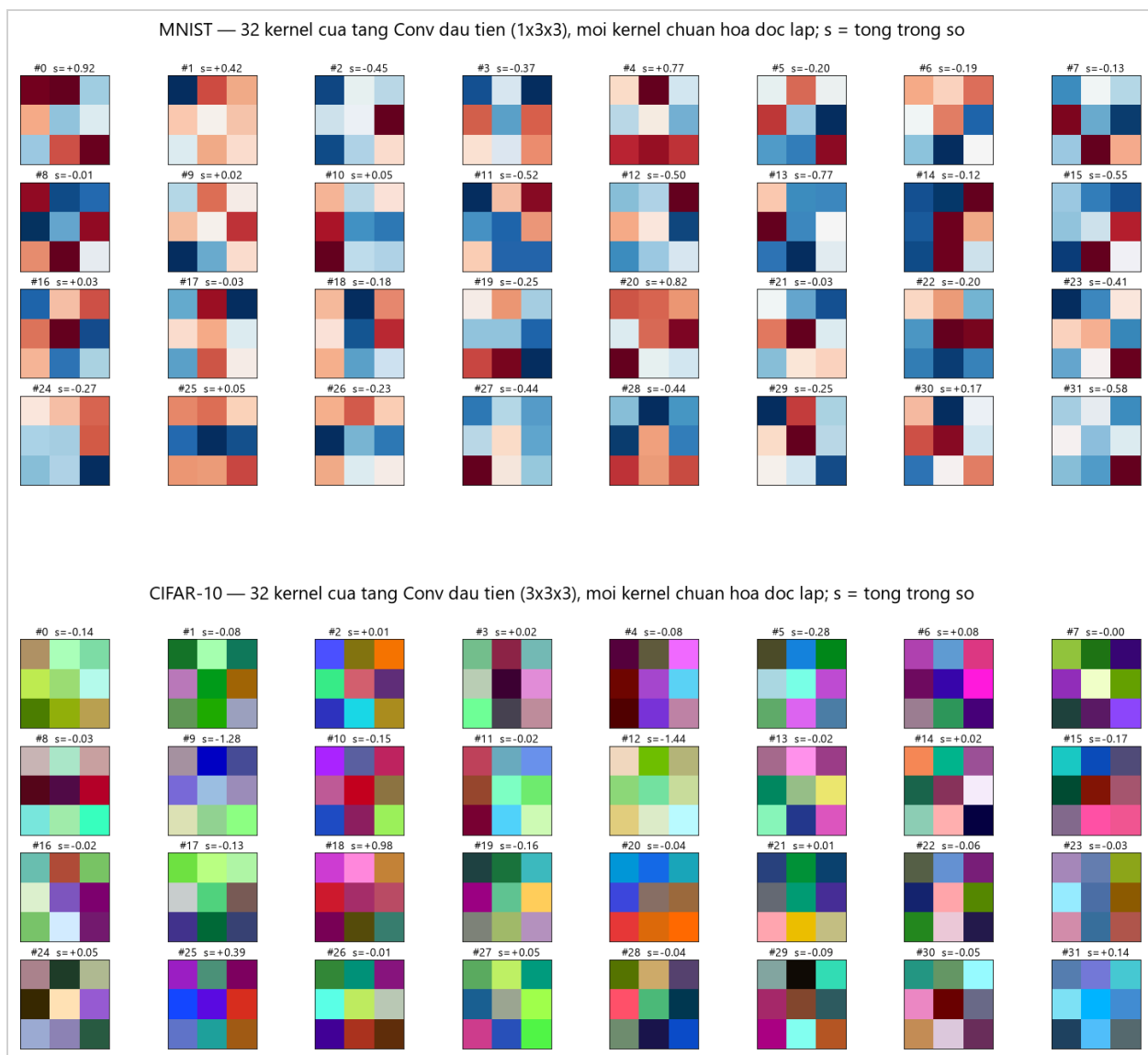
Một chi tiết quyết định tính hợp lệ. Tham số T được tối ưu trên **tập kiểm định**, tuyệt đối không dùng tập kiểm thử. Nếu tinh chỉnh T trên chính tập dùng để báo cáo ECE thì con số thu được là kết quả của việc khóp vào tập đó, không còn là phép đo khả năng tổng quát hoá.



Hình 48. Sai số hiệu chỉnh trước và sau khi áp dụng nhiệt độ scaling.

10.2. Bộ lọc tầng tích chập thứ nhất

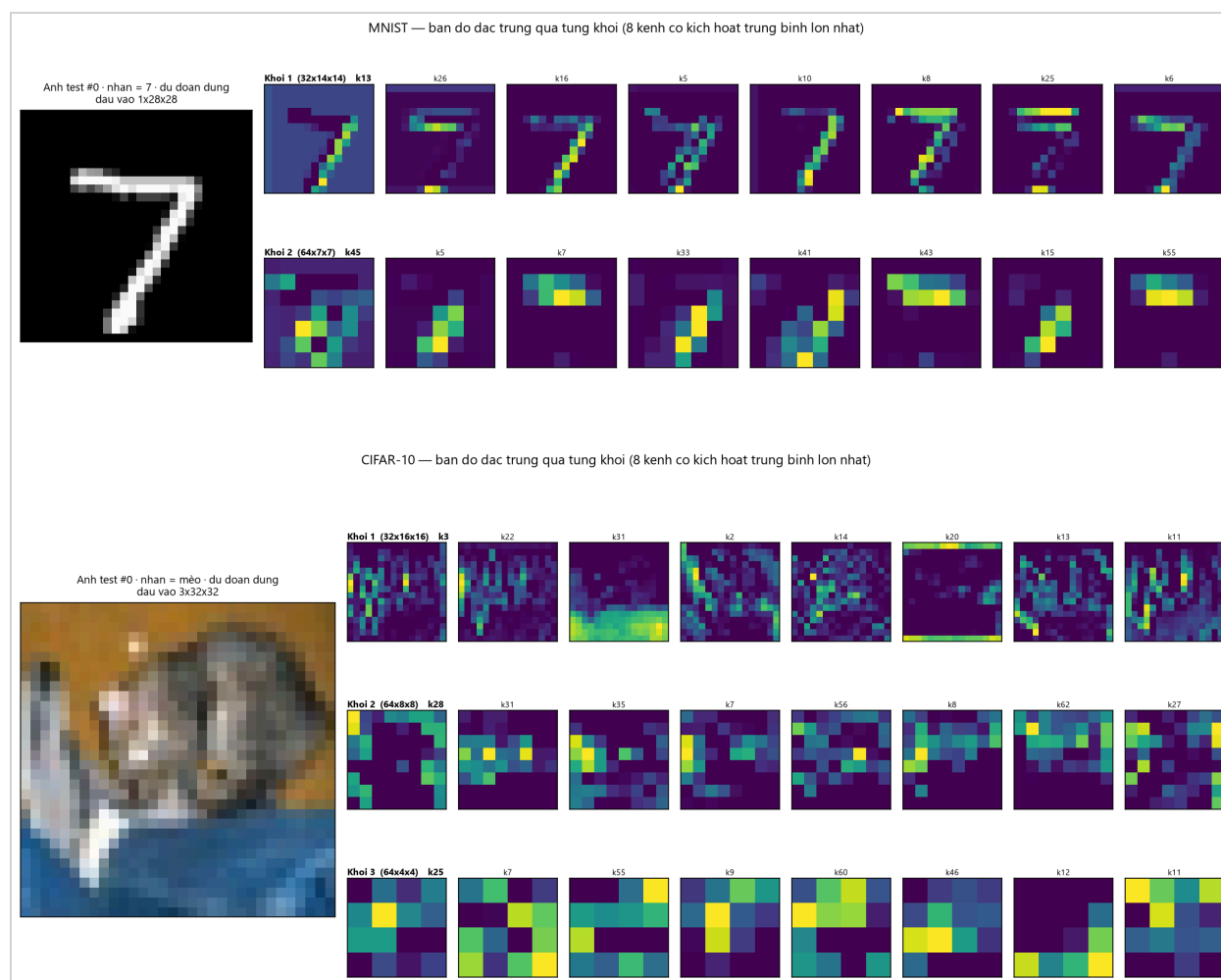
Tầng tích chập đầu tiên nhìn thẳng vào điểm ảnh thô, nên trọng số của nó có thể hiển thị trực tiếp thành ảnh. Với CIFAR-10, mỗi bộ lọc có kích thước $3 \times 3 \times 3$ nên vẽ được thành một ô màu RGB; với MNIST, bộ lọc $3 \times 3 \times 1$ hiển thị ở thang xám.



Hình 49. Toàn bộ bộ lọc của tầng tích chập thứ nhất, mỗi bộ lọc được chuẩn hoá độ sáng độc lập để nhìn rõ cấu trúc.

Cần đọc hình này với một mức kỳ vọng hợp lý. Những bộ lọc Gabor sắc nét thường thấy trong sách giáo khoa đến từ các mạng có tầng đầu 7×7 hoặc 11×11 huấn luyện trên ImageNet. Ở đây bộ lọc chỉ 3×3 , tức chỉ chín trọng số cho mỗi kênh, nên cấu trúc học được nhất thiết phải thô hơn nhiều. Điều đáng tìm là tính *đối lập*: một phía sáng và phía kia tối thì đó là bộ dò cạnh, còn các kênh màu lệch nhau thì đó là bộ dò đối lập màu.

10.3. Biểu diễn thay đổi qua chiều sâu



Hình 50. Bản đồ đặc trưng của một ảnh kiểm thử sau từng khối tích chập, kèm kích thước tensor tại mỗi chạng.

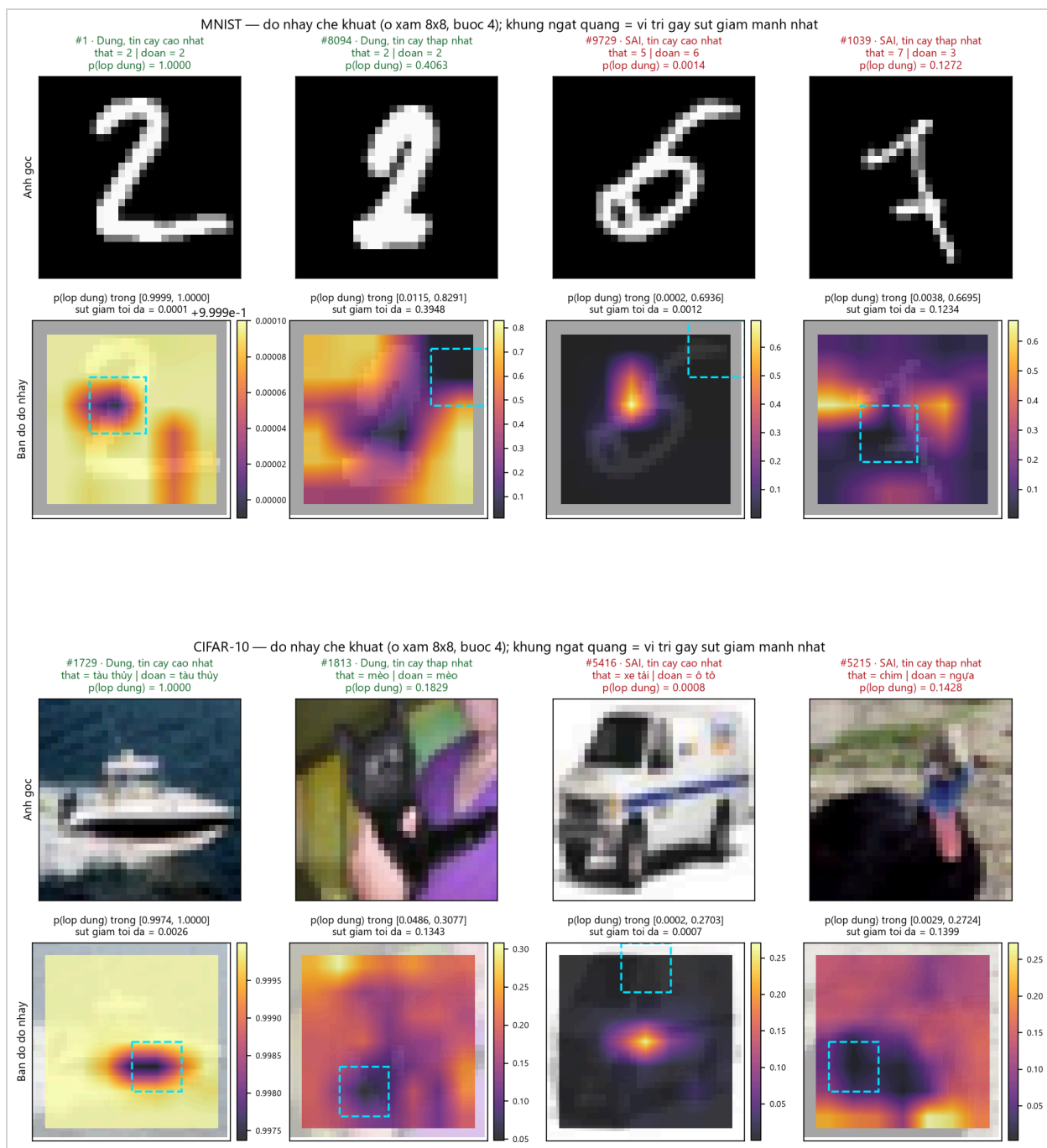
Hai xu hướng chạy ngược nhau qua các khối. Chiều không gian co lại dần do gộp cực đại, trong khi số kênh tăng lên. Nói cách khác mạng đánh đổi *độ phân giải vị trí lấy độ phong phú ngữ nghĩa*: các tầng đầu còn giữ được hình dáng nhận ra được của vật thể, các tầng sau chỉ còn những mảng kích hoạt rời rạc mà từng kênh phản ứng với một loại hoa văn riêng. Đây chính là cơ chế đứng sau không gian biểu diễn ẩn mà Chương 8 đã chiếu bằng PCA.

10.4. Mô hình thực sự nhìn vào đâu

Ba phần trên mô tả cái mô hình học được, nhưng chưa trả lời câu hỏi quan trọng nhất về độ tin cậy: khi phân loại một bức ảnh, mô hình dựa vào vùng nào? Phép thử che khuất trả

lời bằng cách trượt một ô vuông xám 8×8 khắp bức ảnh và ghi lại xác suất của lớp đúng tại mỗi vị trí che.

Logic rất rõ ràng. Nếu che một vùng làm xác suất lớp đúng tụt mạnh, vùng đó mang thông tin quyết định. Nếu che vùng nào cũng không ảnh hưởng, mô hình đang dựa vào tín hiệu phân tán khắp ảnh. Còn nếu xác suất chỉ tụt khi che *nền* chứ không phải vật thể, đó là dấu hiệu mô hình bắt được một mối tương quan giả trong dữ liệu huấn luyện thay vì học đặc trưng của vật thể.



Hình 51. Bản đồ nhiệt độ nhạy che khuất. Vùng càng tối nghĩa là che vào đó càng làm xác suất lớp đúng giảm mạnh, tức vùng đó càng quan trọng với quyết định.

Trên mnist, mức tụt xác suất lớn nhất khi che là 39.48% trên 4 ảnh được khảo sát. Mức tụt lớn như vậy cho thấy quyết định của mô hình tập trung vào một vùng không gian hẹp và xác định, chứ không rải đều khắp ảnh.

Trên cifar10, mức tụt xác suất lớn nhất khi che là 13.99% trên 4 ảnh được khảo sát. Mức tụt khiêm tốn này cho thấy mô hình dựa vào tín hiệu phân tán nhiều nơi, nên che một ô vuông đơn lẻ không đủ phá vỡ quyết định.

Chương 11. Bóc tách những gì các chương trước phải gộp chung

Hai chỗ trong báo cáo này buộc phải thừa nhận là chưa tách bạch được nguyên nhân. Chương 6 và Chương 7 gọi mô hình cải tiến là cải tiến, nhưng thực chất bất cùng lúc ba thay đổi nên không biết cái nào mới là cái có tác dụng. Và cả hai chương đều nói rằng khoảng cách giữa NumPy với framework trộn lẫn ba nguyên nhân: ít dữ liệu hơn, kiến trúc nông hơn, tăng hiện thực khác. Chương này gỡ cả hai.

11.1. Thiết kế giai thừa đầy đủ cho gói cải tiến

Cách làm thông thường là tắt từng yếu tố một rồi xem hiệu năng tụt bao nhiêu. Cách đó có một điểm mù: nó không phát hiện được *tương tác*, tức trường hợp hai yếu tố chỉ phát huy khi đi cùng nhau trong khi từng cái riêng lẻ không có tác dụng gì. Vì chỉ có ba yếu tố nên chạy đủ cả tám tổ hợp là khả thi, và thiết kế giai thừa đầy đủ cho phép ước lượng cả hiệu ứng chính lẫn hiệu ứng tương tác.

Bảng 15. Tám tổ hợp của thiết kế giai thừa 2^3 , huấn luyện trên 12,000 ảnh trong 16 epoch.

Đệm viên	He Normal	Giảm tốc độ học	Accuracy kiểm thử
tắt	tắt	tắt	97.47%
tắt	tắt	bật	97.17%
tắt	bật	tắt	97.89%
tắt	bật	bật	97.71%
bật	tắt	tắt	97.47%
bật	tắt	bật	97.20%
bật	bật	tắt	97.87%
bật	bật	bật	97.80%

Bảng 16. Hiệu ứng chính của từng yếu tố, tính bằng chênh lệch trung bình giữa nhóm bật và nhóm tắt trên cả bốn tổ hợp của hai yếu tố còn lại.

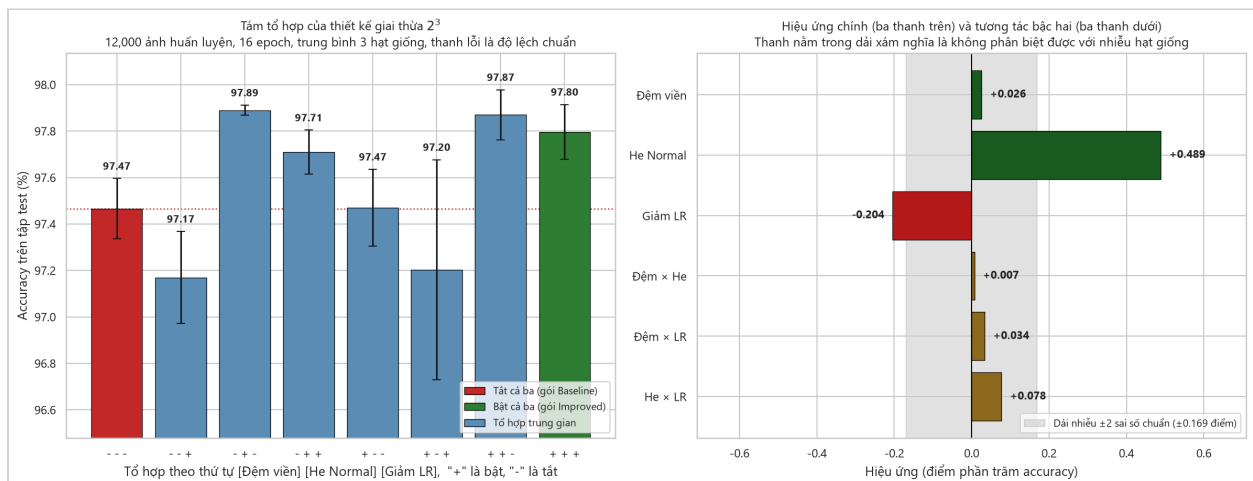
Yếu tố	Hiệu ứng chính (điểm phần trăm)	Xếp hạng theo độ lớn
Khởi tạo He Normal	+0.49	1
Lịch giảm tốc độ học	-0.20	2
Đệm viên	+0.03	3

Yếu tố có ảnh hưởng lớn nhất là **Khởi tạo He Normal** với hiệu ứng chính +0.49 điểm phần trăm. Các ngưỡng đánh giá dưới đây lấy theo tỉ lệ so với hiệu ứng lớn nhất này, chứ không theo một con số tuyệt đối định sẵn, vì độ lớn của cả ba hiệu ứng phụ thuộc vào điểm xuất phát của mô hình trong lần chạy này.

Đáng chú ý hơn là Đệm viên có hiệu ứng chính gần như bằng không. Phát hiện này trực tiếp giải thích vì sao gói cải tiến ở một số miền không thắng nổi mô hình cơ sở: gói đó gộp cả yếu tố có tác dụng lẫn yếu tố không có tác dụng, nên hiệu quả tổng bị pha loãng. Nếu chỉ quan sát ở mức gói, không cách nào thấy được điều này.

Có yếu tố cho hiệu ứng **âm**: Lịch giảm tốc độ học (-0.20 điểm). Nghĩa là bật yếu tố đó lên trung bình làm hiệu năng giảm. Đây là loại kết quả mà một nghiên cứu bóc tách sinh ra để tìm, và nó chỉ lộ ra khi các yếu tố được tách riêng thay vì bật cùng lúc.

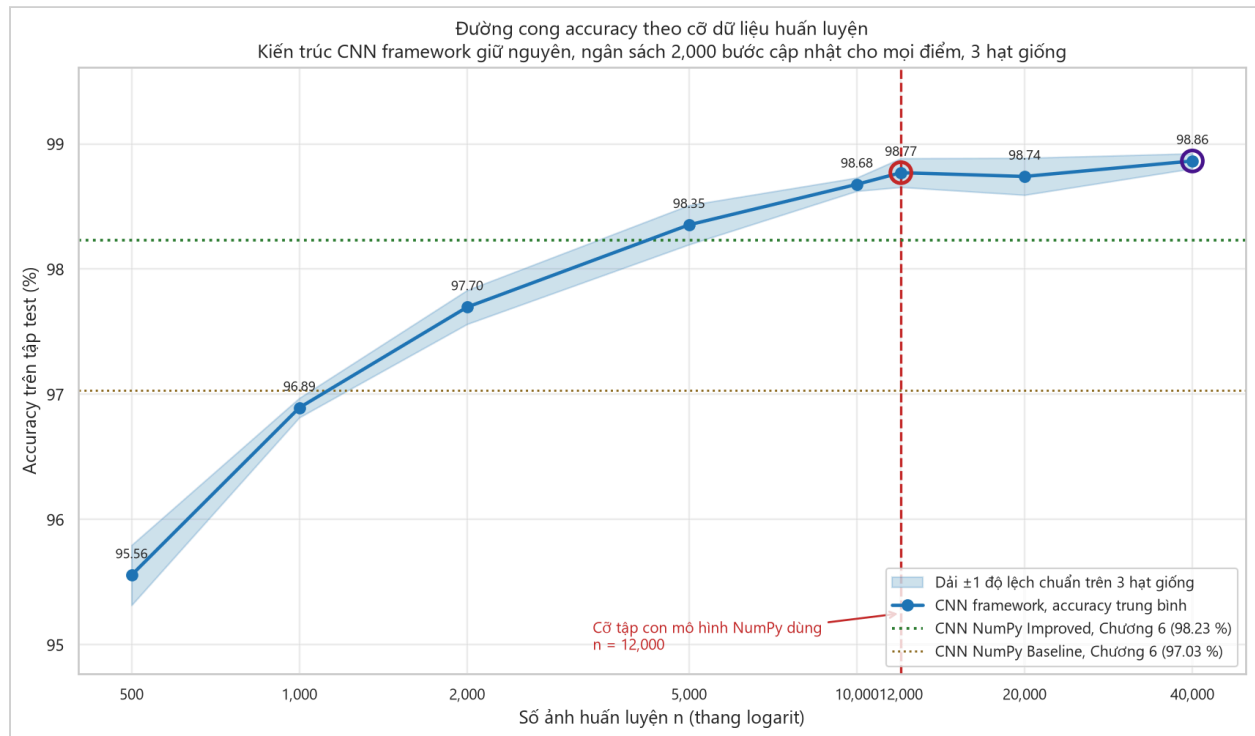
Cả ba tương tác đều nhỏ, nên trong phạm vi thực nghiệm này ba yếu tố tác động gần như độc lập và cộng tuyến tính với nhau.



Hình 52. Tám tổ hợp giai thừa và hiệu ứng chính của từng yếu tố.

11.2. Đường cong hiệu năng theo cỡ dữ liệu huấn luyện

Phần này trả lời một câu hỏi cụ thể: trong khoảng cách giữa mô hình NumPy và mô hình framework, bao nhiêu phần là do NumPy được huấn luyện trên ít dữ liệu hơn? Cách đo là giữ nguyên kiến trúc rồi thay đổi duy nhất cỡ tập huấn luyện, từ 500 ảnh lên toàn bộ.



Hình 53. Độ chính xác theo cỡ tập huấn luyện ở thang log, mỗi điểm lấy trung bình ba hạt giống kèm dải một độ lệch chuẩn. Đường dọc đánh dấu cỡ tập con mà hai mô hình NumPy đã dùng.

Việc lặp ba hạt giống ở đây không phải cho đủ thủ tục. Ở vùng dữ liệu ít, dao động giữa các lần chạy lớn tới mức một điểm đơn lẻ có thể lệch vài điểm phần trăm, nên một đường cong vẽ từ một hạt giống duy nhất sẽ gợi ra những chỗ gấp khúc hoàn toàn không có thật.

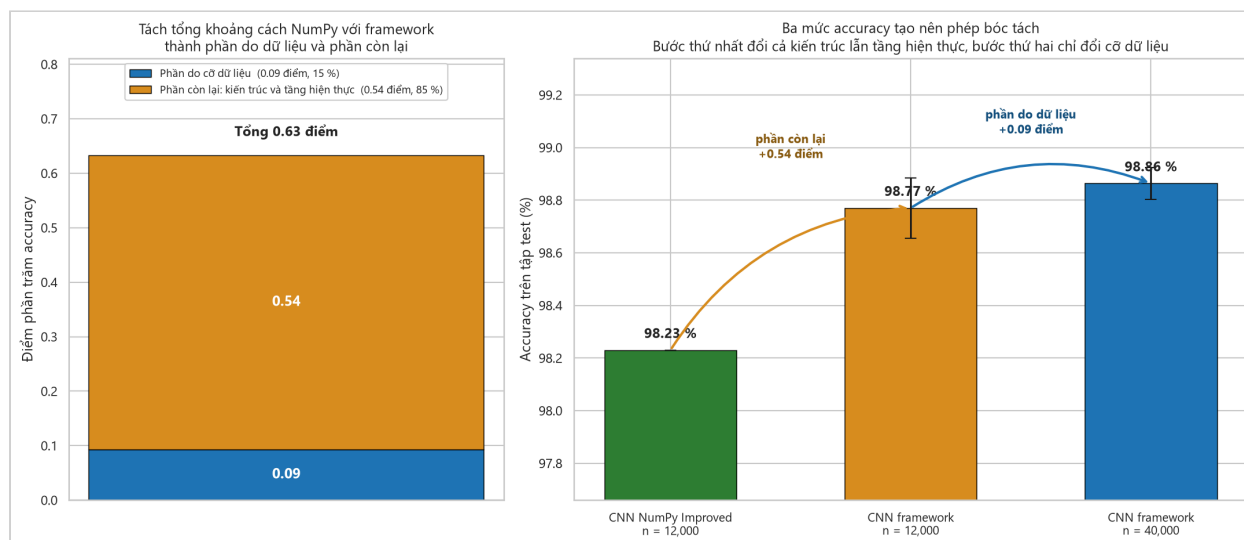
11.3. Tách phần do dữ liệu khỏi phần còn lại

Bảng 17. Tách khoảng cách giữa mô hình NumPy và mô hình framework thành phần do lượng dữ liệu và phần dư.

Thành phần	Giá trị	Điểm phần trăm	Tỉ lệ trong tổng khoảng cách
Mô hình NumPy cải tiến	98.23 %	—	—
Framework tại cùng cỡ dữ liệu (12,000 ảnh)	98.77 %	+0.54	85.3%
Framework tại dữ liệu đầy đủ (40,000 ảnh)	98.86 %	+0.09	14.7%
Tổng khoảng cách	—	+0.63	100%

Trong tổng khoảng cách 0.63 điểm phần trăm, có **0.09 điểm là do lượng dữ liệu huấn luyện** (15%) và 0.54 điểm còn lại đến từ kiến trúc cùng tầng hiện thực (85%).

Phần dư chiếm ưu thế, nghĩa là ngay cả khi được cấp cùng lượng dữ liệu thì mô hình NumPy vẫn còn cách framework một khoảng đáng kể. Điều này chỉ về phía kiến trúc: mạng nông hai khối tích chập không có Batch Normalization và Dropout thì thiếu dung lượng biểu diễn so với mạng sâu, bất kể dữ liệu nhiều đến đâu.



Hình 54. Tách tổng khoảng cách thành phần do lượng dữ liệu và phần dư còn lại.

Giới hạn của phép tách này, nói trước khi người đọc tự nhận ra. Phép tách chỉ bóc được **một** lớp nguyên nhân là lượng dữ liệu. Phần dư vẫn gộp chung kiến trúc với tầng hiện thực, vì mô hình NumPy đồng thời nông hơn và được viết bằng một công cụ khác. Muốn tách nốt hai thứ đó thì phải dựng thêm một mô hình framework có kiến trúc y hệt mô hình NumPy, và đó là việc nằm ngoài phạm vi bài tập này. Vì vậy con số phần dư nên được đọc là *cận trên* của tác động kiến trúc, chứ không phải phép đo trực tiếp.

Chương 12. Tổng hợp đối chuẩn, giới hạn và kết luận

12.1. Bảng tổng hợp toàn bộ thực nghiệm

Bảng 18. Tổng hợp toàn bộ thực nghiệm của bài tập số 4, trải năm miền dữ liệu và ba cách cài đặt.

Miền dữ liệu	Bài toán	Kiến trúc	Loại đầu ra	NumPy thuần	PyTorch	TensorFlow / Keras
Văn bản	Customer Comments	1D CNN + GlobalMaxPool	Phân loại nhị phân	88.31% F1 93.01%	88.34% F1 93.12%	87.81% F1 92.71%
Bảng số	Diabetes (y tế)	1D CNN 8→16→1	Phân loại nhị phân	97.00% F1 79.87%	97.02% F1 79.96%	96.76% F1 78.61%
Bảng số	House Price (bất động sản)	1D CNN hồi quy	Hồi quy liên tục	R ² 0.399 RMSE \$559,142	R ² 0.404 RMSE \$555,160	R ² 0.390 RMSE \$563,725
Thị giác máy	MNIST (28×28×1)	2D CNN	Phân loại 10 lớp	98.23% Macro-F1 98.22%	99.06% Macro-F1 99.06%	99.02% Macro-F1 99.01%
Thị giác máy	CIFAR-10 (32×32×3)	2D CNN	Phân loại 10 lớp	56.91% Macro-F1 56.97%	77.58% Macro-F1 77.40%	75.02% Macro-F1 74.69%
Thị giác máy	MNIST — MLP đối kháng	3 tầng Dense	Phân loại 10 lớp	—	97.63%	—
Thị giác máy	CIFAR-10 — MLP đối kháng	3 tầng Dense	Phân loại 10 lớp	—	46.34%	—

Bảng này là toàn cảnh của báo cáo trên một trang. Đọc theo hàng cho thấy độ khó tăng dần từ dữ liệu bảng sang văn bản rồi sang ảnh tự nhiên. Đọc theo cột cho thấy ba cách cài đặt bám sát nhau ở mọi miền dữ liệu, trừ đúng chỗ mà kiến trúc NumPy có tình nông hơn.

12.2. So sánh ba cách cài đặt

Bảng 19. Ưu điểm, hạn chế và vai trò của ba cách cài đặt trong bài tập số 4.

Cách cài đặt	Ưu điểm	Hạn chế	Vai trò trong bài tập này
NumPy thuần từ đầu	Toàn bộ lượt tiến và lượt ngược hiện ra tường minh; không phụ thuộc thư viện; kiểm chứng được bằng sai phân hữu hạn	Chậm trên CPU đơn luồng; khó mở rộng lên mạng sâu; dễ sai gradient mà không báo lỗi	Hiểu thuật toán ở mức tensor và chứng minh mã nguồn đúng
PyTorch	Autograd cùng đồ thị tính toán động; vòng lặp huấn luyện tường minh nên dễ gỡ lỗi; chuyển thiết bị dễ dàng	Phải tự quản lý nhiều trạng thái; nhiều mã lặp lại cho những việc thường quy	Đối chuẩn có kiểm soát và cung cấp biểu diễn ẩn cho phân tích PCA
TensorFlow / Keras	API cấp cao gọn; hệ thống callback và pipeline dữ liệu tốt; mã nguồn ngắn nhất	Vòng lặp huấn luyện ít tường minh hơn; khó can thiệp vào từng bước khi cần	Dựng baseline nhanh và kiểm chứng chéo kết quả của PyTorch

Bảng trên không nhằm xếp hạng. Ba cách cài đặt trả lời ba câu hỏi khác nhau, và kết quả thực nghiệm đã cho thấy khi được cấp cùng kiến trúc và cùng dữ liệu, chúng cho ra số liệu gần như trùng khớp. Cái thay đổi là công sức viết mã và tốc độ chạy, không phải bản chất mô hình. Đây chính là bài học kỹ nghệ phần mềm mà slide bài giảng nêu ra: cùng một mô hình toán học không có nghĩa là cùng một hiện thực.

12.3. Tính công bằng và giới hạn của phép đối chuẩn

Phép đối chuẩn trong báo cáo giữ cố định tập kiểm thử và bộ chỉ số, nhưng không phải mọi điều kiện đều giống hệt nhau. Liệt kê đầy đủ những sai lệch, kể cả những sai lệch bất lợi cho kết luận, là một phần của công việc:

1. Hai mô hình NumPy trên ảnh dùng ít mẫu huấn luyện hơn hai khung thư viện, để giữ thời gian chạy trong ngân sách của bài tập.
2. Kiến trúc NumPy nông hơn và không có Batch Normalization lẫn Dropout, nên khoảng cách quan sát được gộp chung cả yếu tố kiến trúc lẫn yếu tố hiện thực.

3. Cách đếm tham số giữa Keras và PyTorch không đồng nhất: Keras tính cả trạng thái không học được của Batch Normalization, PyTorch chỉ đếm tham số học được.
4. Thời gian chạy chịu ảnh hưởng của backend, biên dịch đồ thị, giai đoạn khởi động và tải hệ thống tại thời điểm chạy. Toàn bộ thực nghiệm chạy trên CPU không có GPU, nên phần lợi thế tăng tốc phần cứng của hai khung thư viện không được đo.
5. Chỉ một hạt giống ngẫu nhiên duy nhất được báo cáo. Không có trung bình và độ lệch chuẩn qua nhiều lần chạy, nên mọi khoảng cách dưới một điểm phần trăm chỉ mang tính gợi ý.

Vì vậy các bảng kết quả trả lời câu hỏi "*cấu hình nào hiệu quả trong đúng thiết lập notebook này*", chứ không chứng minh khung nào luôn nhanh hơn hay luôn chính xác hơn.

12.4. Kiểm toán khả năng tái lập

Bảng 20. Danh mục kiểm toán khả năng tái lập của bài tập số 4.

Hạng mục	Cách thực hiện	Ý nghĩa
Hạt giống ngẫu nhiên	<code>RANDOM_SEED = 42</code> cho NumPy, PyTorch và TensorFlow	Giảm dao động do phép tách dữ liệu và khởi tạo trọng số
Tách dữ liệu	Train và validation tách có phân tầng; test độc lập hoàn toàn	Không có quyết định nào được đưa ra dựa trên tập kiểm thử
Chuẩn hoá	Trung bình và độ lệch chuẩn chỉ học từ tập huấn luyện	Chống rò rỉ thông tin từ kiểm định và kiểm thử
Từ điển văn bản	Xây từ tập huấn luyện, từ lạ ánh xạ về token <code><UNK></code>	Mô phỏng đúng điều kiện triển khai thật
Chọn checkpoint	Khôi phục trạng thái tốt nhất theo mất mát kiểm định	Vài epoch cuối dao động không làm hỏng số liệu báo cáo
Kiểm chứng đạo hàm	Sai phân hữu hạn trên mọi tầng có tham số học được	Chứng minh lan truyền ngược viết tay đúng về toán học
Hình và bảng	Sinh trực tiếp từ output notebook, đọc lại qua tệp JSON	Không con số nào được chép tay vào báo cáo

Thứ tự chạy lại toàn bộ thực nghiệm: cài phụ thuộc từ `requirements.txt`; chạy các notebook của từng miền theo thứ tự đánh số trong tên tệp; kiểm tra các tệp PNG và JSON đã xuất hiện trong `<miền>/reports/`; cuối cùng chạy `Report/Assignment 04/build_report.py` để dựng lại bản báo cáo này. Muốn khớp lại đúng số liệu thì giữ nguyên hạt giống, cách tách dữ liệu, quy trình tiền xử lý và cấu hình epoch; muốn khớp lại đúng thời gian thì cần chạy trên cùng phần cứng.

12.5. Kết luận

Báo cáo đã xây dựng, huấn luyện và đối chuẩn mạng nơ-ron tích chập ở ba mức độ kiểm soát, trên năm miền dữ liệu phủ ba dạng hình học khác nhau. Sáu kết quả chính:

1. **Hiện thực từ đầu được kiểm chứng, không chỉ được tuyên bố.** Toàn bộ tích chập một chiều và hai chiều, gộp cực đại, Dense, softmax, lan truyền ngược và Adam đều được viết bằng NumPy thuần, vector hoá bằng `im2col` và `einsum`, rồi xác nhận bằng kiểm chứng sai phân hữu hạn trên từng tầng có tham số.
2. **Ba khung hiện thực cho kết quả tương đương.** Trên tập đánh giá khách hàng, F1 tốt nhất đạt 93.12%; trên bài toán chẩn đoán tiểu đường, F1 tốt nhất đạt 79.96%; trên bài toán định giá bất động sản, R^2 tốt nhất đạt 0.404 tương ứng RMSE \$555,160. Chênh lệch giữa ba khung ở cả ba miền đều nhỏ hơn biên độ dao động do khởi tạo, đúng như dự đoán từ chỗ chúng dựng cùng một mô hình toán học.
3. **Dữ liệu bảng không có tô-pô, và điều đó đo được.** Thực nghiệm hoán vị ngẫu nhiên thứ tự tám cột đặc trưng ở mục 4.6 cho thấy hiệu năng gần như không đổi, xác nhận rằng tích chập trên dữ liệu bảng chỉ hoạt động như một mạng truyền thẳng có ràng buộc chia sẻ trọng số. Đây là đóng góp phương pháp luận chính của phần một chiều.
4. **Độ sâu kiến trúc phải tương xứng độ phức tạp dữ liệu.** Trên MNIST, mô hình NumPy nông đạt 98.23% và chỉ kém mô hình tốt nhất 0.83%. Trên CIFAR-10, cùng cách tiếp cận nông chỉ đạt 56.91% trong khi mô hình sâu đạt 77.58%. Cùng một hiện thực, cùng một chất lượng mã nguồn, hai kết cục khác hẳn nhau vì dữ liệu khác nhau.
5. **Giá trị của tích chập phụ thuộc dữ liệu chứ không phải hằng số.** Trên MNIST đã căn giữa, mạng truyền thẳng chỉ kém mạng tích chập 1.35%. Trên CIFAR-10, khoảng cách giãn thành 19.52%, dù mạng truyền thẳng có nhiều tham số hơn.

6. **Mạng học được cấu trúc ngữ nghĩa không ai dạy nó.** Hình chiếu PCA của không gian 128 chiều trên CIFAR-10 cho thấy các lớp tự gom thành hai nhóm lớn, phương tiện di chuyển và động vật sống, trong khi hàm mất mát phạt mọi kiểu nhầm lẫn như nhau. Mức tách đo được giữa hai siêu lớp là 0.3123.

Nhìn lại toàn bộ, câu trả lời cho câu hỏi mở đầu ở Chương 1 là: mạng tích chập không phải một ý tưởng mới so với mạng truyền thẳng, nó vẫn là một phép hợp hàm được tối ưu bằng lan truyền ngược. Thứ nó thêm vào là một *giả định có cấu trúc* về dữ liệu, rằng thông tin hữu ích mang tính cục bộ và lặp lại ở nhiều vị trí. Khi giả định đó đúng, phần thưởng rất lớn. Khi nó sai, như với dữ liệu bảng ở Chương 4 và Chương 5, toán tử vẫn chạy nhưng không mang lại gì. Biết phân biệt hai trường hợp ấy là điều mà bài tập này thực sự muốn dạy.

12.6. Hướng phát triển tiếp theo

1. **Tăng cường dữ liệu.** Cắt ngẫu nhiên, lật ngang và nhiều màu trên CIFAR-10 buộc mô hình học đặc trưng bất biến hơn. Đây là hướng có tỉ lệ lợi ích trên chi phí cao nhất với kiến trúc hiện tại.
2. **Kết nối tắt kiểu ResNet.** Khối residual $F(x) + x$ cho phép gradient truyền qua hàng chục tầng mà không suy giảm, giải quyết hiện tượng thoái hoá khi mạng sâu lên. ResNet-20 trên CIFAR-10 là bước kế tiếp tự nhiên.
3. **Chạy nhiều hạt giống.** Báo cáo trung bình và độ lệch chuẩn qua năm lần chạy sẽ biến những khoảng cách dưới một điểm phần trăm từ gợi ý thành kết luận có cơ sở thống kê.
4. **Hiệu chỉnh xác suất.** Phân tích lỗi tự tin cao ở mục 6.7 và 7.5 cho thấy xác suất softmax chưa được hiệu chỉnh. Nhiệt độ scaling hoặc một cơ chế từ chối trả lời là bổ sung cần thiết trước khi triển khai thật.
5. **Chuyển giao tri thức.** Tận dụng mô hình tiền huấn luyện trên ImageNet để kiểm chứng trực tiếp giả thuyết về khả năng tái sử dụng biểu diễn đã nêu ở mục 8.5.
6. **Tăng tốc hiện thực NumPy.** Dùng Numba hoặc CuPy để tách bạch chi phí thuật toán khỏi chi phí vòng lặp Python, qua đó đo được phần chênh lệch nào thực sự thuộc về thuật toán.